

Módulo 1 - Prácticas de Fundamentos de Entity Framework Core

Punto 1.1 - Qué es un ORM y por qué existe Entity Framework Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Inicio del proyecto AceriaData. Se crea la solución, el proyecto de consola y se instalan los primeros paquetes de EF Core con SQL Server.

Ejercicio: Crear la solución AceriaData, el proyecto de consola AceriaData.Console, instalar los paquetes de EF Core con SQL Server y verificar que el proyecto compila y se ejecuta.

Paso 1: Crear el directorio de trabajo

```
mkdir AceriaData
cd AceriaData
```

mkdir AceriaData → crea la carpeta raíz del proyecto. cd AceriaData → entra en esa carpeta para trabajar dentro de ella.

Paso 2: Crear la solución

```
dotnet new sln -n AceriaData
```

dotnet new sln → invoca la plantilla de solución de la CLI de .NET. -n AceriaData → asigna el nombre AceriaData al archivo de solución generado, que será AceriaData.sln.

Error común: si ya existe un archivo .sln con ese nombre en la carpeta, el comando falla. Se debe eliminar el archivo existente o usar otro nombre.

Paso 3: Crear el proyecto de consola

```
dotnet new console -n AceriaData.Console
```

dotnet new console → invoca la plantilla de proyecto de consola. -n AceriaData.Console → asigna el nombre AceriaData.Console al proyecto y a la carpeta que lo contiene.

Error común: si la carpeta AceriaData.Console ya existe, el comando falla. Se debe eliminar la carpeta o usar otro nombre.

Paso 4: Añadir el proyecto a la solución

```
dotnet sln add AceriaData.Console/AceriaData.Console.csproj
```

dotnet sln add → añade un proyecto a la solución. AceriaData.Console/AceriaData.Console.csproj → ruta al archivo de proyecto que se quiere añadir.

Error común: si la ruta al .csproj es incorrecta, el comando falla. Se debe verificar que el proyecto se creó en la carpeta esperada.

Paso 5: Entrar en el proyecto

```
cd AceriaData.Console
```

cd AceriaData.Console → entra en la carpeta del proyecto de consola para ejecutar los comandos de instalación de paquetes dentro de él.

Paso 6: Instalar el proveedor de SQL Server

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.31
```

dotnet add package → añade una referencia a un paquete NuGet al proyecto actual; `--version 8.0.31` fija una versión compatible con EF Core 8. Microsoft.EntityFrameworkCore.SqlServer → nombre del paquete que contiene el proveedor de EF Core para SQL Server y Azure SQL.

Error común: si la versión del paquete no es compatible con .NET 8, se producen errores de compilación. Se debe verificar que la versión instalada sea la 8.x.

Paso 7: Instalar las herramientas de diseño

```
dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.31
```

Microsoft.EntityFrameworkCore.Design → paquete que contiene las herramientas necesarias para generar migraciones y ejecutar comandos de diseño en tiempo de compilación.

Error común: si no se instala este paquete, los comandos dotnet ef migrations y dotnet ef database update no funcionan.

Paso 8: Verificar el archivo .csproj

Abrir el archivo AceriaData.Console.csproj y comprobar que contiene las referencias a los paquetes:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.31" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.31">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
  </ItemGroup>

</Project>
```

Línea 1: → declara el proyecto con el SDK de .NET. Línea 3: → agrupa propiedades de configuración del proyecto. Línea 4: Exe → indica que el proyecto genera un ejecutable. Línea 5: net8.0 → especifica .NET 8 como framework destino. Línea 6: enable → habilita la importación automática de espacios de nombres comunes. Línea 7: enable → habilita los tipos de referencia anulables. Línea 10: → referencia al proveedor de SQL Server. Línea 11: → referencia a las herramientas de diseño. Línea 12: all → indica que este paquete no se propaga a proyectos que referencien este. Línea 13: ... → especifica qué recursos del paquete se incluyen.

Error común: si las versiones de los paquetes no coinciden entre sí, se producen conflictos de dependencias. Se deben usar versiones alineadas.

Paso 9: Restaurar y compilar

```
dotnet build
```

dotnet build → restaura los paquetes NuGet y compila el proyecto. Si la compilación termina sin errores, EF Core está correctamente referenciado.

Error común: si hay errores de restauración, se debe verificar la conexión a internet o la configuración de las fuentes NuGet.

Paso 10: Verificar la versión de EF Core instalada

```
dotnet list package
```

dotnet list package → muestra la lista de paquetes NuGet referenciados por el proyecto con sus versiones.

Resultado esperado: aparecen Microsoft.EntityFrameworkCore.SqlServer y Microsoft.EntityFrameworkCore.Design con versión 8.x.

Paso 11: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto de consola. Como el proyecto aún contiene el código generado por la plantilla, se muestra Hello, World! en la consola.

Resultado esperado: la aplicación se ejecuta sin errores y muestra el mensaje de la plantilla.

Paso 12: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por:

```
Console.WriteLine("=== ACERÍA DEL NORTE ===");  
Console.WriteLine("Sistema de gestión de órdenes de fabricación");  
Console.WriteLine("Proyecto AceriaData inicializado");
```

Línea 1: Console.WriteLine("=== ACERÍA DEL NORTE ==="); → escribe el título de la acería en la consola. Línea 2: Console.WriteLine("Sistema de gestión de órdenes de fabricación"); → escribe la descripción del sistema. Línea 3: Console.WriteLine("Proyecto AceriaData inicializado"); → confirma que el proyecto está listo.

Error común: si se olvida el punto y coma al final de una línea, el compilador muestra un error de sintaxis indicando la línea.

Paso 13: Ejecutar de nuevo

```
dotnet run
```

Resultado esperado: se muestran las tres líneas del mensaje de la acería.

Errores comunes del ejercicio completo

Error	Causa	Solución
dotnet no se reconoce	El SDK no está en el PATH	Reiniciar terminal o reinstalar el SDK
dotnet new sln falla	Ya existe un archivo .sln	Eliminar el archivo existente o usar otro nombre
dotnet add package falla	Sin conexión a NuGet	Verificar conexión o configuración de fuentes
Conflicto de versiones	Paquetes con versiones incompatibles	Alinear todas las versiones a 8.x

Error	Causa	Solución
dotnet ef no se reconoce	Herramientas de EF Core no instaladas	Ejecutar dotnet tool install --global dotnet-ef
Error de compilación por Nullable	Propiedades no inicializadas	Inicializar con = string.Empty o = null!

Retos resueltos: Añadir un mensaje de bienvenida con fecha

Retos: Modificar Program.cs para que muestre el nombre del proyecto, la versión de .NET y la fecha actual.

Solución paso a paso

Paso 1: Modificar Program.cs:

```
Console.WriteLine("=== ACERÍA DEL NORTE ===");
Console.WriteLine("Sistema de gestión de órdenes de fabricación");
Console.WriteLine($"Proyecto: AceriaData");
Console.WriteLine($"Framework:
{System.Runtime.InteropServices.RuntimeInformation.FrameworkDescription}");
Console.WriteLine($"Fecha: {DateTime.Now:dd/MM/yyyy HH:mm}");
```

Línea 1: Console.WriteLine("=== ACERÍA DEL NORTE ==="); → título. Línea 2: Console.WriteLine("Sistema de gestión de órdenes de fabricación"); → descripción. Línea 3: Console.WriteLine(\$"Proyecto: AceriaData"); → muestra el nombre del proyecto usando interpolación de cadenas. Línea 4: Console.WriteLine(\$"Framework: {System.Runtime.InteropServices.RuntimeInformation.FrameworkDescription}"); → obtiene la descripción del framework en tiempo de ejecución y la muestra. Línea 5: Console.WriteLine(\$"Fecha: {DateTime.Now:dd/MM/yyyy HH:mm}"); → muestra la fecha y hora actual con formato.

Paso 2: Ejecutar:

```
dotnet run
```

Paso 3: Verificar que aparecen las cinco líneas con la información correcta.

Analogía final

Este ejercicio es como preparar el terreno y montar la estructura básica de una acería antes de empezar a producir. Se ha creado el espacio de trabajo, se han instalado las herramientas básicas de EF Core con el motor de SQL Server LocalDB, y se ha verificado que todo funciona. A partir de aquí, los siguientes puntos añadirán el horno, el tren de laminación y los sistemas de control, que en el proyecto corresponden al DbContext, las entidades y las consultas.

Resultado esperado

Al final del ejercicio, deberías haber:

Creado la solución AceriaData.

Creado el proyecto de consola AceriaData.Console.

Instalado los paquetes Microsoft.EntityFrameworkCore.SqlServer y Microsoft.EntityFrameworkCore.Design.

Verificado el archivo .csproj con las referencias correctas.

Compilado y ejecutado el proyecto.

Modificado Program.cs con un mensaje personalizado.

Diagnosticado errores comunes de instalación y compilación.

| Conclusión y enlace al siguiente punto

En este punto se ha creado la base del proyecto AceriaData y se han instalado los paquetes de EF Core con el proveedor de SQL Server. En el siguiente punto se estudiará la arquitectura general de Entity Framework Core, identificando cada uno de sus componentes y cómo se relacionan entre sí dentro del proyecto.

Punto 1.2 - Arquitectura general de Entity Framework Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se añade al proyecto AceriaData el primer DbContext y la primera entidad, conectados a SQL Server LocalDB.

Ejercicio: Añadir al proyecto AceriaData el primer DbContext, la primera entidad y la conexión a SQL Server LocalDB. Crear la base de datos con EnsureCreated y verificar que todo funciona.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto creado en el punto anterior. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Error común: si la carpeta no existe, se debe verificar que el punto anterior se completó correctamente.

Paso 2: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}

public class Program
{
    public static void Main()
    {
        using var context = new AceriaDbContext();
        context.Database.EnsureCreated();
        Console.WriteLine("Base de datos AceriaDB creada correctamente en LocalDB.");
    }
}
```

Línea 1: using Microsoft.EntityFrameworkCore; → importa el espacio de nombres de EF Core, que contiene DbContext, DbSet y los métodos de extensión como UseSqlServer. Línea 3: namespace AceriaData.ConsoleApp; → declara el espacio de nombres del proyecto de consola. Línea 5: public class OrdenFabricacion → declara la entidad que representa una orden de fabricación. EF Core la interpretará como una tabla. Línea 7: public int Id { get; set; } → propiedad que EF Core detecta por convención como clave primaria, ya que se llama Id. Línea 8: public string NumeroOrden { get; set; } =

`string.Empty;` → propiedad que almacena el número de orden. Se inicializa para evitar nulos en tiempo de compilación. Línea 9: `public string Cliente { get; set; } = string.Empty;` → propiedad que almacena el nombre del cliente. Línea 10: `public DateTime FechaCreacion { get; set; }` → propiedad que almacena la fecha de creación de la orden. Línea 13: `public class AceriaDbContext : DbContext` → declara el `DbContext` heredando de `DbContext`, la clase base de EF Core. Línea 15: `public DbSet OrdenesFabricacion { get; set; } = null!` → expone la colección de órdenes como una tabla. El `null!` suprime la advertencia de nulabilidad porque EF Core inicializa esta propiedad internamente. Línea 17: `protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)` → sobrescribe el método que configura las opciones del `DbContext`. Línea 19: `optionsBuilder.UseSqlServer()` → invoca el método de extensión que registra el proveedor de SQL Server. Línea 20: `"Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"` → cadena de conexión a la instancia de LocalDB por defecto, base de datos AceriaDB y autenticación de Windows. Línea 23: `public class Program` → clase principal del programa. Línea 25: `public static void Main()` → punto de entrada de la aplicación. Línea 27: `using var context = new AceriaDbContext();` → crea una instancia del `DbContext`. El `using` asegura que se libere al final del bloque. Línea 28: `context.Database.EnsureCreated();` → comprueba si la base de datos y las tablas existen. Si no existen, las crea. Línea 29: `Console.WriteLine("Base de datos AceriaDB creada correctamente en LocalDB.");` → muestra un mensaje de confirmación.

Error común: si la instancia de LocalDB no está iniciada, `EnsureCreated` puede tardar unos segundos la primera vez. Si la cadena de conexión tiene un error de sintaxis, se produce una excepción `SqlException` indicando el problema.

Paso 3: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparece el mensaje Base de datos AceriaDB creada correctamente en LocalDB. y se crea el archivo de base de datos en la instancia de LocalDB.

Error común: si el proyecto no compila, se debe revisar que el paquete `Microsoft.EntityFrameworkCore.SqlServer` esté instalado y que las versiones sean coherentes.

Paso 4: Verificar la base de datos en Visual Studio

Abrir Visual Studio. En el menú Ver, seleccionar Explorador de objetos de SQL Server. Expandir la instancia (localdb)\MSSQLLocalDB, expandir Bases de datos y comprobar que aparece AceriaDB. Expandir AceriaDB, expandir Tablas y comprobar que aparece `dbo.OrdenesFabricacion`.

Resultado esperado: la tabla `dbo.OrdenesFabricacion` aparece con las columnas `Id`, `NumeroOrden`, `Cliente` y `FechaCreacion`.

Paso 5: Insertar una orden desde el programa

Modificar el método `Main` para insertar una orden:

```

public static void Main()
{
    using var context = new AceriaDbContext();
    context.Database.EnsureCreated();

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "OF-001",
        Cliente = "Constructora del Norte",
        FechaCreacion = DateTime.Now
    };

    context OrdenesFabricacion.Add(orden);
    context.SaveChanges();

    Console.WriteLine($"Orden {orden.NumeroOrden} insertada con Id {orden.Id}.");
}

```

Línea 27: `using var context = new AceriaDbContext();` → crea el `DbContext`. Línea 28: `context.Database.EnsureCreated();` → asegura que la base de datos existe. Línea 30: `var orden = new OrdenFabricacion` → crea una nueva instancia de la entidad. Línea 32: `NumeroOrden = "OF-001",` → asigna el número de orden. Línea 33: `Cliente = "Constructora del Norte",` → asigna el cliente. Línea 34: `FechaCreacion = DateTime.Now` → asigna la fecha y hora actuales. Línea 37: `context OrdenesFabricacion.Add(orden);` → añade la entidad al Change Tracker con estado Added. Línea 38: `context.SaveChanges();` → genera el INSERT contra SQL Server y lo ejecuta. Línea 40: `Console.WriteLine($"Orden {orden.NumeroOrden} insertada con Id {orden.Id}.");` → muestra el Id generado por SQL Server.

Error común: si se ejecuta el programa varias veces, se insertan varias órdenes con el mismo número. La tabla no tiene restricción de unicidad sobre `NumeroOrden` todavía.

| Paso 6: Consultar las órdenes

Modificar el método `Main` para consultar las órdenes:

```

public static void Main()
{
    using var context = new AceriaDbContext();
    context.Database.EnsureCreated();

    var ordenes = context OrdenesFabricacion.ToList();

    foreach (var orden in ordenes)
    {
        Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente} |
Fecha: {orden.FechaCreacion:dd/MM/yyyy}");
    }
}

```

Línea 27: `using var context = new AceriaDbContext();` → crea el `DbContext`. Línea 28: `context.Database.EnsureCreated();` → asegura que la base de datos existe. Línea 30: `var ordenes = context OrdenesFabricacion.ToList();` → construye una consulta SELECT contra la tabla `OrdenesFabricacion` y materializa los resultados en una lista. Línea 32: `foreach (var orden in ordenes)` → itera sobre las órdenes recuperadas. Línea 34: `Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente} | Fecha: {orden.FechaCreacion:dd/MM/yyyy}");` → muestra los datos de cada orden con formato.

Error común: si la consulta no devuelve resultados, se debe verificar que el INSERT del paso anterior se ejecutó correctamente.

| Paso 7: Diagnosticar un error común

Eliminar la línea `context.Database.EnsureCreated();` y ejecutar el programa.

Resultado esperado: se produce una excepción `SqlException` indicando que la tabla `OrdenesFabricacion` no existe.

Solución: volver a añadir `context.Database.EnsureCreated();` o aplicar migraciones.

| Paso 8: Insertar una plancha asociada a una orden

Modificar `Program.cs` para añadir la entidad `PlanchaAcero` y su relación con `OrdenFabricacion`:

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 1: `public class PlanchaAcero` → declara la entidad que representa una plancha de acero. Línea 3: `public int Id { get; set; }` → clave primaria por convención. Línea 4: `public int OrdenId { get; set; }` → clave foránea hacia `OrdenFabricacion`. EF Core la detecta por el nombre `OrdenId`. Línea 5: `public double Espesor { get; set; }` → espesor de la plancha en milímetros. Línea 6: `public double Ancho { get; set; }` → ancho de la plancha en milímetros. Línea 7: `public double Largo { get; set; }` → largo de la plancha en milímetros. Línea 8: `public OrdenFabricacion Orden { get; set; } = null!;` → propiedad de navegación que EF Core interpreta como la relación con la orden.

| Paso 9: Añadir el DbSet de planchas

Añadir la propiedad `DbSet` al `AceriaDbContext`:

```
public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
```

Línea 1: `public DbSet PlanchasAcero { get; set; } = null!;` → expone la tabla `PlanchasAcero` en el contexto.

| Paso 10: Insertar una plancha asociada

Modificar el método `Main` para insertar una plancha:

```

public static void Main()
{
    using var context = new AceriaDbContext();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "OF-001",
        Cliente = "Constructora del Norte",
        FechaCreacion = DateTime.Now
    };

    context OrdenesFabricacion.Add(orden);
    context.SaveChanges();

    var plancha = new PlanchaAcero
    {
        OrdenId = orden.Id,
        Espesor = 10.5,
        Ancho = 1500,
        Largo = 3000
    };

    context.PlanchasAcero.Add(plancha);
    context.SaveChanges();

    Console.WriteLine($"Plancha insertada con Id {plancha.Id} para la orden {plancha.OrdenId}.");
}

```

Línea 27: `using var context = new AceriaDbContext();` → crea el DbContext. Línea 28: `context.Database.EnsureDeleted();` → elimina la base de datos si existe, para partir de cero. Línea 29: `context.Database.EnsureCreated();` → crea la base de datos con el nuevo esquema que incluye `PlanchasAcero`. Línea 31: `var orden = new OrdenFabricacion` → crea una nueva orden. Línea 38: `context.OrdenesFabricacion.Add(orden);` → registra la orden en el Change Tracker. Línea 39: `context.SaveChanges();` → inserta la orden y obtiene el Id generado. Línea 41: `var plancha = new PlanchaAcero` → crea una nueva plancha. Línea 43: `OrdenId = orden.Id,` → asigna la clave foránea con el Id de la orden recién insertada. Línea 44: `Espesor = 10.5,` → asigna el espesor. Línea 45: `Ancho = 1500,` → asigna el ancho. Línea 46: `Largo = 3000` → asigna el largo. Línea 49: `context.PlanchasAcero.Add(plancha);` → registra la plancha en el Change Tracker. Línea 50: `context.SaveChanges();` → inserta la plancha con la clave foránea correspondiente. Línea 52: `Console.WriteLine($"Plancha insertada con Id {plancha.Id} para la orden {plancha.OrdenId}.");` → muestra el resultado.

Error común: si se olvida `SaveChanges` después de añadir la orden, `orden.Id` será 0 y la plancha se insertará con una clave foránea inválida, provocando una excepción de integridad referencial.

Errores comunes del ejercicio completo

Error Causa Solución
 SqlException al ejecutar LocalDB no está iniciada Esperar unos segundos y reintentar
 Tabla no existe No se llamó a `EnsureCreated` Añadir `context.Database.EnsureCreated();`
 Clave foránea inválida Se insertó la plancha antes que la orden Guardar la orden primero con `SaveChanges`
 Cadena de conexión incorrecta Barra invertida mal escapada Usar `\` en la cadena o `@` delante
 DbSet nulo No se inicializó la propiedad Añadir `= null!`; o usar constructor

Reto resuelto: Insertar una orden con dos planchas asociadas

Reto: Modificar el método `Main` para insertar una orden con dos planchas asociadas y consultar la orden con sus planchas.

Solución paso a paso

Paso 1: Modificar Program.cs:

```
public static void Main()
{
    using var context = new AceriaDbContext();
    context.Database.EnsureDeleted();
    context.Database.EnsureCreated();

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "OF-001",
        Cliente = "Constructora del Norte",
        FechaCreacion = DateTime.Now
    };

    context.OrdenesFabricacion.Add(orden);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000 };
    var plancha2 = new PlanchaAcero { OrdenId = orden.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500 };

    context.PlanchasAcero.AddRange(plancha1, plancha2);
    context.SaveChanges();

    var ordenRecuperada = context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .FirstOrDefault(o => o.Id == orden.Id);

    Console.WriteLine($"Orden: {ordenRecuperada!.NumeroOrden}");
    foreach (var plancha in ordenRecuperada.Planchas)
    {
        Console.WriteLine($" Plancha Id {plancha.Id} | Espesor: {plancha.Espesor} | Ancho:
{plancha.Ancho} | Largo: {plancha.Largo}");
    }
}
```

Línea 30: `var plancha1 = new PlanchaAcero { ... };` → crea la primera plancha con la clave foránea de la orden. Línea 31: `var plancha2 = new PlanchaAcero { ... };` → crea la segunda plancha con la misma clave foránea. Línea 33: `context.PlanchasAcero.AddRange(plancha1, plancha2);` → añade ambas planchas al Change Tracker en una sola llamada. Línea 34: `context.SaveChanges();` → inserta ambas planchas en la base de datos. Línea 36: `var ordenRecuperada = context.OrdenesFabricacion` → inicia la consulta sobre órdenes. Línea 37: `.Include(o => o.Planchas)` → indica a EF Core que cargue también las planchas relacionadas. Sin esta línea, la colección `Planchas` estaría vacía. Línea 38: `.FirstOrDefault(o => o.Id == orden.Id);` → filtra por Id y devuelve la primera coincidencia o null. Línea 40: `Console.WriteLine($"Orden: {ordenRecuperada!.NumeroOrden}");` → muestra el número de orden. El `!` suprime la advertencia de nulabilidad. Línea 41: `foreach (var plancha in ordenRecuperada.Planchas)` → itera sobre las planchas cargadas por Include. Línea 43: `Console.WriteLine($" Plancha Id {plancha.Id} | ...");` → muestra los datos de cada plancha.

Paso 2: Ejecutar:

```
dotnet run
```

Paso 3: Verificar que aparecen la orden y las dos planchas en la consola.

Analogía final

La arquitectura de EF Core es como el organigrama de una acería. El `DbContext` es el jefe de planta que coordina todas las operaciones. El modelo es el plano que indica qué se fabrica y cómo se relacionan las piezas. El Change Tracker es el supervisor que anota cada cambio en el libro de producción. El proveedor es el fabricante del horno que traduce las órdenes al lenguaje de la máquina. Las migraciones son los

planos de reforma que permiten adaptar la planta a nuevas necesidades. Las consultas LINQ son las órdenes de búsqueda en el archivo central. Todos estos componentes trabajan juntos para que la acería funcione sin que el operario tenga que preocuparse por cada detalle interno.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido la entidad OrdenFabricacion al proyecto AceriaData.

Añadido la entidad PlanchaAcero con su relación.

Configurado el AceriaDbContext con el proveedor de SQL Server.

Creado la base de datos AceriaDB en LocalDB.

Insertado órdenes y planchas.

Consultado la orden con sus planchas mediante Include.

Diagnosticado errores comunes de conexión y de claves foráneas.

| Conclusión y enlace al siguiente punto

En este punto se ha configurado la arquitectura básica de EF Core en el proyecto AceriaData: el DbContext, las entidades, el proveedor de SQL Server y la conexión a LocalDB. En el siguiente punto se estudiarán en detalle los componentes principales de EF Core, identificando cada uno de ellos en el código del proyecto y comprendiendo su responsabilidad concreta.

Punto 1.3 - Componentes principales: DbContext, DbSet, Change Tracker, proveedores y migraciones

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se identifican y utilizan de forma explícita cada uno de los componentes de EF Core dentro del proyecto AceriaData, conectado a SQL Server LocalDB.

Ejercicio: Inspeccionar los cinco componentes de EF Core dentro del proyecto AceriaData, generar la primera migración con dotnet ef y aplicar los cambios a SQL Server LocalDB.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Instalar la herramienta global dotnet-ef

```
dotnet tool install --global dotnet-ef --version 8.0.31
```

dotnet tool install → instala una herramienta global de .NET. --global → indica que la herramienta estará disponible en cualquier carpeta del sistema. dotnet-ef → nombre de la herramienta que permite generar y aplicar migraciones.

Error común: si la herramienta ya está instalada, el comando devuelve un mensaje indicando que ya existe. Se puede actualizar con dotnet tool update --global dotnet-ef --version 8.0.31.

Paso 3: Verificar la instalación de la herramienta

```
dotnet ef --version
```

dotnet ef → invoca la herramienta de EF Core. --version → muestra la versión instalada.

Resultado esperado: aparece la versión 8.x de la herramienta.

Paso 4: Inspeccionar el DbContext y el proveedor

Modificar Program.cs para inspeccionar el proveedor y el modelo:

```

using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}

public class Program
{
    public static void Main()
    {
        using var context = new AceriaDbContext();

        Console.WriteLine($"Proveedor: {context.Database.ProviderName}");
        Console.WriteLine($"Entidades en el modelo: {context.Model.GetEntityTypes().Count()}");

        foreach (var entidad in context.Model.GetEntityTypes())
        {
            Console.WriteLine($" Entidad: {entidad.ClrType.Name} → Tabla: {entidad.GetTableName()}");
        }
    }
}

```

Línea 1: using Microsoft.EntityFrameworkCore; → importa el espacio de nombres de EF Core. Línea 3: namespace AceriaData.ConsoleApp; → declara el espacio de nombres del proyecto. Línea 5: public class OrdenFabricacion → declara la entidad de orden. Línea 7: public int Id { get; set; } → clave primaria. Línea 8: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 9: public string Cliente { get; set; } = string.Empty; → cliente. Línea 10: public DateTime FechaCreacion { get; set; } → fecha de creación. Línea 13: public class PlanchaAcero → declara la entidad de plancha. Línea 15: public int Id { get; set; } → clave primaria. Línea 16: public int OrdenId { get; set; } → clave foránea hacia la orden. Línea 17: public double Espesor { get; set; } → espesor. Línea 18: public double Ancho { get; set; } → ancho. Línea 19: public double Largo { get; set; } → largo. Línea 20: public OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación. Línea 23: public class AceriaDbContext : DbContext → declara el DbContext. Línea 25: public DbSet OrdenesFabricacion { get; set; } = null!; → DbSet de órdenes. Línea 26: public DbSet PlanchasAcero { get; set; } = null!; → DbSet de planchas. Línea 28: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → método de configuración. Línea 30: optionsBuilder.UseSqlServer(→ registra el proveedor de SQL Server. Línea 31: "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;" → cadena de conexión. Línea 35: public class Program → clase principal. Línea 37: public static void Main() → punto de entrada. Línea 39: using var context = new AceriaDbContext(); → crea el DbContext. Línea

41: `Console.WriteLine($"Proveedor: {context.Database.ProviderName}");` → muestra el nombre del proveedor activo. Línea 42: `Console.WriteLine($"Entidades en el modelo: {context.Model.GetEntityTypes().Count()}");` → cuenta las entidades registradas en el modelo. Línea 44: `foreach (var entidad in context.Model.GetEntityTypes())` → itera sobre cada entidad del modelo. Línea 46: `Console.WriteLine($" Entidad: {entidad.ClrType.Name} → Tabla: {entidad.GetTableName()}");` → muestra el nombre de la clase y el nombre de la tabla.

Error común: si `context.Model` está vacío, se debe verificar que las entidades estén referenciadas por algún `DbSet` o que estén configuradas explícitamente.

Paso 5: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las líneas con el proveedor, el número de entidades y la correspondencia entre clases y tablas.

Paso 6: Inspeccionar el Change Tracker

Modificar el método `Main` para observar los estados de las entidades:

```
public static void Main()
{
    using var context = new AceriaDbContext();

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "OF-001",
        Cliente = "Constructora del Norte",
        FechaCreacion = DateTime.Now
    };

    Console.WriteLine($"Estado antes de Add: {context.Entry(orden).State}");

    context OrdenesFabricacion.Add(orden);

    Console.WriteLine($"Estado después de Add: {context.Entry(orden).State}");

    orden.Cliente = "Constructora del Sur";

    Console.WriteLine($"Estado después de modificar: {context.Entry(orden).State}");
}
```

Línea 39: `using var context = new AceriaDbContext();` → crea el `DbContext`. Línea 41: `var orden = new OrdenFabricacion` → crea la entidad. Línea 48: `Console.WriteLine($"Estado antes de Add: {context.Entry(orden).State}");` → muestra el estado inicial, que es `Detached`. Línea 50: `context OrdenesFabricacion.Add(orden);` → registra la entidad en el `Change Tracker`. Línea 52: `Console.WriteLine($"Estado después de Add: {context.Entry(orden).State}");` → muestra el estado `Added`. Línea 54: `orden.Cliente = "Constructora del Sur";` → modifica una propiedad. Línea 56: `Console.WriteLine($"Estado después de modificar: {context.Entry(orden).State}");` → muestra el estado `Added`, que prevalece sobre `Modified` porque la entidad aún no se ha guardado.

Error común: si la entidad no aparece en el `Change Tracker`, se debe verificar que se haya añadido con `Add`, `Attach` o `Update`.

Paso 7: Ejecutar el proyecto

```
dotnet run
```

Resultado esperado: aparecen los estados `Detached`, `Added` y `Added` en la consola.

Paso 8: Inspeccionar el modelo completo

Modificar el método Main para recorrer las propiedades de cada entidad:

```
public static void Main()
{
    using var context = new AceriaDbContext();

    foreach (var entidad in context.Model.GetEntityTypes())
    {
        Console.WriteLine($"Entidad: {entidad.ClrType.Name} → Tabla: {entidad.GetTableName()}");

        foreach (var propiedad in entidad.GetProperties())
        {
            Console.WriteLine($"  Propiedad: {propiedad.Name} | Columna: {propiedad.GetColumnName()} |
Tipo: {propiedad.ClrType.Name}");
        }
    }
}
```

Línea 41: `foreach (var entidad in context.Model.GetEntityTypes())` → itera sobre cada entidad del modelo. Línea 43: `Console.WriteLine($"Entidad: {entidad.ClrType.Name} → Tabla: {entidad.GetTableName()}");` → muestra el nombre de la entidad y su tabla. Línea 45: `foreach (var propiedad in entidad.GetProperties())` → itera sobre cada propiedad de la entidad. Línea 47: `Console.WriteLine($" Propiedad: {propiedad.Name} | Columna: {propiedad.GetColumnName()} | Tipo: {propiedad.ClrType.Name}");` → muestra el nombre de la propiedad, su columna y su tipo.

Resultado esperado: aparecen todas las propiedades de `OrdenFabricacion` y `PlanchaAcero` con sus columnas y tipos.

Paso 9: Generar la primera migración

Antes de generar la migración, asegurarse de que el proyecto compila:

```
dotnet build
```

`dotnet build` → compila el proyecto y verifica que no hay errores.

A continuación, generar la migración:

```
dotnet ef migrations add InitialCreate
```

`dotnet ef migrations add` → invoca el comando para añadir una migración. `InitialCreate` → nombre de la migración. Por convención, la primera se llama `InitialCreate`.

Resultado esperado: se crea la carpeta `Migrations` con tres archivos: `InitialCreate.cs`, `InitialCreate.Designer.cs` y `AceriaDbContextModelSnapshot.cs`.

Error común: si el comando falla indicando que no se encuentra el `DbContext`, se debe verificar que la clase `AceriaDbContext` tenga un constructor sin parámetros o que implemente `IDesignTimeDbContextFactory`.

Paso 10: Revisar el archivo de migración

Abrir `Migrations/20240115120000_InitialCreate.cs`:

```

public partial class InitialCreate : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "OrdenesFabricacion",
            columns: table => new
            {
                Id = table.Column<int>(type: "int", nullable: false)
                    .Annotation("SqlServer:Identity", "1, 1"),
                NumeroOrden = table.Column<string>(type: "nvarchar(max)", nullable: false),
                Cliente = table.Column<string>(type: "nvarchar(max)", nullable: false),
                FechaCreacion = table.Column<DateTime>(type: "datetime2", nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_OrdenesFabricacion", x => x.Id);
            });

        migrationBuilder.CreateTable(
            name: "PlanchasAcero",
            columns: table => new
            {
                Id = table.Column<int>(type: "int", nullable: false)
                    .Annotation("SqlServer:Identity", "1, 1"),
                OrdenId = table.Column<int>(type: "int", nullable: false),
                Espesor = table.Column<double>(type: "float", nullable: false),
                Ancho = table.Column<double>(type: "float", nullable: false),
                Largo = table.Column<double>(type: "float", nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_PlanchasAcero", x => x.Id);
                table.ForeignKey(
                    name: "FK_PlanchasAcero_OrdenesFabricacion_OrdenId",
                    column: x => x.OrdenId,
                    principalTable: "OrdenesFabricacion",
                    principalColumn: "Id",
                    onDelete: ReferentialAction.Cascade);
            });
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(name: "PlanchasAcero");
        migrationBuilder.DropTable(name: "OrdenesFabricacion");
    }
}

```

Línea 1: public partial class InitialCreate : Migration → declara la migración. Línea 3: protected override void Up(MigrationBuilder migrationBuilder) → método que aplica los cambios. Línea 5: migrationBuilder.CreateTable(→ crea la tabla OrdenesFabricacion. Línea 6: name: "OrdenesFabricacion", → nombre de la tabla. Línea 7: columns: table => new → define las columnas. Línea 9: Id = table.Column(type: "int", nullable: false) → columna Id de tipo int no anulable. Línea 10: .Annotation("SqlServer:Identity", "1, 1"), → configura la columna como identidad autoincremental de SQL Server. Línea 11: NumeroOrden = table.Column(type: "nvarchar(max)", nullable: false), → columna NumeroOrden de tipo nvarchar(max). Línea 12: Cliente = table.Column(type: "nvarchar(max)", nullable: false), → columna Cliente. Línea 13: FechaCreacion = table.Column(type: "datetime2", nullable: false) → columna FechaCreacion de tipo datetime2. Línea 16: table.PrimaryKey("PK_OrdenesFabricacion", x => x.Id); → define la clave primaria. Línea 19: migrationBuilder.CreateTable(→ crea la tabla PlanchasAcero. Línea 25: Id = table.Column(type: "int", nullable: false) → columna Id. Línea 26: .Annotation("SqlServer:Identity", "1, 1"), → identidad autoincremental. Línea 27: OrdenId = table.Column(type: "int", nullable: false), → columna OrdenId. Línea 28: Espesor = table.Column(type: "float", nullable: false), → columna Espesor. Línea 29: Ancho = table.Column(type: "float", nullable: false), → columna Ancho. Línea 30: Largo = table.Column(type: "float", nullable: false) → columna Largo. Línea 33: table.PrimaryKey("PK_PlanchasAcero", x => x.Id); → clave primaria. Línea 34: table.ForeignKey(→ define la clave foránea hacia OrdenesFabricacion. Línea 37: principalTable:

"OrdenesFabricacion", → tabla principal de la relación. Línea 39: onDelete: ReferentialAction.Cascade); → elimina las planchas al eliminar la orden. Línea 43: protected override void Down(MigrationBuilder migrationBuilder) → método que revierte los cambios. Línea 45: migrationBuilder.DropTable(name: "PlanchasAcero"); → elimina la tabla PlanchasAcero. Línea 46: migrationBuilder.DropTable(name: "OrdenesFabricacion"); → elimina la tabla OrdenesFabricacion.

Error común: si la migración incluye tablas inesperadas, se debe revisar que no haya propiedades de navegación mal configuradas que EF Core interprete como entidades adicionales.

Paso 11: Aplicar la migración

```
dotnet ef database update
```

dotnet ef database update → aplica todas las migraciones pendientes a la base de datos.

Resultado esperado: se crean las tablas OrdenesFabricacion y PlanchasAcero en la base de datos AceriaDB.

Error común: si la base de datos ya existe con tablas creadas por EnsureCreated, el comando falla porque las tablas ya existen. Se debe eliminar la base de datos o usar EnsureDeleted antes de aplicar migraciones.

Paso 12: Verificar la migración aplicada

```
dotnet ef migrations list
```

dotnet ef migrations list → lista todas las migraciones y su estado.

Resultado esperado: aparece InitialCreate con la marca (Applied).

Errores comunes del ejercicio completo

Error Causa Solución dotnet ef no se reconoce La herramienta global no está instalada Ejecutar dotnet tool install --global dotnet-ef --version 8.0.31 No se encuentra el DbContext Falta constructor o factory de diseño Añadir constructor sin parámetros o IDesignTimeDbContextFactory Tablas ya existen Se usó EnsureCreated antes Eliminar la base de datos o usar migraciones desde el inicio Migración vacía El modelo no cambió Verificar que los DbSet están declarados Error de clave foránea Orden de creación de tablas EF Core resuelve el orden automáticamente Snapshot desactualizado Se modificó el modelo sin regenerar Ejecutar dotnet ef migrations add de nuevo

Reto resuelto: Añadir la entidad Aleacion con migración

Reto: Añadir una entidad Aleacion con propiedades Id, Nombre, PorcentajeCarbono y PorcentajeManganeso. Añadir el DbSet correspondiente. Generar y aplicar la migración.

Solución paso a paso

Paso 1: Añadir la entidad Aleacion:

```
public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}
```

Línea 1: public class Aleacion → declara la entidad. Línea 3: public int Id { get; set; } → clave primaria.

Línea 4: public string Nombre { get; set; } = string.Empty; → nombre de la aleación. Línea 5: public

double PorcentajeCarbono { get; set; } → porcentaje de carbono. Línea 6: public double PorcentajeManganeso { get; set; } → porcentaje de manganeso.

Paso 2: Añadir el DbSet al AceriaDbContext:

```
public DbSet<Aleacion> Aleaciones { get; set; } = null!;
```

Línea 1: public DbSet Aleaciones { get; set; } = null!; → expone la tabla Aleaciones.

Paso 3: Compilar y generar la migración:

```
dotnet build
dotnet ef migrations add AddAleacion
```

InitialCreate → primera migración. AddAleacion → segunda migración.

Paso 4: Aplicar la migración:

```
dotnet ef database update
```

Paso 5: Verificar con dotnet ef migrations list que aparecen ambas migraciones aplicadas.

Analogía final

Los cinco componentes de EF Core son como los cinco puestos clave de una acería. El DbContext es el jefe de planta que coordina todas las operaciones. Los DbSet son los cajones donde se guardan las órdenes y las planchas. El Change Tracker es el supervisor que anota cada cambio en el libro de producción. El proveedor es el fabricante del horno que traduce las órdenes al lenguaje de la máquina. Las migraciones son los planos de reforma que permiten ampliar la planta sin detener la producción. Todos ellos trabajan juntos para que la acería funcione sin que el operario tenga que preocuparse por cada detalle interno.

Resultado esperado

Al final del ejercicio, deberías haber:

Inspeccionado el DbContext, el modelo y el proveedor.

Observado los estados del Change Tracker.

Recorrido las entidades y propiedades del modelo.

Instalado la herramienta global dotnet-ef.

Generado la primera migración InitialCreate.

Aplicado la migración a la base de datos AceriaDB.

Verificado las tablas creadas.

Añadido la entidad Aleacion y su migración.

Conclusión y enlace al siguiente punto

En este punto se han identificado y utilizado los cinco componentes de EF Core dentro del proyecto AceriaData. El DbContext, los DbSet, el Change Tracker, el proveedor de SQL Server y las migraciones ya forman parte del proyecto. En el siguiente punto se estudiará en detalle el DbContext, sus responsabilidades y las propiedades que expone para gestionar la sesión con la base de datos.

Punto 1.4 - El DbContext: rol, responsabilidades y propiedades DbSet

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se profundiza en el DbContext del proyecto AceriaData, se inspecciona la entidad Aleacion incorporada en 1.3 y se añade EstadoOrden con su DbSet y migración.

Ejercicio: Inspeccionar las propiedades del DbContext, observar SaveChanges, revisar Aleacion ya incorporada y añadir EstadoOrden con su DbSet y migración sobre SQL Server LocalDB.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Revisar la entidad Aleacion incorporada en 1.3

Abrir Program.cs y comprobar la clase Aleacion ya incorporada en el punto 1.3:

```
public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}
```

Línea 1: public class Aleacion → declara la entidad que representa una aleación de acero. Línea 3: public int Id { get; set; } → clave primaria por convención. Línea 4: public string Nombre { get; set; } = string.Empty; → nombre de la aleación. Se inicializa para evitar nulos. Línea 5: public double PorcentajeCarbono { get; set; } → porcentaje de carbono en la aleación. Línea 6: public double PorcentajeManganeso { get; set; } → porcentaje de manganeso en la aleación.

Paso 3: Verificar el DbSet de Aleacion en AceriaDbContext

Comprobar que la propiedad DbSet ya está presente en AceriaDbContext:

```
public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}
```

Línea 3: public DbSet OrdenesFabricacion { get; set; } = null!; → expone la tabla OrdenesFabricacion. Línea 4: public DbSet PlanchasAcero { get; set; } = null!; → expone la tabla PlanchasAcero. Línea 5: public DbSet Aleaciones { get; set; } = null!; → expone la tabla Aleaciones. Línea 7: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → configura las opciones del DbContext. Línea 9: optionsBuilder.UseSqlServer(→ registra el proveedor de SQL Server. Línea 10: "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;" → cadena de conexión a LocalDB.

Error común: si se olvida añadir el DbSet, la entidad no se incluye en el modelo y no se crea la tabla correspondiente.

Paso 4: Inspeccionar las propiedades del DbContext

Sustituir el método Main para inspeccionar las propiedades del DbContext:

```
public static void Main()
{
    using var context = new AceriaDbContext();

    Console.WriteLine($"Proveedor: {context.Database.ProviderName}");
    Console.WriteLine($"Entidades: {context.Model.GetEntityTypes().Count()}");

    foreach (var entidad in context.Model.GetEntityTypes())
    {
        Console.WriteLine($" {entidad.ClrType.Name} → {entidad.GetTableName()}");
    }
}
```

Línea 1: public static void Main() → punto de entrada del programa. Línea 3: using var context = new AceriaDbContext(); → crea el DbContext. Línea 5: Console.WriteLine(\$"Proveedor: {context.Database.ProviderName}"); → muestra el nombre del proveedor. Línea 6: Console.WriteLine(\$"Entidades: {context.Model.GetEntityTypes().Count()}"); → cuenta las entidades del modelo. Línea 8: foreach (var entidad in context.Model.GetEntityTypes()) → itera sobre cada entidad. Línea 10: Console.WriteLine(\$" {entidad.ClrType.Name} → {entidad.GetTableName()}"); → muestra la clase y su tabla.

Resultado esperado: aparecen tres entidades: OrdenFabricacion, PlanchaAcero y Aleacion.

Paso 5: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: la consola muestra el proveedor Microsoft.EntityFrameworkCore.SqlServer, el número de entidades y la correspondencia entre clases y tablas.

Paso 6: Observar SaveChanges en acción

Modificar el método Main para observar cómo SaveChanges devuelve el número de filas afectadas:

```
public static void Main()
{
    using var context = new AceriaDbContext();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var aleacion = new Aleacion
    {
        Nombre = "AISI 1045",
        PorcentajeCarbono = 0.45,
        PorcentajeManganeso = 0.75
    };

    context.Aleaciones.Add(aleacion);
    var filas = context.SaveChanges();

    Console.WriteLine($"Filas afectadas: {filas}");
    Console.WriteLine($"Aleación insertada con Id {aleacion.Id}");
}
```

Línea 3: `using var context = new AceriaDbContext();` → crea el `DbContext`. Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos si existe. Línea 5: `context.Database.Migrate();` → crea la base de datos con el esquema actualizado. Línea 7: `var aleacion = new Aleacion` → crea una nueva aleación. Línea 9: `Nombre = "AISI 1045",` → asigna el nombre. Línea 10: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono. Línea 11: `PorcentajeManganeso = 0.75` → asigna el porcentaje de manganeso. Línea 14: `context.Aleaciones.Add(aleacion);` → añade la aleación al Change Tracker con estado Added. Línea 15: `var filas = context.SaveChanges();` → ejecuta el INSERT y devuelve el número de filas afectadas. Línea 17: `Console.WriteLine($"Filas afectadas: {filas}");` → muestra el número de filas afectadas. Línea 18: `Console.WriteLine($"Aleación insertada con Id {aleacion.Id}");` → muestra el Id generado.

Error común: si se ejecuta el programa varias veces sin `EnsureDeleted`, la tabla `Aleaciones` acumula registros. Con `EnsureDeleted` al principio, la base de datos se recrea cada vez.

Paso 7: Ejecutar el proyecto

```
dotnet run
```

Resultado esperado: aparecen las líneas Filas afectadas: 1 y Aleación insertada con Id 1.

Paso 8: Observar el Change Tracker antes y después de SaveChanges

Modificar el método `Main` para observar el Change Tracker:

```
public static void Main()
{
    using var context = new AceriaDbContext();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var aleacion = new Aleacion
    {
        Nombre = "AISI 1045",
        PorcentajeCarbono = 0.45,
        PorcentajeManganeso = 0.75
    };

    context.Aleaciones.Add(aleacion);

    Console.WriteLine("Antes de SaveChanges:");
    foreach (var entrada in context.ChangeTracker.Entries())
    {
        Console.WriteLine($" {entrada.Entity.GetType().Name}: {entrada.State}");
    }

    context.SaveChanges();

    Console.WriteLine("Después de SaveChanges:");
    foreach (var entrada in context.ChangeTracker.Entries())
    {
        Console.WriteLine($" {entrada.Entity.GetType().Name}: {entrada.State}");
    }
}
```

Línea 14: `context.Aleaciones.Add(aleacion);` → registra la aleación con estado Added. Línea 16: `Console.WriteLine("Antes de SaveChanges:");` → cabecera del primer listado. Línea 17: `foreach (var entrada in context.ChangeTracker.Entries())` → itera sobre las entidades rastreadas. Línea 19: `Console.WriteLine($" {entrada.Entity.GetType().Name}: {entrada.State}");` → muestra el tipo y el estado. Línea 22: `context.SaveChanges();` → ejecuta el INSERT. Línea 24: `Console.WriteLine("Después de SaveChanges:");` → cabecera del segundo listado. Línea 25: `foreach (var entrada in context.ChangeTracker.Entries())` → itera de nuevo sobre las entidades rastreadas. Línea 27: `Console.WriteLine($" {entrada.Entity.GetType().Name}: {entrada.State}");` → muestra el nuevo estado, que es `Unchanged`.

Resultado esperado: antes de SaveChanges la entidad aparece como Added. Después de SaveChanges aparece como Unchanged.

Paso 9: Añadir la entidad EstadoOrden

Añadir la entidad EstadoOrden al modelo:

```
public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}
```

Añadir también el DbSet:

```
public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;
```

Paso 10: Generar la migración AddEstadoOrden

```
dotnet ef migrations add AddEstadoOrden
```

La migración debe contener únicamente el cambio asociado a EstadoOrden, porque Aleacion ya quedó incorporada en 1.3.

Paso 11: Aplicar la migración

```
dotnet ef database update
```

Paso 12: Verificar la migración y la tabla

```
dotnet ef migrations list
```

Verificar en el Explorador de objetos de SQL Server que existen OrdenesFabricacion, PlanchasAcero, Aleaciones y EstadosOrden.

Errores comunes del ejercicio completo

Error	Causa	Solución
Entidad no aparece en el modelo	Falta el DbSet	Añadir public DbSet Aleaciones { get; set; }
Migración vacía	No se modificó el modelo	Verificar que el DbSet está declarado
Tabla no se crea	No se aplicó la migración	Ejecutar dotnet ef database update
SaveChanges no devuelve filas	La entidad no estaba en estado Added	Verificar que se llamó a Add antes
Estado incorrecto	Se modificó después de guardar	El Change Tracker actualiza el estado en la siguiente operación
Error de conexión	LocalDB no responde	Reiniciar Visual Studio o esperar unos segundos

Reto resuelto: Consultar el modelo completo tras AddEstadoOrden

Con EstadoOrden ya incorporado, inspeccionar el modelo completo:

Paso 1: Consultar el modelo completo:

```
public static void Main()
{
    using var context = new AceriaDbContext();

    foreach (var entidad in context.Model.GetEntityTypes())
    {
        Console.WriteLine($"Entidad: {entidad.ClrType.Name} → Tabla: {entidad.GetTableName()}");

        foreach (var propiedad in entidad.GetProperties())
        {
            Console.WriteLine($" Propiedad: {propiedad.Name} | Columna: {propiedad.GetColumnName()} |
Tipo: {propiedad.ClrType.Name}");
        }
    }
}
```

Línea 5: `foreach (var entidad in context.Model.GetEntityTypes())` → itera sobre cada entidad del modelo.
 Línea 7: `Console.WriteLine($"Entidad: {entidad.ClrType.Name} → Tabla: {entidad.GetTableName()}");` → muestra la clase y la tabla.
 Línea 9: `foreach (var propiedad in entidad.GetProperties())` → itera sobre cada propiedad de la entidad.
 Línea 11: `Console.WriteLine($" Propiedad: {propiedad.Name} | Columna: {propiedad.GetColumnName()} | Tipo: {propiedad.ClrType.Name}");` → muestra la propiedad, su columna y su tipo.

Analogía final

El `DbContext` es como el jefe de planta de una acería. No fabrica acero él mismo, pero coordina todos los recursos: los hornos (proveedores), el libro de producción (`Change Tracker`), los cajones de almacenamiento (`DbSet`) y los planos de la planta (modelo). Cuando llega una orden de fabricación, el jefe de planta decide qué recursos intervienen y cómo registrar el resultado. Cada `DbSet` es un acceso concreto a un conjunto de entidades. El jefe de planta no mantiene una unidad de trabajo abierta indefinidamente: la usa durante una operación coherente y libera los recursos al terminar. Así funciona el `DbContext`: vive poco, coordina mucho y se libera al final de la unidad de trabajo.

Resultado esperado

Al final del ejercicio, deberías haber:

Revisado la entidad `Aleacion` y su `DbSet`, incorporados en 1.3.

Inspeccionado las propiedades `Database`, `Model` y `ChangeTracker` del `DbContext`.

Observado el cambio de estado de `Added` a `Unchanged` tras `SaveChanges`.

Añadido la entidad `EstadoOrden` con su `DbSet`.

Generado y aplicado la migración `AddEstadoOrden`.

Verificado las cuatro tablas en el Explorador de objetos de SQL Server.

Consultado el modelo completo en el reto resuelto.

Conclusión y enlace al siguiente punto

En este punto se ha profundizado en el `DbContext` como coordinador de la unidad de trabajo y se han explorado sus propiedades `Database`, `Model`, `ChangeTracker` y el método `SaveChanges`. `Aleacion` se ha mantenido como parte del estado heredado de 1.3 y `EstadoOrden` se ha incorporado con su `DbSet` y su migración. En el siguiente punto se estudiará el ciclo de vida del `DbContext` en aplicaciones de consola, web y servicios, y cómo gestionarlo correctamente en cada escenario.

Punto 1.5 - Ciclo de vida del DbContext en aplicaciones de consola, web y servicios

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se refactoriza el proyecto AceriaData para gestionar correctamente el ciclo de vida del DbContext en aplicaciones de consola, y se prepara la estructura para su uso futuro en aplicaciones web y servicios, siempre con SQL Server LocalDB.

Ejercicio: Refactorizar AceriaData para aplicar una unidad de trabajo corta, crear una fábrica manual de DbContext y observar el Change Tracker en distintos ciclos de vida. EstadoOrden y su migración proceden ya del punto 1.4.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
```

```

    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}

public static class AceriaDbContextFactory
{
    public static AceriaDbContext Create()
    {
        return new AceriaDbContext();
    }
}

public class Program
{
    public static void Main()
    {
        using (var context = AceriaDbContextFactory.Create())
        {
            context.Database.EnsureDeleted();
            context.Database.Migrate();

            InsertarOrden("OF-001", "Constructora del Norte");
            InsertarOrden("OF-002", "Constructora del Sur");

            ListarOrdenes();
        }

        public static void InsertarOrden(string numero, string cliente)
        {
            using var context = AceriaDbContextFactory.Create();
            var orden = new OrdenFabricacion
            {
                NumeroOrden = numero,
                Cliente = cliente,
                FechaCreacion = DateTime.Now
            };
            context OrdenesFabricacion.Add(orden);
            context.SaveChanges();
        }

        public static void ListarOrdenes()
        {
            using var context = AceriaDbContextFactory.Create();
            var ordenes = context OrdenesFabricacion.ToList();
            foreach (var orden in ordenes)
            {
                Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente:
{orden.Cliente}");
            }
        }
    }
}

```

Línea 1: using Microsoft.EntityFrameworkCore; → importa el espacio de nombres de EF Core. Línea 3: namespace AceriaData.ConsoleApp; → declara el espacio de nombres del proyecto. Línea 5: public class OrdenFabricacion → entidad de orden. Línea 7: public int Id { get; set; } → clave primaria. Línea 8: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 9: public string Cliente { get; set; } = string.Empty; → cliente. Línea 10: public DateTime FechaCreacion { get; set; } → fecha de creación. Línea 13: public class PlanchaAcero → entidad de plancha. Línea 15: public int Id { get; set; } → clave primaria. Línea 16: public int OrdenId { get; set; } → clave foránea. Línea 17: public double Espesor { get; set; } → espesor. Línea 18: public double Ancho { get; set; } → ancho. Línea 19: public double Largo { get; set; } → largo. Línea 20: public OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación. Línea 23: public class Aleacion → entidad de aleación. Línea 25: public int Id { get; set; } →

clave primaria. Línea 26: `public string Nombre { get; set; } = string.Empty;` → nombre. Línea 27: `public double PorcentajeCarbono { get; set; }` → porcentaje de carbono. Línea 28: `public double PorcentajeManganeso { get; set; }` → porcentaje de manganeso. `EstadoOrden` heredada de 1.4: contiene `Id`, `Nombre` y `Descripcion` y permanece en el modelo al iniciar este punto. Línea 31: `public class AceriaDbContext : DbContext` → declara el `DbContext`. Línea 33: `public DbSet OrdenesFabricacion { get; set; } = null!;` → `DbSet` de órdenes. Línea 34: `public DbSet PlanchasAcero { get; set; } = null!;` → `DbSet` de planchas. Línea 35: `public DbSet Aleaciones { get; set; } = null!;` → `DbSet` de aleaciones. `DbSet EstadosOrden` → expone la entidad `EstadoOrden` incorporada en el punto anterior. Línea 37: `protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)` → configuración del contexto. Línea 39: `optionsBuilder.UseSqlServer()` → registra el proveedor de SQL Server. Línea 40: `"Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"` → cadena de conexión. Línea 44: `public static class AceriaDbContextFactory` → declara la fábrica manual. Línea 46: `public static AceriaDbContext Create()` → método que devuelve una instancia nueva. Línea 48: `return new AceriaDbContext();` → crea el `DbContext` y lo devuelve. Línea 52: `public class Program` → clase principal. Línea 54: `public static void Main()` → punto de entrada. Línea 56: `using (var context = AceriaDbContextFactory.Create())` → primera unidad de trabajo: recreación de la base de datos. Línea 58: `context.Database.EnsureDeleted();` → elimina la base de datos. Línea 59: `context.Database.Migrate();` → crea la base de datos con el esquema actual. Línea 62: `InsertarOrden("OF-001", "Constructora del Norte");` → primera unidad de trabajo: inserta una orden. Línea 63: `InsertarOrden("OF-002", "Constructora del Sur");` → segunda unidad de trabajo: inserta otra orden. Línea 65: `ListarOrdenes();` → tercera unidad de trabajo: consulta las órdenes. Línea 68: `public static void InsertarOrden(string numero, string cliente)` → método que inserta una orden. Línea 70: `using var context = AceriaDbContextFactory.Create();` → crea un `DbContext` para esta operación. Línea 71: `var orden = new OrdenFabricacion` → crea la entidad. Línea 77: `context.OrdenesFabricacion.Add(orden);` → registra la entidad con estado `Added`. Línea 78: `context.SaveChanges();` → ejecuta el `INSERT`. Línea 81: `public static void ListarOrdenes()` → método que consulta las órdenes. Línea 83: `using var context = AceriaDbContextFactory.Create();` → crea un `DbContext` para esta operación. Línea 84: `var ordenes = context.OrdenesFabricacion.ToList();` → consulta todas las órdenes. Línea 85: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 87: `Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");` → muestra los datos.

Error común: si se comparte el `DbContext` entre los tres métodos, el `Change Tracker` acumula entidades y la memoria crece. Con la fábrica y el patrón de unidad de trabajo, cada operación usa una instancia nueva.

Paso 3: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las dos órdenes insertadas, `OF-001` y `OF-002`, con sus respectivos clientes.

Paso 4: Observar el Change Tracker con unidades de trabajo

Modificar el método `InsertarOrden` para inspeccionar el `Change Tracker` antes y después de `SaveChanges`:

```

public static void InsertarOrden(string numero, string cliente)
{
    using var context = AceriaDbContextFactory.Create();
    var orden = new OrdenFabricacion
    {
        NumeroOrden = numero,
        Cliente = cliente,
        FechaCreacion = DateTime.Now
    };
    context OrdenesFabricacion.Add(orden);

    Console.WriteLine($"Entidades rastreadas antes de SaveChanges:
{context.ChangeTracker.Entries().Count()}");

    context.SaveChanges();

    Console.WriteLine($"Entidades rastreadas después de SaveChanges:
{context.ChangeTracker.Entries().Count()}");
}

```

Línea 70: `using var context = AceriaDbContextFactory.Create();` → crea el DbContext. Línea 71: `var orden = new OrdenFabricacion` → crea la entidad. Línea 77: `context OrdenesFabricacion.Add(orden);` → registra la entidad. Línea 79: `Console.WriteLine($"Entidades rastreadas antes de SaveChanges: {context.ChangeTracker.Entries().Count()}");` → muestra el número de entidades rastreadas antes de guardar. Línea 81: `context.SaveChanges();` → ejecuta el INSERT. Línea 83: `Console.WriteLine($"Entidades rastreadas después de SaveChanges: {context.ChangeTracker.Entries().Count()}");` → muestra el número de entidades rastreadas después de guardar, que sigue siendo uno pero con estado `Unchanged`.

Resultado esperado: aparecen dos líneas con 1 y 1, confirmando que el Change Tracker contiene la entidad en ambos momentos.

Paso 5: Ejecutar el proyecto

```
dotnet run
```

Resultado esperado: además de las órdenes, aparecen los mensajes del Change Tracker.

Paso 6: Demostrar el problema del DbContext compartido

Modificar el método Main para usar un DbContext compartido:

```

public static void Main()
{
    using (var context = AceriaDbContextFactory.Create())
    {
        context.Database.EnsureDeleted();
        context.Database.Migrate();
    }

    using var contextCompartido = AceriaDbContextFactory.Create();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte",
FechaCreacion = DateTime.Now };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-002", Cliente = "Constructora del Sur",
FechaCreacion = DateTime.Now };

    contextCompartido OrdenesFabricacion.Add(orden1);
    contextCompartido OrdenesFabricacion.Add(orden2);
    contextCompartido.SaveChanges();

    Console.WriteLine($"Entidades en el Change Tracker:
{contextCompartido.ChangeTracker.Entries().Count()}");

    var ordenes = contextCompartido OrdenesFabricacion.ToList();
    Console.WriteLine($"Órdenes recuperadas: {ordenes.Count()}");
}

```

Línea 56: `using (var context = AceriaDbContextFactory.Create())` → crea un contexto para recrear la base de datos. Línea 58: `context.Database.EnsureDeleted();` → elimina la base de datos. Línea 59: `context.Database.Migrate();` → crea la base de datos. Línea 62: `using var contextCompartido = AceriaDbContextFactory.Create();` → crea un contexto compartido para varias operaciones. Línea 64: `var orden1 = new OrdenFabricacion { ... };` → crea la primera orden. Línea 65: `var orden2 = new OrdenFabricacion { ... };` → crea la segunda orden. Línea 67: `contextCompartido.OrdenesFabricacion.Add(orden1);` → registra la primera orden. Línea 68: `contextCompartido.OrdenesFabricacion.Add(orden2);` → registra la segunda orden. Línea 69: `contextCompartido.SaveChanges();` → inserta ambas órdenes. Línea 71: `Console.WriteLine($"Entidades en el Change Tracker: {contextCompartido.ChangeTracker.Entries().Count()}");` → muestra cuántas entidades hay en el Change Tracker. Línea 73: `var ordenes = contextCompartido.OrdenesFabricacion.ToList();` → consulta las órdenes. Línea 74: `Console.WriteLine($"Órdenes recuperadas: {ordenes.Count()}");` → muestra el número de órdenes recuperadas.

Resultado esperado: el Change Tracker contiene dos entidades después de insertar. La consulta devuelve las mismas dos entidades porque el DbContext las reconoce por su clave primaria en la caché de identidad.

Paso 7: Ejecutar el proyecto

```
dotnet run
```

Resultado esperado: se muestra que el Change Tracker contiene dos entidades y que la consulta devuelve dos órdenes.

Paso 8: Verificar EstadoOrden heredado del punto 1.4

Comprobar que `EstadoOrden` y `DbSet<EstadoOrden>` ya forman parte del proyecto. No deben volver a crearse en este punto.

```
public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}
```

Paso 9: Verificar el DbSet

```
public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;
```

Paso 10: Verificar el historial de migraciones

```
dotnet ef migrations list
```

Debe aparecer `AddEstadoOrden` como migración ya creada en 1.4. No se genera una segunda migración con el mismo propósito.

Paso 11: Aplicar migraciones pendientes

```
dotnet ef database update
```

Paso 12: Confirmar el estado del esquema

Verificar en SQL Server LocalDB que la tabla `EstadosOrden` existe antes de continuar con las demostraciones de ciclo de vida.

Errores comunes del ejercicio completo

Error	Causa	Solución
Change Tracker crece indefinidamente	DbContext compartido	Crear una instancia por operación
Datos obsoletos en consultas	Caché de identidad	Usar un DbContext nuevo para cada consulta
Conexión retenida	DbContext no liberado	Usar using o llamar a Dispose
Excepción de concurrencia	DbContext compartido entre hilos	Crear una instancia por hilo
Migración vacía	No se modificó el modelo	Verificar que el DbSet está declarado
Error de conexión	LocalDB no responde	Reiniciar Visual Studio o esperar

Reto resuelto: Insertar una orden con dos planchas en una sola unidad de trabajo

Reto: Crear una unidad de trabajo que inserte una orden y dos planchas asociadas, y que devuelva el número de filas afectadas. Todo dentro de un único DbContext.

Solución paso a paso

Paso 1: Añadir el método InsertarOrdenConPlanchas:

```
public static void InsertarOrdenConPlanchas()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "OF-003",
        Cliente = "Constructora del Este",
        FechaCreacion = DateTime.Now
    };

    context.OrdenesFabricacion.Add(orden);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000 };
    var plancha2 = new PlanchaAcero { OrdenId = orden.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500 };

    context.PlanchasAcero.AddRange(plancha1, plancha2);
    var filas = context.SaveChanges();

    Console.WriteLine($"Orden {orden.NumeroOrden} insertada con Id {orden.Id}");
    Console.WriteLine($"Planchas insertadas: {filas}");
}
```

Línea 1: `public static void InsertarOrdenConPlanchas()` → declara el método. Línea 3: `using var context = AceriaDbContextFactory.Create();` → crea el DbContext para esta unidad de trabajo. Línea 5: `var orden = new OrdenFabricacion` → crea la orden. Línea 11: `context.OrdenesFabricacion.Add(orden);` → registra la orden. Línea 12: `context.SaveChanges();` → inserta la orden y obtiene el Id. Línea 14: `var plancha1 = new PlanchaAcero { ... }` → crea la primera plancha con la clave foránea de la orden. Línea 15: `var plancha2 = new PlanchaAcero { ... }` → crea la segunda plancha con la misma clave foránea. Línea 17: `context.PlanchasAcero.AddRange(plancha1, plancha2);` → registra ambas planchas en una sola llamada. Línea 18: `var filas = context.SaveChanges();` → inserta ambas planchas y devuelve el número de filas afectadas. Línea 20: `Console.WriteLine($"Orden {orden.NumeroOrden} insertada con Id {orden.Id}");` →

muestra el número de orden. Línea 21: `Console.WriteLine($"Planchas insertadas: {filas}");` → muestra el número de planchas insertadas.

Paso 2: Llamar al método desde Main:

```
InsertarOrdenConPlanchas();
```

Paso 3: Ejecutar dotnet run y verificar que aparece la orden OF-003 con dos planchas insertadas.

Analogía final

El ciclo de vida del DbContext es como el turno de trabajo de un operario en una acería. El operario llega al inicio del turno, recibe las herramientas (el DbContext), realiza las tareas asignadas (consultas, inserciones, actualizaciones), anota los resultados en el libro de producción (Change Tracker) y al terminar el turno devuelve las herramientas y se marcha (Dispose). Si el operario se quedara en la planta durante días sin marcharse, acumularía trabajo pendiente, confundiría las órdenes antiguas con las nuevas y bloquearía recursos que otros operarios necesitan. Por eso cada turno es una unidad de trabajo independiente, y cada operario tiene su propio libro de producción. Así funciona el DbContext: vive poco, trabaja mucho y libera los recursos al terminar.

Resultado esperado

Al final del ejercicio, deberías haber:

Refactorizado el proyecto AceriaData con el patrón de unidad de trabajo.

Creado una fábrica manual de DbContext.

Observado el comportamiento del Change Tracker en distintos ciclos de vida.

Comprobado el problema de compartir un DbContext entre operaciones.

Verificado EstadoOrden, su DbSet y la migración AddEstadoOrden heredados de 1.4.

Aplicado la migración a SQL Server LocalDB.

Insertado una orden con dos planchas en una sola unidad de trabajo.

Conclusión y enlace al siguiente punto

En este punto se ha estudiado el ciclo de vida del DbContext en aplicaciones de consola, web y servicios. Se ha aplicado el patrón de unidad de trabajo, se ha creado una fábrica manual y se ha demostrado el problema de compartir un DbContext. En el siguiente punto se estudiarán los DbSet y las operaciones básicas de acceso a datos, con ejemplos de consulta, inserción, actualización y eliminación sobre el proyecto AceriaData.

Punto 1.6 - DbSet y operaciones básicas de acceso a datos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se añaden al proyecto AceriaData las operaciones básicas de acceso a datos sobre las entidades existentes, todas contra SQL Server LocalDB.

Ejercicio: Añadir al proyecto AceriaData un conjunto de métodos que ejecuten operaciones básicas sobre las entidades existentes: consulta, inserción, actualización, eliminación, búsqueda por clave y conteo. Cada operación se ejecuta en su propia unidad de trabajo.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
```

```

    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}

public static class AceriaDbContextFactory
{
    public static AceriaDbContext Create()
    {
        return new AceriaDbContext();
    }
}

public class Program
{
    public static void Main()
    {
        using (var context = AceriaDbContextFactory.Create())
        {
            context.Database.EnsureDeleted();
            context.Database.Migrate();

            InsertarOrden("OF-001", "Constructora del Norte");
            InsertarOrden("OF-002", "Constructora del Sur");
            InsertarOrden("OF-003", "Constructora del Este");

            ListarOrdenes();

            var ordenActualizada = ActualizarCliente("OF-002", "Constructora del Oeste");
            Console.WriteLine($"Actualizada: {ordenActualizada}");

            var existe = ExisteOrden("OF-003");
            Console.WriteLine($"Existe OF-003: {existe}");

            var total = ContarOrdenes();
            Console.WriteLine($"Total de órdenes: {total}");

            var encontrada = BuscarPorId(1);
            Console.WriteLine($"Orden con Id 1: {encontrada?.NumeroOrden}");

            EliminarOrden("OF-001");

            ListarOrdenes();
        }

        public static void InsertarOrden(string numero, string cliente)
        {
            using var context = AceriaDbContextFactory.Create();
            var orden = new OrdenFabricacion
            {
                NumeroOrden = numero,
                Cliente = cliente,
                FechaCreacion = DateTime.Now
            };
            context.OrdenesFabricacion.Add(orden);
            context.SaveChanges();
        }

        public static void ListarOrdenes()
        {
            using var context = AceriaDbContextFactory.Create();
            var ordenes = context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
            Console.WriteLine("--- Órdenes ---");
            foreach (var orden in ordenes)
            {
                Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");
            }
        }
    }
}

```

```

    }

    public static bool ActualizarCliente(string numero, string nuevoCliente)
    {
        using var context = AceriaDbContextFactory.Create();
        var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numero);
        if (orden is null)
        {
            return false;
        }
        orden.Cliente = nuevoCliente;
        context.SaveChanges();
        return true;
    }

    public static bool ExisteOrden(string numero)
    {
        using var context = AceriaDbContextFactory.Create();
        return context.OrdenesFabricacion.Any(o => o.NumeroOrden == numero);
    }

    public static int ContarOrdenes()
    {
        using var context = AceriaDbContextFactory.Create();
        return context.OrdenesFabricacion.Count();
    }

    public static OrdenFabricacion? BuscarPorId(int id)
    {
        using var context = AceriaDbContextFactory.Create();
        return context.OrdenesFabricacion.Find(id);
    }

    public static bool EliminarOrden(string numero)
    {
        using var context = AceriaDbContextFactory.Create();
        var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numero);
        if (orden is null)
        {
            return false;
        }
        context.OrdenesFabricacion.Remove(orden);
        context.SaveChanges();
        return true;
    }
}

```

Línea 1: using Microsoft.EntityFrameworkCore; → importa el espacio de nombres de EF Core. Línea 3: namespace AceriaData.ConsoleApp; → declara el espacio de nombres. Línea 5: public class OrdenFabricacion → entidad de orden. Línea 7: public int Id { get; set; } → clave primaria. Línea 8: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 9: public string Cliente { get; set; } = string.Empty; → cliente. Línea 10: public DateTime FechaCreacion { get; set; } → fecha de creación. Línea 13: public class PlanchaAcero → entidad de plancha. Línea 15: public int Id { get; set; } → clave primaria. Línea 16: public int OrdenId { get; set; } → clave foránea. Línea 17: public double Espesor { get; set; } → espesor. Línea 18: public double Ancho { get; set; } → ancho. Línea 19: public double Largo { get; set; } → largo. Línea 20: public OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación. Línea 23: public class Aleacion → entidad de aleación. Línea 25: public int Id { get; set; } → clave primaria. Línea 26: public string Nombre { get; set; } = string.Empty; → nombre. Línea 27: public double PorcentajeCarbono { get; set; } → porcentaje de carbono. Línea 28: public double PorcentajeManganeso { get; set; } → porcentaje de manganeso. Línea 31: public class EstadoOrden → entidad de estado. Línea 33: public int Id { get; set; } → clave primaria. Línea 34: public string Nombre { get; set; } = string.Empty; → nombre del estado. Línea 35: public string Descripcion { get; set; } = string.Empty; → descripción. Línea 38: public class AceriaDbContext : DbContext → declara el DbContext. Línea 40: public DbSet OrdenesFabricacion { get; set; } = null!; → DbSet de órdenes. Línea 41: public DbSet PlanchasAcero { get; set; } = null!; → DbSet de planchas. Línea 42: public DbSet Aleaciones { get; set; } = null!; → DbSet de aleaciones. Línea 43: public DbSet EstadosOrden { get; set; } = null!; → DbSet de estados. Línea 45: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) →

configuración. Línea 47: `optionsBuilder.UseSqlServer(` → registra el proveedor de SQL Server. Línea 48: `"Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"` → cadena de conexión. Línea 52: `public static class AceriaDbContextFactory` → fábrica manual. Línea 54: `public static AceriaDbContext Create()` → método que crea el contexto. Línea 56: `return new AceriaDbContext();` → devuelve una instancia nueva. Línea 60: `public class Program` → clase principal. Línea 62: `public static void Main()` → punto de entrada. Línea 64: `using (var context = AceriaDbContextFactory.Create())` → unidad de trabajo para recrear la base. Línea 66: `context.Database.EnsureDeleted();` → elimina la base. Línea 67: `context.Database.Migrate();` → crea la base. Línea 70: `InsertarOrden("OF-001", "Constructora del Norte");` → inserta la primera orden. Línea 71: `InsertarOrden("OF-002", "Constructora del Sur");` → inserta la segunda. Línea 72: `InsertarOrden("OF-003", "Constructora del Este");` → inserta la tercera. Línea 74: `ListarOrdenes();` → muestra las órdenes. Línea 76: `var ordenActualizada = ActualizarCliente("OF-002", "Constructora del Oeste");` → actualiza el cliente de OF-002. Línea 77: `Console.WriteLine($"Actualizada: {ordenActualizada}");` → muestra si se actualizó. Línea 79: `var existe = ExisteOrden("OF-003");` → comprueba si existe OF-003. Línea 80: `Console.WriteLine($"Existe OF-003: {existe}");` → muestra el resultado. Línea 82: `var total = ContarOrdenes();` → cuenta las órdenes. Línea 83: `Console.WriteLine($"Total de órdenes: {total}");` → muestra el total. Línea 85: `var encontrada = BuscarPorId(1);` → busca la orden con Id 1. Línea 86: `Console.WriteLine($"Orden con Id 1: {encontrada?.NumeroOrden}");` → muestra el número o null. Línea 88: `EliminarOrden("OF-001");` → elimina OF-001. Línea 90: `ListarOrdenes();` → vuelve a listar. Línea 93: `public static void InsertarOrden(string numero, string cliente)` → método de inserción. Línea 95: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 96: `var orden = new OrdenFabricacion` → crea la entidad. Línea 102: `context OrdenesFabricacion.Add(orden);` → registra la entidad. Línea 103: `context.SaveChanges();` → ejecuta el INSERT. Línea 106: `public static void ListarOrdenes()` → método de listado. Línea 108: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 109: `var ordenes = context OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → consulta ordenada. Línea 110: `Console.WriteLine("--- Órdenes ---");` → separador. Línea 111: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 113: `Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");` → muestra los datos. Línea 117: `public static bool ActualizarCliente(string numero, string nuevoCliente)` → método de actualización. Línea 119: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 120: `var orden = context OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numero);` → busca la orden. Línea 121: `if (orden is null)` → comprueba si no existe. Línea 123: `return false;` → devuelve falso si no existe. Línea 125: `orden.Cliente = nuevoCliente;` → modifica la propiedad. Línea 126: `context.SaveChanges();` → ejecuta el UPDATE. Línea 127: `return true;` → devuelve verdadero. Línea 130: `public static bool ExisteOrden(string numero)` → método de comprobación. Línea 132: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 133: `return context OrdenesFabricacion.Any(o => o.NumeroOrden == numero);` → devuelve true o false. Línea 136: `public static int ContarOrdenes()` → método de conteo. Línea 138: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 139: `return context OrdenesFabricacion.Count();` → devuelve el número de órdenes. Línea 142: `public static OrdenFabricacion? BuscarPorId(int id)` → método de búsqueda por Id. Línea 144: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 145: `return context OrdenesFabricacion.Find(id);` → busca por clave primaria. Línea 148: `public static bool EliminarOrden(string numero)` → método de eliminación. Línea 150: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 151: `var orden = context OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numero);` → busca la orden. Línea 152: `if (orden is null)` → comprueba si existe. Línea 154: `return false;` → devuelve falso. Línea 156: `context OrdenesFabricacion.Remove(orden);` → marca para eliminar. Línea 157: `context.SaveChanges();` → ejecuta el DELETE. Línea 158: `return true;` → devuelve verdadero.

Error común: si se olvida llamar a `SaveChanges` después de `Add`, `Update` o `Remove`, los cambios no se persisten en la base de datos.

Paso 3: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las tres órdenes iniciales, el resultado de la actualización, la comprobación de existencia, el total de órdenes, la búsqueda por Id y, tras la eliminación, las dos órdenes restantes.

Paso 4: Observar la consulta con AsNoTracking

Modificar el método ListarOrdenes para usar AsNoTracking:

```
public static void ListarOrdenes()
{
    using var context = AceriaDbContextFactory.Create();
    var ordenes = context.OrdenesFabricacion.AsNoTracking().OrderBy(o => o.Id).ToList();
    Console.WriteLine("--- Órdenes ---");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");
    }
    Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
}
```

Línea 109: var ordenes = context.OrdenesFabricacion.AsNoTracking().OrderBy(o => o.Id).ToList(); → ejecuta la consulta sin registrar entidades en el Change Tracker. Línea 114:

Console.WriteLine(\$"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}"); → muestra el número de entidades rastreadas, que es cero.

Resultado esperado: aparece Entidades rastreadas: 0 tras el listado.

Paso 5: Ejecutar el proyecto

```
dotnet run
```

Resultado esperado: además de las órdenes, aparece Entidades rastreadas: 0, confirmando que AsNoTracking no registra entidades.

Paso 6: Observar la caché de identidad con Find

Modificar el método Main para llamar a Find dos veces sobre la misma entidad en el mismo contexto:

```
public static void Main()
{
    using (var context = AceriaDbContextFactory.Create())
    {
        context.Database.EnsureDeleted();
        context.Database.Migrate();
    }

    InsertarOrden("OF-001", "Constructora del Norte");

    using (var context = AceriaDbContextFactory.Create())
    {
        var orden1 = context.OrdenesFabricacion.Find(1);
        var orden2 = context.OrdenesFabricacion.Find(1);

        Console.WriteLine($"Misma instancia: {ReferenceEquals(orden1, orden2)}");
    }
}
```

Línea 64: using (var context = AceriaDbContextFactory.Create()) → unidad de trabajo para recrear la base. Línea 66: context.Database.EnsureDeleted(); → elimina la base. Línea 67: context.Database.Migrate(); → crea la base. Línea 70: InsertarOrden("OF-001", "Constructora del Norte"); → inserta una orden. Línea 72: using (var context = AceriaDbContextFactory.Create()) → unidad de trabajo para la consulta. Línea 74: var orden1 = context.OrdenesFabricacion.Find(1); → primera

búsqueda, carga desde la base. Línea 75: `var orden2 = context.OrdenesFabricacion.Find(1);` → segunda búsqueda, devuelve desde la caché. Línea 77: `Console.WriteLine($"Misma instancia: {ReferenceEquals(orden1, orden2)}");` → comprueba si son la misma instancia.

Resultado esperado: aparece `Misma instancia: True`, confirmando que la caché de identidad devuelve la misma instancia.

Paso 7: Ejecutar el proyecto

```
dotnet run
```

Resultado esperado: aparece `Misma instancia: True`.

Paso 8: Diagnosticar un error común

Modificar el método `EliminarOrden` para eliminar una entidad que no existe:

```
public static bool EliminarOrden(string numero)
{
    using var context = AceriaDbContextFactory.Create();
    var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numero);
    if (orden is null)
    {
        return false;
    }
    context.OrdenesFabricacion.Remove(orden);
    context.SaveChanges();
    return true;
}
```

Si se llama a `EliminarOrden("OF-999")`, el método devuelve `false` sin lanzar excepción. Si se llamara a `Remove` con `null`, se lanzaría una excepción `ArgumentNullException`.

Error común: intentar eliminar una entidad que no existe sin comprobar el resultado de `FirstOrDefault`.

Paso 9: Verificar las operaciones en la base de datos

Abrir Visual Studio. En `Ver` → `Explorador de objetos de SQL Server`. Expandir `(localdb)\MSSQLLocalDB` → `Bases de datos` → `AceriaDB` → `Tablas` → `dbo.OrdenesFabricacion`. Clic derecho → `Ver datos`. Comprobar que aparecen las órdenes insertadas y que la orden `OF-001` ya no está tras la eliminación.

Resultado esperado: la tabla contiene `OF-002` y `OF-003`, pero no `OF-001`.

Errores comunes del ejercicio completo

Error	Causa	Solución
Los cambios no se guardan	Falta <code>SaveChanges</code>	Añadir <code>context.SaveChanges()</code>
<code>Remove</code> con entidad nula	No se comprobó <code>FirstOrDefault</code>	Añadir comprobación <code>is null</code>
Actualización no aplicada	Entidad no rastreada	Usar <code>Update</code> o cargar la entidad antes
Consulta devuelve datos obsoletos	Caché de identidad	Usar un contexto nuevo
<code>Find</code> devuelve <code>null</code>	La clave no existe	Comprobar el resultado antes de usar
<code>AsNoTracking</code> no aplicado	Se olvidó la llamada	Añadir <code>.AsNoTracking()</code> antes de materializar

Reto resuelto: Insertar una plancha asociada a una orden existente

Reto: Crear un método que inserte una plancha asociada a una orden existente por su número de orden. El método debe devolver true si la inserción se realizó correctamente y false si la orden no existe.

Solución paso a paso

Paso 1: Añadir el método InsertarPlancha:

```
public static bool InsertarPlancha(string numeroOrden, double espesor, double ancho, double largo)
{
    using var context = AceriaDbContextFactory.Create();

    var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    if (orden is null)
    {
        return false;
    }

    var plancha = new PlanchaAcero
    {
        OrdenId = orden.Id,
        Espesor = espesor,
        Ancho = ancho,
        Largo = largo
    };

    context.PlanchasAcero.Add(plancha);
    context.SaveChanges();
    return true;
}
```

Línea 1: public static bool InsertarPlancha(string numeroOrden, double espesor, double ancho, double largo) → declara el método con los parámetros de la plancha. Línea 3: using var context = AceriaDbContextFactory.Create(); → crea la unidad de trabajo. Línea 5: var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden); → busca la orden por número. Línea 6: if (orden is null) → comprueba si existe. Línea 8: return false; → devuelve falso si no existe. Línea 11: var plancha = new PlanchaAcero → crea la plancha. Línea 13: OrdenId = orden.Id, → asigna la clave foránea con el Id de la orden. Línea 14: Espesor = espesor, → asigna el espesor. Línea 15: Ancho = ancho, → asigna el ancho. Línea 16: Largo = largo → asigna el largo. Línea 19: context.PlanchasAcero.Add(plancha); → registra la plancha. Línea 20: context.SaveChanges(); → ejecuta el INSERT. Línea 21: return true; → devuelve verdadero.

Paso 2: Llamar al método desde Main:

```
var insertada = InsertarPlancha("0F-002", 10.5, 1500, 3000);
Console.WriteLine($"Plancha insertada: {insertada}");
```

Paso 3: Ejecutar dotnet run y verificar que la plancha se inserta correctamente.

Analogía final

Los DbSet son como los cajones de una acería donde se guardan las órdenes y las planchas. Cada cajón tiene su propio tipo de contenido: uno guarda órdenes, otro guarda planchas, otro guarda aleaciones. El operario puede abrir un cajón y consultar su contenido (ToList), buscar una pieza concreta (FirstOrDefault), añadir una nueva (Add), modificar una existente (Update) o retirar una que ya no sirve (Remove). Todas estas operaciones se anotan en el libro de producción (Change Tracker), pero ninguna se hace efectiva hasta que el operario firma el parte de trabajo (SaveChanges). Si el operario necesita solo consultar sin modificar, puede usar el modo de solo lectura (AsNoTracking), que no anota nada en el

libro. Así funcionan los DbSet: son la puerta de acceso a los datos, y cada operación tiene su propósito concreto.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido métodos de inserción, consulta, actualización y eliminación sobre las entidades del proyecto.

Ejecutado cada operación en su propia unidad de trabajo.

Observado el comportamiento de AsNoTracking y de la caché de identidad.

Diagnosticado errores comunes de operaciones sobre DbSet.

Insertado una plancha asociada a una orden existente.

Verificado los datos en el Explorador de objetos de SQL Server.

| Conclusión y enlace al siguiente punto

En este punto se han estudiado las operaciones básicas de acceso a datos sobre un DbSet: consulta, inserción, actualización, eliminación, búsqueda por clave y conteo. Se ha observado el comportamiento de AsNoTracking y de la caché de identidad. En el siguiente punto se estudiará en detalle el Change Tracker, sus estados y cómo detecta los cambios en las entidades del proyecto AceriaData.

Punto 1.7 - Change Tracker: estados de las entidades y detección de cambios

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se profundiza en el Change Tracker del proyecto AceriaData, observando cómo detecta cambios en las entidades y cómo se pueden inspeccionar y manipular sus estados, siempre contra SQL Server LocalDB.

Ejercicio: Añadir al proyecto AceriaData un conjunto de métodos que inspeccionen el Change Tracker antes y después de las operaciones, que observen la detección de cambios, que manipulen estados manualmente y que demuestren el comportamiento de Update, Attach y AsNoTracking.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
```

```

    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}

public static class AceriaDbContextFactory
{
    public static AceriaDbContext Create()
    {
        return new AceriaDbContext();
    }
}

public class Program
{
    public static void Main()
    {
        using (var context = AceriaDbContextFactory.Create())
        {
            context.Database.EnsureDeleted();
            context.Database.Migrate();
        }

        InsertarOrden("OF-001", "Constructora del Norte");
        InsertarOrden("OF-002", "Constructora del Sur");

        ObservarEstados();
        ObservarDeteccionDeCambios();
        ObservarValoresOriginales();
        ObservarEntidadesRastreadas();
        DemostrarUpdate();
        DemostrarAttach();
        DemostrarAsNoTracking();
        DemostrarClear();
    }

    public static void InsertarOrden(string numero, string cliente)
    {
        using var context = AceriaDbContextFactory.Create();
        var orden = new OrdenFabricacion
        {
            NumeroOrden = numero,
            Cliente = cliente,
            FechaCreacion = DateTime.Now
        };
        context OrdenesFabricacion.Add(orden);
        context.SaveChanges();
    }

    public static void ObservarEstados()
    {
        using var context = AceriaDbContextFactory.Create();

        var nueva = new OrdenFabricacion { NumeroOrden = "OF-003", Cliente = "Constructora del Este",
FechaCreacion = DateTime.Now };
        Console.WriteLine($"Estado de entidad nueva: {context.Entry(nueva).State}");

        context OrdenesFabricacion.Add(nueva);
        Console.WriteLine($"Estado tras Add: {context.Entry(nueva).State}");

        context.SaveChanges();
        Console.WriteLine($"Estado tras SaveChanges: {context.Entry(nueva).State}");

        context OrdenesFabricacion.Remove(nueva);
        Console.WriteLine($"Estado tras Remove: {context.Entry(nueva).State}");
    }
}

```

```

public static void ObservarDeteccionDeCambios()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001")!;
    Console.WriteLine($"Estado inicial: {context.Entry(orden).State}");

    orden.Cliente = "Constructora del Oeste";
    context.ChangeTracker.DetectChanges();
    Console.WriteLine($"Estado tras modificar Cliente: {context.Entry(orden).State}");
}

public static void ObservarValoresOriginales()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001")!;
    orden.Cliente = "Constructora del Oeste";

    var entry = context.Entry(orden);
    Console.WriteLine($"Cliente actual: {entry.CurrentValues["Cliente"]}");
    Console.WriteLine($"Cliente original: {entry.OriginalValues["Cliente"]}");
    Console.WriteLine($"¿Cliente modificado?: {entry.Property(o => o.Cliente).IsModified}");
}

public static void ObservarEntidadesRastreadas()
{
    using var context = AceriaDbContextFactory.Create();

    context.OrdenesFabricacion.ToList();
    context.Aleaciones.ToList();

    Console.WriteLine("Entidades rastreadas:");
    foreach (var entry in context.ChangeTracker.Entries())
    {
        Console.WriteLine($" {entry.Entity.GetType().Name}: {entry.State}");
    }
}

public static void DemostrarUpdate()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        Id = 1,
        NumeroOrden = "OF-001",
        Cliente = "Constructora del Norte Actualizada",
        FechaCreacion = DateTime.Now
    };

    context.OrdenesFabricacion.Update(orden);
    Console.WriteLine($"Estado tras Update: {context.Entry(orden).State}");

    context.SaveChanges();
    Console.WriteLine($"Estado tras SaveChanges: {context.Entry(orden).State}");
}

public static void DemostrarAttach()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        Id = 2,
        NumeroOrden = "OF-002",
        Cliente = "Constructora del Sur",
        FechaCreacion = DateTime.Now
    };

    context.OrdenesFabricacion.Attach(orden);
    Console.WriteLine($"Estado tras Attach: {context.Entry(orden).State}");
}

```

```

        orden.Cliente = "Constructora del Sur Actualizada";
        context.ChangeTracker.DetectChanges();
        Console.WriteLine($"Estado tras modificar y detectar: {context.Entry(orden).State}");

        context.SaveChanges();
        Console.WriteLine($"Estado tras SaveChanges: {context.Entry(orden).State}");
    }

    public static void DemostrarAsNoTracking()
    {
        using var context = AceriaDbContextFactory.Create();

        var ordenes = context.OrdenesFabricacion.AsNoTracking().ToList();
        Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");
        Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
    }

    public static void DemostrarClear()
    {
        using var context = AceriaDbContextFactory.Create();

        context.OrdenesFabricacion.ToList();
        Console.WriteLine($"Entidades rastreadas antes de Clear:
{context.ChangeTracker.Entries().Count()}");

        context.ChangeTracker.Clear();
        Console.WriteLine($"Entidades rastreadas después de Clear:
{context.ChangeTracker.Entries().Count()}");
    }
}

```

Línea 1: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres de EF Core. Línea 3: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres. Línea 5: `public class OrdenFabricacion` → entidad de orden. Línea 7: `public int Id { get; set; }` → clave primaria. Línea 8: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden. Línea 9: `public string Cliente { get; set; } = string.Empty;` → cliente. Línea 10: `public DateTime FechaCreacion { get; set; }` → fecha de creación. Línea 13: `public class PlanchaAcero` → entidad de plancha. Línea 15: `public int Id { get; set; }` → clave primaria. Línea 16: `public int OrdenId { get; set; }` → clave foránea. Línea 17: `public double Espesor { get; set; }` → espesor. Línea 18: `public double Ancho { get; set; }` → ancho. Línea 19: `public double Largo { get; set; }` → largo. Línea 20: `public OrdenFabricacion Orden { get; set; } = null!;` → propiedad de navegación. Línea 23: `public class Aleacion` → entidad de aleación. Línea 25: `public int Id { get; set; }` → clave primaria. Línea 26: `public string Nombre { get; set; } = string.Empty;` → nombre. Línea 27: `public double PorcentajeCarbono { get; set; }` → porcentaje de carbono. Línea 28: `public double PorcentajeManganeso { get; set; }` → porcentaje de manganeso. Línea 31: `public class EstadoOrden` → entidad de estado. Línea 33: `public int Id { get; set; }` → clave primaria. Línea 34: `public string Nombre { get; set; } = string.Empty;` → nombre del estado. Línea 35: `public string Descripcion { get; set; } = string.Empty;` → descripción. Línea 38: `public class AceriaDbContext : DbContext` → declara el DbContext. Línea 40: `public DbSet OrdenesFabricacion { get; set; } = null!;` → DbSet de órdenes. Línea 41: `public DbSet PlanchasAcero { get; set; } = null!;` → DbSet de planchas. Línea 42: `public DbSet Aleaciones { get; set; } = null!;` → DbSet de aleaciones. Línea 43: `public DbSet EstadosOrden { get; set; } = null!;` → DbSet de estados. Línea 45: `protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)` → configuración. Línea 47: `optionsBuilder.UseSqlServer(` → registra el proveedor de SQL Server. Línea 48: `"Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"` → cadena de conexión. Línea 52: `public static class AceriaDbContextFactory` → fábrica manual. Línea 54: `public static AceriaDbContext Create()` → método que crea el contexto. Línea 56: `return new AceriaDbContext();` → devuelve una instancia nueva. Línea 60: `public class Program` → clase principal. Línea 62: `public static void Main()` → punto de entrada. Línea 64: `using (var context = AceriaDbContextFactory.Create())` → unidad de trabajo para recrear la base. Línea 66: `context.Database.EnsureDeleted();` → elimina la base. Línea 67: `context.Database.Migrate();` → crea la base. Línea 70: `InsertarOrden("OF-001", "Constructora del Norte");` → inserta la primera orden. Línea 71: `InsertarOrden("OF-002", "Constructora del Sur");` → inserta la segunda orden. Línea 73: `ObservarEstados();` → llama al método que observa los estados. Línea 74: `ObservarDeteccionDeCambios();` → llama al método que observa la detección de cambios. Línea 75: `ObservarValoresOriginales();` → llama al método que observa los valores originales. Línea 76:

ObservarEntidadesRastreadas(); → llama al método que observa las entidades rastreadas. Línea 77: DemostrarUpdate(); → llama al método que demuestra Update. Línea 78: DemostrarAttach(); → llama al método que demuestra Attach. Línea 79: DemostrarAsNoTracking(); → llama al método que demuestra AsNoTracking. Línea 80: DemostrarClear(); → llama al método que demuestra Clear. Línea 83: public static void InsertarOrden(string numero, string cliente) → método de inserción. Línea 85: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 86: var orden = new OrdenFabricacion → crea la entidad. Línea 92: context.OrdenesFabricacion.Add(orden); → registra la entidad. Línea 93: context.SaveChanges(); → ejecuta el INSERT. Línea 96: public static void ObservarEstados() → método que observa los estados. Línea 98: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 100: var nueva = new OrdenFabricacion { ... }; → crea una entidad nueva. Línea 101: Console.WriteLine(\$"Estado de entidad nueva: {context.Entry(nueva).State}"); → muestra el estado inicial, que es Detached. Línea 103: context.OrdenesFabricacion.Add(nueva); → registra la entidad. Línea 104: Console.WriteLine(\$"Estado tras Add: {context.Entry(nueva).State}"); → muestra el estado Added. Línea 106: context.SaveChanges(); → ejecuta el INSERT. Línea 107: Console.WriteLine(\$"Estado tras SaveChanges: {context.Entry(nueva).State}"); → muestra el estado Unchanged. Línea 109: context.OrdenesFabricacion.Remove(nueva); → marca la entidad para eliminar. Línea 110: Console.WriteLine(\$"Estado tras Remove: {context.Entry(nueva).State}"); → muestra el estado Deleted. Línea 113: public static void ObservarDeteccionDeCambios() → método que observa la detección de cambios. Línea 115: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 117: var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001"); → carga la orden. Línea 118: Console.WriteLine(\$"Estado inicial: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 120: orden.Cliente = "Constructora del Oeste"; → modifica la propiedad. Línea 121: context.ChangeTracker.DetectChanges(); → fuerza la detección de cambios. Línea 122: Console.WriteLine(\$"Estado tras modificar Cliente: {context.Entry(orden).State}"); → muestra el estado Modified. Línea 125: public static void ObservarValoresOriginales() → método que observa los valores originales. Línea 127: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 129: var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001"); → carga la orden. Línea 130: orden.Cliente = "Constructora del Oeste"; → modifica la propiedad. Línea 132: var entry = context.Entry(orden); → obtiene el EntityEntry. Línea 133: Console.WriteLine(\$"Cliente actual: {entry.CurrentValues["Cliente"]}"); → muestra el valor actual. Línea 134: Console.WriteLine(\$"Cliente original: {entry.OriginalValues["Cliente"]}"); → muestra el valor original. Línea 135: Console.WriteLine(\$"¿Cliente modificado?: {entry.Property(o => o.Cliente).IsModified}"); → muestra si la propiedad está modificada. Línea 138: public static void ObservarEntidadesRastreadas() → método que observa las entidades rastreadas. Línea 140: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 142: context.OrdenesFabricacion.ToList(); → carga las órdenes. Línea 143: context.Aleaciones.ToList(); → carga las aleaciones. Línea 145: Console.WriteLine("Entidades rastreadas:"); → cabecera. Línea 146: foreach (var entry in context.ChangeTracker.Entries()) → itera sobre las entidades rastreadas. Línea 148: Console.WriteLine(\$" {entry.Entity.GetType().Name}: {entry.State}"); → muestra el tipo y el estado. Línea 152: public static void DemostrarUpdate() → método que demuestra Update. Línea 154: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 156: var orden = new OrdenFabricacion { ... }; → crea una entidad desconectada. Línea 163: context.OrdenesFabricacion.Update(orden); → marca la entidad como Modified. Línea 164: Console.WriteLine(\$"Estado tras Update: {context.Entry(orden).State}"); → muestra el estado Modified. Línea 166: context.SaveChanges(); → ejecuta el UPDATE. Línea 167: Console.WriteLine(\$"Estado tras SaveChanges: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 171: public static void DemostrarAttach() → método que demuestra Attach. Línea 173: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 175: var orden = new OrdenFabricacion { ... }; → crea una entidad desconectada. Línea 182: context.OrdenesFabricacion.Attach(orden); → registra la entidad con estado Unchanged. Línea 183: Console.WriteLine(\$"Estado tras Attach: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 185: orden.Cliente = "Constructora del Sur Actualizada"; → modifica la propiedad. Línea 186: context.ChangeTracker.DetectChanges(); → fuerza la detección de cambios. Línea 187: Console.WriteLine(\$"Estado tras modificar y detectar: {context.Entry(orden).State}"); → muestra el estado Modified. Línea 189: context.SaveChanges(); → ejecuta el UPDATE. Línea 190: Console.WriteLine(\$"Estado tras SaveChanges: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 194: public static void DemostrarAsNoTracking() → método que demuestra

AsNoTracking. Línea 196: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 198: `var ordenes = context.OrdenesFabricacion.AsNoTracking().ToList();` → carga las órdenes sin registrarlas. Línea 199: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes cargadas. Línea 200: `Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");` → muestra el número de entidades rastreadas, que es cero. Línea 204: `public static void DemostrarClear()` → método que demuestra Clear. Línea 206: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 208: `context.OrdenesFabricacion.ToList();` → carga las órdenes. Línea 209: `Console.WriteLine($"Entidades rastreadas antes de Clear: {context.ChangeTracker.Entries().Count()}");` → muestra el número de entidades rastreadas. Línea 211: `context.ChangeTracker.Clear();` → desvincula todas las entidades. Línea 212: `Console.WriteLine($"Entidades rastreadas después de Clear: {context.ChangeTracker.Entries().Count()}");` → muestra el número de entidades rastreadas, que es cero.

Error común: si se modifica una propiedad de una entidad en estado Detached, el Change Tracker no lo detecta porque la entidad no está registrada. Es necesario llamar a Attach o Update para que el cambio se detecte.

Paso 3: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen los estados de la entidad nueva (Detached, Added, Unchanged, Deleted), el estado inicial Unchanged y el estado Modified tras modificar la propiedad, los valores actual y original, las entidades rastreadas, y los resultados de Update, Attach, AsNoTracking y Clear.

Paso 4: Observar el cambio de estado en el depurador

Abrir el proyecto en Visual Studio Code. Colocar un punto de interrupción en la línea `orden.Cliente = "Constructora del Oeste";` del método `ObservarDeteccionDeCambios`. Ejecutar el proyecto en modo depuración. Cuando se detenga, inspeccionar la variable `orden` y comprobar que su estado es Unchanged. Avanzar una línea y comprobar que el estado cambia a Modified después de llamar a `DetectChanges`.

Resultado esperado: el depurador muestra el cambio de estado de la entidad.

Paso 5: Verificar los cambios en la base de datos

Abrir Visual Studio. En Ver → Explorador de objetos de SQL Server. Expandir (localdb)\MSSQLLocalDB → Bases de datos → AceriaDB → Tablas → `dbo.OrdenesFabricacion`. Clic derecho → Ver datos. Comprobar que los clientes de OF-001 y OF-002 han sido actualizados.

Resultado esperado: la tabla muestra los valores actualizados.

Paso 6: Diagnosticar un error común

Modificar el método `DemostrarAttach` para modificar una propiedad antes de llamar a `Attach`:

```
public static void DemostrarAttach()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        Id = 2,
        NumeroOrden = "OF-002",
        Cliente = "Constructora del Sur Modificada Antes",
        FechaCreacion = DateTime.Now
    };

    orden.Cliente = "Constructora del Sur Actualizada";
    context.OrdenesFabricacion.Attach(orden);
    Console.WriteLine($"Estado tras Attach: {context.Entry(orden).State}");

    context.ChangeTracker.DetectChanges();
    Console.WriteLine($"Estado tras detectar cambios: {context.Entry(orden).State}");
}
```

Resultado esperado: tras Attach, el estado es Unchanged aunque la propiedad se haya modificado antes. Al llamar a DetectChanges, el estado sigue siendo Unchanged porque el valor original que se guarda al hacer Attach es el valor actual en ese momento. Los cambios anteriores a Attach no se detectan.

Solución: modificar la propiedad después de Attach, no antes.

Errores comunes del ejercicio completo

Error	Causa	Solución
Cambios no detectados	Entidad en estado Detached	Llamar a Attach o Update antes de modificar
Estado no cambia tras modificar	No se llamó a DetectChanges	Llamar a DetectChanges o a SaveChanges
Valores originales incorrectos	Se modificó antes de Attach	Modificar después de Attach
Update genera UPDATE completo	Entidad desconectada	Cargar la entidad o usar Attach + modificar
Entidades rastreadas excesivas	DbContext compartido	Usar instancias cortas o Clear
AsNoTracking no funciona	Se olvidó la llamada	Añadir .AsNoTracking() antes de materializar

Reto resuelto: Detectar y reportar cambios antes de guardar

Reto: Crear un método que cargue todas las órdenes, modifique algunas propiedades, detecte los cambios y reporte cuántas entidades están en estado Modified antes de guardar.

Solución paso a paso

Paso 1: Añadir el método ReportarCambios:

```
public static void ReportarCambios()
{
    using var context = AceriaDbContextFactory.Create();

    var ordenes = context.OrdenesFabricacion.ToList();
    Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");

    foreach (var orden in ordenes)
    {
        if (orden.NumeroOrden == "OF-001")
        {
            orden.Cliente = "Constructora del Norte Modificada";
        }
        if (orden.NumeroOrden == "OF-002")
        {
            orden.Cliente = "Constructora del Sur Modificada";
        }
    }

    context.ChangeTracker.DetectChanges();

    var modificadas = context.ChangeTracker.Entries()
        .Where(e => e.State == EntityState.Modified)
        .ToList();

    Console.WriteLine($"Entidades modificadas: {modificadas.Count}");

    foreach (var entry in modificadas)
    {
        var orden = (OrdenFabricacion)entry.Entity;
        Console.WriteLine($" {orden.NumeroOrden}: {entry.OriginalValues["Cliente"]} → {entry.CurrentValues["Cliente"]}");
    }

    context.SaveChanges();
    Console.WriteLine("Cambios guardados.");
}
```

Línea 1: `public static void ReportarCambios()` → declara el método. Línea 3: `using var context = AceriaDbContextFactory.Create();` → crea la unidad de trabajo. Línea 5: `var ordenes = context.OrdenesFabricacion.ToList();` → carga todas las órdenes. Línea 6: `Console.WriteLine($"Órdenes cargadas: {ordenes.Count}");` → muestra el número de órdenes cargadas. Línea 8: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 10: `if (orden.NumeroOrden == "OF-001")` → comprueba si es la primera orden. Línea 12: `orden.Cliente = "Constructora del Norte Modificada";` → modifica el cliente. Línea 14: `if (orden.NumeroOrden == "OF-002")` → comprueba si es la segunda orden. Línea 16: `orden.Cliente = "Constructora del Sur Modificada";` → modifica el cliente. Línea 20: `context.ChangeTracker.DetectChanges();` → fuerza la detección de cambios. Línea 22: `var modificadas = context.ChangeTracker.Entries()` → obtiene las entidades rastreadas. Línea 23: `.Where(e => e.State == EntityState.Modified)` → filtra las modificadas. Línea 24: `.ToList();` → materializa la lista. Línea 26: `Console.WriteLine($"Entidades modificadas: {modificadas.Count}");` → muestra el número. Línea 28: `foreach (var entry in modificadas)` → itera sobre las entidades modificadas. Línea 30: `var orden = (OrdenFabricacion)entry.Entity;` → convierte la entidad al tipo correcto. Línea 31: `Console.WriteLine($" {orden.NumeroOrden}: {entry.OriginalValues["Cliente"]} → {entry.CurrentValues["Cliente"]}");` → muestra el valor original y el actual. Línea 34: `context.SaveChanges();` → guarda los cambios. Línea 35: `Console.WriteLine("Cambios guardados.");` → confirma el guardado.

Paso 2: Llamar al método desde Main:

```
ReportarCambios();
```

Paso 3: Ejecutar dotnet run y verificar que se reportan dos entidades modificadas con sus valores originales y actuales.

Analogía final

El Change Tracker es como el supervisor de calidad de una acería que lleva un registro detallado de cada plancha que pasa por la línea de producción. Cuando una plancha llega a la línea, el supervisor la anota en su libro con su estado inicial (Unchanged). Si la plancha se modifica, el supervisor lo detecta comparando el estado actual con el que anotó al principio y marca la plancha como Modified. Si se añade una plancha nueva, la marca como Added. Si se retira una plancha, la marca como Deleted. Cuando el jefe de planta firma el parte de trabajo (SaveChanges), el supervisor entrega al operario la lista de tareas: insertar las planchas nuevas, actualizar las modificadas y eliminar las retiradas. El supervisor no fabrica nada él mismo, pero sin su registro sería imposible saber qué ha cambiado y qué debe hacerse. Así funciona el Change Tracker: observa, registra y reporta, para que SaveChanges sepa exactamente qué hacer.

Resultado esperado

Al final del ejercicio, deberías haber:

Observado los cinco estados de una entidad.

Comprendido cómo el Change Tracker detecta cambios.

Inspeccionado los valores actuales y originales.

Listado las entidades rastreadas con sus estados.

Demostrado el comportamiento de Update, Attach, AsNoTracking y Clear.

Diagnosticado errores comunes de detección de cambios.

Reportado cambios antes de guardar.

Conclusión y enlace al siguiente punto

En este punto se ha estudiado en detalle el Change Tracker: sus cinco estados, la detección de cambios, la inspección de valores actuales y originales, y los métodos para manipular estados. En el siguiente punto se estudiará la gestión de entidades: Add, Update, Remove, Attach y Entry, con sus implicaciones en el Change Tracker y en las operaciones contra la base de datos.

Punto 1.8 - Gestión de entidades: Add, Update, Remove, Attach y Entry

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se profundiza en la gestión de entidades del proyecto AceriaData, utilizando los métodos Add, Update, Remove, Attach y Entry sobre las entidades existentes, siempre contra SQL Server LocalDB.

Ejercicio: Añadir al proyecto AceriaData un conjunto de métodos que demuestren el comportamiento de Add, Update, Remove, Attach y Entry, comparando los SQL generados y observando los estados de las entidades en cada caso.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
```

```

    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}

public static class AceriaDbContextFactory
{
    public static AceriaDbContext Create()
    {
        return new AceriaDbContext();
    }
}

public class Program
{
    public static void Main()
    {
        using (var context = AceriaDbContextFactory.Create())
        {
            context.Database.EnsureDeleted();
            context.Database.Migrate();
        }

        InsertarOrdenesIniciales();

        DemostrarAdd();
        DemostrarAddRange();
        DemostrarUpdate();
        DemostrarAttach();
        DemostrarEntry();
        DemostrarRemove();
        DemostrarEntryProperty();
    }

    public static void InsertarOrdenesIniciales()
    {
        using var context = AceriaDbContextFactory.Create();
        var ordenes = new List<OrdenFabricacion>
        {
            new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "OF-002", Cliente = "Constructora del Sur",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "OF-003", Cliente = "Constructora del Este",
            FechaCreacion = DateTime.Now }
        };
        context.OrdenesFabricacion.AddRange(ordenes);
        context.SaveChanges();
    }

    public static void DemostrarAdd()
    {
        using var context = AceriaDbContextFactory.Create();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-004",
            Cliente = "Constructora del Oeste",
            FechaCreacion = DateTime.Now
        };

        Console.WriteLine($"Estado antes de Add: {context.Entry(orden).State}");
        context.OrdenesFabricacion.Add(orden);
        Console.WriteLine($"Estado tras Add: {context.Entry(orden).State}");
        context.SaveChanges();
        Console.WriteLine($"Estado tras SaveChanges: {context.Entry(orden).State}");
    }
}

```

```

public static void DemostrarAddRange()
{
    using var context = AceriaDbContextFactory.Create();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-005", Cliente = "Constructora del Norte",
FechaCreacion = DateTime.Now },
        new OrdenFabricacion { NumeroOrden = "OF-006", Cliente = "Constructora del Sur",
FechaCreacion = DateTime.Now }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    Console.WriteLine($"Estado tras AddRange: {context.ChangeTracker.Entries().Count()} entidades
rastreadas");
    context.SaveChanges();
    Console.WriteLine($"Órdenes insertadas: {ordenes.Count}");
}

public static void DemostrarUpdate()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        Id = 1,
        NumeroOrden = "OF-001",
        Cliente = "Constructora del Norte Actualizada con Update",
        FechaCreacion = DateTime.Now
    };

    context.OrdenesFabricacion.Update(orden);
    Console.WriteLine($"Estado tras Update: {context.Entry(orden).State}");
    Console.WriteLine($"SQL generado: {context.OrdenesFabricacion.Where(o => o.Id ==
1).ToQueryString()}");
    context.SaveChanges();
    Console.WriteLine($"Estado tras SaveChanges: {context.Entry(orden).State}");
}

public static void DemostrarAttach()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        Id = 2,
        NumeroOrden = "OF-002",
        Cliente = "Constructora del Sur",
        FechaCreacion = DateTime.Now
    };

    context.OrdenesFabricacion.Attach(orden);
    Console.WriteLine($"Estado tras Attach: {context.Entry(orden).State}");

    orden.Cliente = "Constructora del Sur Actualizada con Attach";
    context.ChangeTracker.DetectChanges();
    Console.WriteLine($"Estado tras modificar: {context.Entry(orden).State}");
    context.SaveChanges();
    Console.WriteLine($"Estado tras SaveChanges: {context.Entry(orden).State}");
}

public static void DemostrarEntry()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        Id = 3,
        NumeroOrden = "OF-003",
        Cliente = "Constructora del Este",
        FechaCreacion = DateTime.Now
    };
};

```

```

        var entry = context.Entry(orden);
        Console.WriteLine($"Estado inicial: {entry.State}");

        entry.State = EntityState.Modified;
        Console.WriteLine($"Estado tras cambiar a Modified: {entry.State}");

        context.SaveChanges();
        Console.WriteLine($"Estado tras SaveChanges: {entry.State}");
    }

    public static void DemostrarRemove()
    {
        using var context = AceriaDbContextFactory.Create();

        var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-004")!;
        Console.WriteLine($"Estado antes de Remove: {context.Entry(orden).State}");

        context.OrdenesFabricacion.Remove(orden);
        Console.WriteLine($"Estado tras Remove: {context.Entry(orden).State}");

        context.SaveChanges();
        Console.WriteLine($"Estado tras SaveChanges: {context.Entry(orden).State}");
    }

    public static void DemostrarEntryProperty()
    {
        using var context = AceriaDbContextFactory.Create();

        var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-005")!;
        var entry = context.Entry(orden);

        Console.WriteLine($"Estado inicial: {entry.State}");
        Console.WriteLine($"Cliente original: {entry.Property(o => o.Cliente).OriginalValue}");

        entry.Property(o => o.Cliente).CurrentValue = "Constructora del Norte Modificada con Entry";
        entry.Property(o => o.Cliente).IsModified = true;

        Console.WriteLine($"Estado tras modificar propiedad: {entry.State}");
        Console.WriteLine($"Cliente actual: {entry.Property(o => o.Cliente).CurrentValue}");

        context.SaveChanges();
        Console.WriteLine($"Estado tras SaveChanges: {entry.State}");
    }
}

```

Línea 1: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres de EF Core. Línea 3: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres. Línea 5: `public class OrdenFabricacion` → entidad de orden. Línea 7: `public int Id { get; set; }` → clave primaria. Línea 8: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden. Línea 9: `public string Cliente { get; set; } = string.Empty;` → cliente. Línea 10: `public DateTime FechaCreacion { get; set; }` → fecha de creación. Línea 13: `public class PlanchaAcero` → entidad de plancha. Línea 15: `public int Id { get; set; }` → clave primaria. Línea 16: `public int OrdenId { get; set; }` → clave foránea. Línea 17: `public double Espesor { get; set; }` → espesor. Línea 18: `public double Ancho { get; set; }` → ancho. Línea 19: `public double Largo { get; set; }` → largo. Línea 20: `public OrdenFabricacion Orden { get; set; } = null!;` → propiedad de navegación. Línea 23: `public class Aleacion` → entidad de aleación. Línea 25: `public int Id { get; set; }` → clave primaria. Línea 26: `public string Nombre { get; set; } = string.Empty;` → nombre. Línea 27: `public double PorcentajeCarbono { get; set; }` → porcentaje de carbono. Línea 28: `public double PorcentajeManganeso { get; set; }` → porcentaje de manganeso. Línea 31: `public class EstadoOrden` → entidad de estado. Línea 33: `public int Id { get; set; }` → clave primaria. Línea 34: `public string Nombre { get; set; } = string.Empty;` → nombre del estado. Línea 35: `public string Descripcion { get; set; } = string.Empty;` → descripción. Línea 38: `public class AceriaDbContext : DbContext` → declara el DbContext. Línea 40: `public DbSet OrdenesFabricacion { get; set; } = null!;` → DbSet de órdenes. Línea 41: `public DbSet PlanchasAcero { get; set; } = null!;` → DbSet de planchas. Línea 42: `public DbSet Aleaciones { get; set; } = null!;` → DbSet de aleaciones. Línea 43: `public DbSet EstadosOrden { get; set; } = null!;` → DbSet de estados. Línea 45: `protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)` → configuración. Línea 47: `optionsBuilder.UseSqlServer(` → registra el proveedor de SQL Server. Línea 48:

"Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;" → cadena de conexión. Línea 52: public static class AceriaDbContextFactory → fábrica manual. Línea 54: public static AceriaDbContext Create() → método que crea el contexto. Línea 56: return new AceriaDbContext(); → devuelve una instancia nueva. Línea 60: public class Program → clase principal. Línea 62: public static void Main() → punto de entrada. Línea 64: using (var context = AceriaDbContextFactory.Create()) → unidad de trabajo para recrear la base. Línea 66: context.Database.EnsureDeleted(); → elimina la base. Línea 67: context.Database.Migrate(); → crea la base. Línea 70: InsertarOrdenesIniciales(); → inserta las órdenes iniciales. Línea 72: DemostrarAdd(); → demuestra Add. Línea 73: DemostrarAddRange(); → demuestra AddRange. Línea 74: DemostrarUpdate(); → demuestra Update. Línea 75: DemostrarAttach(); → demuestra Attach. Línea 76: DemostrarEntry(); → demuestra Entry. Línea 77: DemostrarRemove(); → demuestra Remove. Línea 78: DemostrarEntryProperty(); → demuestra Entry.Property. Línea 81: public static void InsertarOrdenesIniciales() → método que inserta órdenes iniciales. Línea 83: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 84: var ordenes = new List → crea la lista de órdenes. Línea 90: context.OrganizacionesFabricacion.AddRange(ordenes); → registra las órdenes. Línea 91: context.SaveChanges(); → ejecuta los INSERT. Línea 94: public static void DemostrarAdd() → método que demuestra Add. Línea 96: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 98: var orden = new OrdenFabricacion → crea la entidad. Línea 104: Console.WriteLine(\$"Estado antes de Add: {context.Entry(orden).State}"); → muestra el estado inicial, que es Detached. Línea 105: context.OrganizacionesFabricacion.Add(orden); → registra la entidad. Línea 106: Console.WriteLine(\$"Estado tras Add: {context.Entry(orden).State}"); → muestra el estado Added. Línea 107: context.SaveChanges(); → ejecuta el INSERT. Línea 108: Console.WriteLine(\$"Estado tras SaveChanges: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 111: public static void DemostrarAddRange() → método que demuestra AddRange. Línea 113: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 115: var ordenes = new List → crea la lista de órdenes. Línea 120: context.OrganizacionesFabricacion.AddRange(ordenes); → registra las órdenes. Línea 121: Console.WriteLine(\$"Estado tras AddRange: {context.ChangeTracker.Entries().Count()} entidades rastreadas"); → muestra el número de entidades rastreadas. Línea 122: context.SaveChanges(); → ejecuta los INSERT. Línea 123: Console.WriteLine(\$"Órdenes insertadas: {ordenes.Count}"); → muestra el número de órdenes insertadas. Línea 126: public static void DemostrarUpdate() → método que demuestra Update. Línea 128: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 130: var orden = new OrdenFabricacion → crea la entidad desconectada. Línea 137: context.OrganizacionesFabricacion.Update(orden); → marca la entidad como Modified. Línea 138: Console.WriteLine(\$"Estado tras Update: {context.Entry(orden).State}"); → muestra el estado Modified. Línea 139: Console.WriteLine(\$"SQL generado: {context.OrganizacionesFabricacion.Where(o => o.Id == 1).ToQueryString()}"); → muestra el SQL que se generaría para una consulta. Línea 140: context.SaveChanges(); → ejecuta el UPDATE. Línea 141: Console.WriteLine(\$"Estado tras SaveChanges: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 144: public static void DemostrarAttach() → método que demuestra Attach. Línea 146: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 148: var orden = new OrdenFabricacion → crea la entidad desconectada. Línea 155: context.OrganizacionesFabricacion.Attach(orden); → registra la entidad con estado Unchanged. Línea 156: Console.WriteLine(\$"Estado tras Attach: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 158: orden.Cliente = "Constructora del Sur Actualizada con Attach"; → modifica la propiedad. Línea 159: context.ChangeTracker.DetectChanges(); → fuerza la detección de cambios. Línea 160: Console.WriteLine(\$"Estado tras modificar: {context.Entry(orden).State}"); → muestra el estado Modified. Línea 161: context.SaveChanges(); → ejecuta el UPDATE. Línea 162: Console.WriteLine(\$"Estado tras SaveChanges: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 165: public static void DemostrarEntry() → método que demuestra Entry. Línea 167: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 169: var orden = new OrdenFabricacion → crea la entidad desconectada. Línea 176: var entry = context.Entry(orden); → obtiene el EntityEntry. Línea 177: Console.WriteLine(\$"Estado inicial: {entry.State}"); → muestra el estado Detached. Línea 179: entry.State = EntityState.Modified; → cambia el estado a Modified. Línea 180: Console.WriteLine(\$"Estado tras cambiar a Modified: {entry.State}"); → muestra el nuevo estado. Línea 182: context.SaveChanges(); → ejecuta el UPDATE. Línea 183: Console.WriteLine(\$"Estado tras SaveChanges: {entry.State}"); → muestra el estado Unchanged. Línea 186: public static void DemostrarRemove() → método que demuestra Remove. Línea 188: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 190: var orden =

context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-004")!; → carga la orden. Línea 191: Console.WriteLine(\$"Estado antes de Remove: {context.Entry(orden).State}"); → muestra el estado Unchanged. Línea 193: context.OrdenesFabricacion.Remove(orden); → marca la entidad como Deleted. Línea 194: Console.WriteLine(\$"Estado tras Remove: {context.Entry(orden).State}"); → muestra el estado Deleted. Línea 196: context.SaveChanges(); → ejecuta el DELETE. Línea 197: Console.WriteLine(\$"Estado tras SaveChanges: {context.Entry(orden).State}"); → muestra el estado Detached. Línea 200: public static void DemostrarEntryProperty() → método que demuestra Entry.Property. Línea 202: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 204: var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-005")!; → carga la orden. Línea 205: var entry = context.Entry(orden); → obtiene el EntityEntry. Línea 207: Console.WriteLine(\$"Estado inicial: {entry.State}"); → muestra el estado Unchanged. Línea 208: Console.WriteLine(\$"Cliente original: {entry.Property(o => o.Cliente).OriginalValue}"); → muestra el valor original. Línea 210: entry.Property(o => o.Cliente).CurrentValue = "Constructora del Norte Modificada con Entry"; → modifica el valor actual. Línea 211: entry.Property(o => o.Cliente).IsModified = true; → marca la propiedad como modificada. Línea 213: Console.WriteLine(\$"Estado tras modificar propiedad: {entry.State}"); → muestra el estado Modified. Línea 214: Console.WriteLine(\$"Cliente actual: {entry.Property(o => o.Cliente).CurrentValue}"); → muestra el valor actual. Línea 216: context.SaveChanges(); → ejecuta el UPDATE. Línea 217: Console.WriteLine(\$"Estado tras SaveChanges: {entry.State}"); → muestra el estado Unchanged.

Error común: si se llama a Add sobre una entidad ya registrada, se lanza una excepción InvalidOperationException. Si se llama a Update sobre una entidad desconectada que no existe en la base de datos, el UPDATE no afecta a ninguna fila y EF Core lanza una excepción de concurrencia.

Paso 3: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen los estados de cada operación y los resultados de cada método. Se observa cómo Add cambia el estado a Added, Update a Modified, Attach a Unchanged, Remove a Deleted y Entry permite manipular el estado manualmente.

Paso 4: Observar la diferencia entre Update y Attach

Comparar los SQL generados por Update y Attach. Para ver el SQL generado por Update, se puede activar el logging de EF Core. Modificar el método OnConfiguring para habilitar el logging:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder
        .UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;")
        .LogTo(Console.WriteLine, Microsoft.Extensions.Logging.LogLevel.Information);
}
```

Línea 45: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → método de configuración. Línea 47: optionsBuilder → objeto de configuración. Línea 48: .UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;") → registra el proveedor de SQL Server. Línea 49: .LogTo(Console.WriteLine, Microsoft.Extensions.Logging.LogLevel.Information); → habilita el logging de EF Core en la consola.

Resultado esperado: en la consola aparecen las sentencias SQL generadas por cada operación. Se observa que Update genera un UPDATE con todas las columnas, mientras que Attach seguido de modificación genera un UPDATE con una sola columna.

Paso 5: Ejecutar el proyecto con logging

```
dotnet run
```

Resultado esperado: la consola muestra las sentencias SQL. Se puede comparar el UPDATE completo de Update con el UPDATE parcial de Attach.

Paso 6: Diagnosticar un error común

Modificar el método DemostrarAdd para llamar a Add dos veces sobre la misma entidad:

```
context.OrdenesFabricacion.Add(orden);  
context.OrdenesFabricacion.Add(orden);
```

Resultado esperado: se lanza una excepción InvalidOperationException indicando que la entidad ya está siendo rastreada.

Solución: comprobar el estado de la entidad antes de llamar a Add, o usar Entry(orden).State para verificar que está en estado Detached.

Paso 7: Verificar los cambios en la base de datos

Abrir Visual Studio. En Ver → Explorador de objetos de SQL Server. Expandir (localdb)\MSSQLLocalDB → Bases de datos → AceriaDB → Tablas → dbo.OrdenesFabricacion. Clic derecho → Ver datos. Comprobar que las órdenes se han insertado, actualizado y eliminado según las operaciones realizadas.

Resultado esperado: la tabla muestra las órdenes OF-005 y OF-006 con los clientes actualizados, y la orden OF-004 eliminada.

Errores comunes del ejercicio completo

Error	Causa	Solución
Add sobre entidad rastreada	La entidad ya estaba registrada	Comprobar el estado antes de llamar a Add
Update no afecta filas	La entidad no existe en la base de datos	Verificar el Id antes de actualizar
Remove no afecta filas	La entidad no existe en la base de datos	Comprobar el resultado de FirstOrDefault
Attach no detecta cambios	Se modificó antes de Attach	Modificar después de Attach
Entry con estado incorrecto	Se cambió el estado manualmente sin motivo	Verificar el estado antes de cambiarlo
SaveChanges no guarda	Falta llamada a SaveChanges	Añadir context.SaveChanges()

Reto resuelto: Actualizar solo una propiedad con Entry.Property

Reto: Crear un método que actualice solo el cliente de una orden, sin cargar la entidad completa ni usar Update. Utilizar Entry.Property para marcar solo la propiedad modificada.

Solución paso a paso

Paso 1: Añadir el método ActualizarSoloCliente:

```
public static void ActualizarSoloCliente(int id, string nuevoCliente)
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion { Id = id };
    context OrdenesFabricacion.Attach(orden);

    orden.Cliente = nuevoCliente;
    context.Entry(orden).Property(o => o.Cliente).IsModified = true;

    context.SaveChanges();
    Console.WriteLine($"Cliente actualizado para la orden {id}");
}
```

Línea 1: `public static void ActualizarSoloCliente(int id, string nuevoCliente)` → declara el método con el `Id` y el nuevo cliente. Línea 3: `using var context = AceriaDbContextFactory.Create();` → crea la unidad de trabajo. Línea 5: `var orden = new OrdenFabricacion { Id = id };` → crea una entidad con solo el `Id`. Línea 6: `context.OrdenesFabricacion.Attach(orden);` → registra la entidad con estado `Unchanged`. Línea 8: `orden.Cliente = nuevoCliente;` → modifica la propiedad `Cliente`. Línea 9: `context.Entry(orden).Property(o => o.Cliente).IsModified = true;` → marca solo la propiedad `Cliente` como modificada. Línea 11: `context.SaveChanges();` → ejecuta el `UPDATE`, que solo actualiza la columna `Cliente`. Línea 12: `Console.WriteLine($"Cliente actualizado para la orden {id}");` → confirma la actualización.

Paso 2: Llamar al método desde Main:

```
ActualizarSoloCliente(5, "Constructora del Norte Modificada Solo Cliente");
```

Paso 3: Ejecutar `dotnet run` y verificar en el logging que el `UPDATE` solo actualiza la columna `Cliente`.

Analogía final

Los métodos de gestión de entidades son como las distintas formas en que un operario de una acería puede registrar una plancha en el libro de producción. `Add` es como dar de alta una plancha nueva que acaba de salir del horno. `Update` es como reemplazar todos los datos de una plancha existente, aunque solo haya cambiado un detalle. `Attach` es como dar de alta una plancha que ya existía pero que no estaba en el libro, y luego anotar solo los cambios que se produzcan. `Remove` es como dar de baja una plancha que ya no se va a usar. `Entry` es como abrir la ficha de una plancha y modificar un campo concreto sin tocar los demás. Cada método tiene su propósito y su momento. Elegir el adecuado evita trabajo innecesario y mantiene el libro de producción coherente. Así funciona la gestión de entidades en EF Core: cada operación tiene su semántica, y conocerla permite escribir código más eficiente y más claro.

Resultado esperado

Al final del ejercicio, deberías haber:

Observado el comportamiento de `Add` y `AddRange`.

Observado el comportamiento de `Update` y `UpdateRange`.

Observado el comportamiento de `Remove` y `RemoveRange`.

Observado el comportamiento de `Attach` y su diferencia con `Update`.

Utilizado `Entry` para manipular estados manualmente.

Utilizado `Entry.Property` para marcar propiedades individuales como modificadas.

Comparado los SQL generados por Update y Attach.

Diagnosticado errores comunes de gestión de entidades.

Actualizado solo una propiedad con Entry.Property.

| Conclusión y enlace al siguiente punto

En este punto se han estudiado los métodos de gestión de entidades: Add, AddRange, Update, UpdateRange, Remove, RemoveRange, Attach y Entry. Se ha comparado el comportamiento de Update y Attach, se ha demostrado el uso de Entry para manipular estados y se ha actualizado solo una propiedad con Entry.Property. En el siguiente punto se estudiará SaveChanges y el patrón de unidad de trabajo, con sus implicaciones en transacciones y en la coherencia de los datos.

Punto 1.9 - SaveChanges y unidad de trabajo

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se profundiza en el método SaveChanges del proyecto AceriaData, observando su comportamiento interno, el uso de transacciones implícitas y la aplicación del patrón de unidad de trabajo, siempre contra SQL Server LocalDB.

Ejercicio: Añadir al proyecto AceriaData un conjunto de métodos que demuestren el comportamiento de SaveChanges: valor devuelto, atomicidad de una llamada, propagación de claves, manejo de errores, uso de SaveChangesAsync y aplicación del patrón de unidad de trabajo.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
```

```

public DbSet<Aleacion> Aleaciones { get; set; } = null!;
public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlServer(
        "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
}

public static class AceriaDbContextFactory
{
    public static AceriaDbContext Create()
    {
        return new AceriaDbContext();
    }
}

public class Program
{
    public static async Task Main()
    {
        using (var context = AceriaDbContextFactory.Create())
        {
            context.Database.EnsureDeleted();
            context.Database.Migrate();

            DemostrarValorDevuelto();
            DemostrarAtomicidadSaveChanges();
            DemostrarPropagacionDeClaves();
            DemostrarManejoDeErrores();
            await DemostrarSaveChangesAsync();
            DemostrarUnidadDeTrabajo();
        }

        public static void DemostrarValorDevuelto()
        {
            using var context = AceriaDbContextFactory.Create();

            var orden1 = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.Now };
            var orden2 = new OrdenFabricacion { NumeroOrden = "OF-002", Cliente = "Constructora del Sur",
            FechaCreacion = DateTime.Now };

            context OrdenesFabricacion.AddRange(orden1, orden2);
            var filas = context.SaveChanges();

            Console.WriteLine($"Filas afectadas en la inserción: {filas}");

            orden1.Cliente = "Constructora del Norte Modificada";
            var filasUpdate = context.SaveChanges();

            Console.WriteLine($"Filas afectadas en la actualización: {filasUpdate}");
        }

        public static void DemostrarAtomicidadSaveChanges()
        {
            using var context = AceriaDbContextFactory.Create();

            var orden = new OrdenFabricacion { NumeroOrden = "OF-003", Cliente = "Constructora del Este",
            FechaCreacion = DateTime.Now };
            context OrdenesFabricacion.Add(orden);

            var planchaInvalida = new PlanchaAcero { OrdenId = 9999, Espesor = 10.5, Ancho = 1500, Largo =
            3000 };
            context.PlanchasAcero.Add(planchaInvalida);

            try
            {
                context.SaveChanges();
            }
            catch (DbUpdateException ex)

```

```

    {
        Console.WriteLine($"Error capturado: {ex.InnerException?.Message}");
    }

    var totalOrdenes = context.OrdenesFabricacion.Count();
    Console.WriteLine($"Órdenes en la base de datos tras el error: {totalOrdenes}");
}

public static void DemostrarPropagacionDeClaves()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "0F-004",
        Cliente = "Constructora del Oeste",
        FechaCreacion = DateTime.Now
    };

    context.OrdenesFabricacion.Add(orden);
    context.SaveChanges();

    Console.WriteLine($"Id generado para la orden: {orden.Id}");

    var plancha = new PlanchaAcero
    {
        OrdenId = orden.Id,
        Espesor = 10.5,
        Ancho = 1500,
        Largo = 3000
    };

    context.PlanchasAcero.Add(plancha);
    context.SaveChanges();

    Console.WriteLine($"Id generado para la plancha: {plancha.Id}");
    Console.WriteLine($"Clave foránea de la plancha: {plancha.OrdenId}");
}

public static void DemostrarManejoDeErrores()
{
    using var context = AceriaDbContextFactory.Create();

    var planchaInvalida = new PlanchaAcero { OrdenId = 999999, Espesor = 8.0, Ancho = 1000, Largo =
2000 };
    context.PlanchasAcero.Add(planchaInvalida);

    try
    {
        context.SaveChanges();
    }
    catch (DbUpdateException ex)
    {
        Console.WriteLine($"Error de base de datos: {ex.InnerException?.Message}");
    }
}

public static async Task DemostrarSaveChangesAsync()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion { NumeroOrden = "0F-005", Cliente = "Constructora Asíncrona",
FechaCreacion = DateTime.Now };
    context.OrdenesFabricacion.Add(orden);

    var filas = await context.SaveChangesAsync();
    Console.WriteLine($"Filas afectadas con SaveChangesAsync: {filas}");
}

public static void DemostrarUnidadDeTrabajo()
{
    using var context = AceriaDbContextFactory.Create();

```

```

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-006",
            Cliente = "Constructora Unidad",
            FechaCreacion = DateTime.Now
        };

        var plancha1 = new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000, Orden = orden };
        var plancha2 = new PlanchaAcero { Espesor = 12.0, Ancho = 1200, Largo = 2500, Orden = orden };

        context.OrdenesFabricacion.Add(orden);
        context.PlanchasAcero.AddRange(plancha1, plancha2);

        var filas = context.SaveChanges();

        Console.WriteLine($"Filas afectadas en la unidad de trabajo: {filas}");
        Console.WriteLine($"Orden Id: {orden.Id}, Plancha 1 Id: {plancha1.Id}, Plancha 2 Id:
{plancha2.Id}");
    }
}

```

Línea 1: using Microsoft.EntityFrameworkCore; → importa el espacio de nombres de EF Core. Línea 3: namespace AceriaData.ConsoleApp; → declara el espacio de nombres. Línea 5: public class OrdenFabricacion → entidad de orden. Línea 7: public int Id { get; set; } → clave primaria. Línea 8: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 9: public string Cliente { get; set; } = string.Empty; → cliente. Línea 10: public DateTime FechaCreacion { get; set; } → fecha de creación. Línea 11: public List Planchas { get; set; } = new(); → colección de planchas relacionadas. Línea 14: public class PlanchaAcero → entidad de plancha. Línea 16: public int Id { get; set; } → clave primaria. Línea 17: public int OrdenId { get; set; } → clave foránea. Línea 18: public double Espesor { get; set; } → espesor. Línea 19: public double Ancho { get; set; } → ancho. Línea 20: public double Largo { get; set; } → largo. Línea 21: public OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación. Línea 24: public class Aleacion → entidad de aleación. Línea 26: public int Id { get; set; } → clave primaria. Línea 27: public string Nombre { get; set; } = string.Empty; → nombre. Línea 28: public double PorcentajeCarbono { get; set; } → porcentaje de carbono. Línea 29: public double PorcentajeManganeso { get; set; } → porcentaje de manganeso. Línea 32: public class EstadoOrden → entidad de estado. Línea 34: public int Id { get; set; } → clave primaria. Línea 35: public string Nombre { get; set; } = string.Empty; → nombre del estado. Línea 36: public string Descripcion { get; set; } = string.Empty; → descripción. Línea 39: public class AceriaDbContext : DbContext → declara el DbContext. Línea 41: public DbSet OrdenesFabricacion { get; set; } = null!; → DbSet de órdenes. Línea 42: public DbSet PlanchasAcero { get; set; } = null!; → DbSet de planchas. Línea 43: public DbSet Aleaciones { get; set; } = null!; → DbSet de aleaciones. Línea 44: public DbSet EstadosOrden { get; set; } = null!; → DbSet de estados. Línea 46: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → configuración. Línea 48: optionsBuilder.UseSqlServer(→ registra el proveedor de SQL Server. Línea 49: "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;" → cadena de conexión. Línea 53: public static class AceriaDbContextFactory → fábrica manual. Línea 55: public static AceriaDbContext Create() → método que crea el contexto. Línea 57: return new AceriaDbContext(); → devuelve una instancia nueva. Línea 61: public class Program → clase principal. Línea 63: public static async Task Main() → punto de entrada asíncrono. Línea 65: using (var context = AceriaDbContextFactory.Create()) → unidad de trabajo para recrear la base. Línea 67: context.Database.EnsureDeleted(); → elimina la base. Línea 68: context.Database.Migrate(); → crea la base. Línea 71: DemostrarValorDevuelto(); → demuestra el valor devuelto por SaveChanges. Línea 72: DemostrarAtomicidadSaveChanges(); → demuestra la atomicidad de una llamada a SaveChanges. Línea 73: DemostrarPropagacionDeClaves(); → demuestra la propagación de claves. Línea 74: DemostrarManejoDeErrores(); → demuestra el manejo de errores. Línea 75: await DemostrarSaveChangesAsync(); → demuestra SaveChangesAsync. Línea 76: DemostrarUnidadDeTrabajo(); → demuestra la unidad de trabajo. Línea 79: public static void DemostrarValorDevuelto() → método que demuestra el valor devuelto. Línea 81: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 83: var orden1 = new OrdenFabricacion { ... }; → crea la primera orden. Línea 84: var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden. Línea 86: context.OrdenesFabricacion.AddRange(orden1, orden2); → registra ambas órdenes. Línea 87: var filas = context.SaveChanges(); → guarda y obtiene el número de filas afectadas. Línea 89:

Console.WriteLine(\$"Filas afectadas en la inserción: {filas}"); → muestra el número de filas. Línea 91: orden1.Cliente = "Constructora del Norte Modificada"; → modifica una propiedad. Línea 92: var filasUpdate = context.SaveChanges(); → guarda y obtiene el número de filas. Línea 94: Console.WriteLine(\$"Filas afectadas en la actualización: {filasUpdate}"); → muestra el número de filas. Línea 97: public static void DemostrarAtomicidadSaveChanges() → método que demuestra la atomicidad de una llamada a SaveChanges. Línea 99: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 101: var orden = new OrdenFabricacion { ... }; → crea una orden. Línea 102: context OrdenesFabricacion.Add(orden); → registra la orden. Línea 104: var planchaInvalida = new PlanchaAcero { OrdenId = 9999, ... }; → crea una plancha con clave foránea inválida. Línea 105: context.PlanchasAcero.Add(planchaInvalida); → registra la plancha. Línea 107: try → inicio del bloque de prueba. Línea 109: context.SaveChanges(); → intenta guardar. Línea 111: catch (DbUpdateException ex) → captura la excepción. Línea 113: Console.WriteLine(\$"Error capturado: {ex.InnerException?.Message}"); → muestra el mensaje del error. Línea 116: var totalOrdenes = context OrdenesFabricacion.Count(); → cuenta las órdenes en la base. Línea 117: Console.WriteLine(\$"Órdenes en la base de datos tras el error: {totalOrdenes}"); → muestra el total. Línea 120: public static void DemostrarPropagacionDeClaves() → método que demuestra la propagación de claves. Línea 122: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 124: var orden = new OrdenFabricacion { ... }; → crea la orden. Línea 131: context OrdenesFabricacion.Add(orden); → registra la orden. Línea 132: context.SaveChanges(); → inserta la orden. Línea 134: Console.WriteLine(\$"Id generado para la orden: {orden.Id}"); → muestra el Id generado. Línea 136: var plancha = new PlanchaAcero { ... }; → crea la plancha. Línea 144: context.PlanchasAcero.Add(plancha); → registra la plancha. Línea 145: context.SaveChanges(); → inserta la plancha. Línea 147: Console.WriteLine(\$"Id generado para la plancha: {plancha.Id}"); → muestra el Id generado. Línea 148: Console.WriteLine(\$"Clave foránea de la plancha: {plancha.OrdenId}"); → muestra la clave foránea. Línea 151: public static void DemostrarManejoDeErrores() → método que demuestra el manejo de errores. Línea 153: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 155: var planchaInvalida = new PlanchaAcero { OrdenId = 999999, ... }; → crea una plancha cuya clave foránea apunta deliberadamente a una orden inexistente. Línea 156: context.PlanchasAcero.Add(planchaInvalida); → registra la plancha inválida para que SQL Server compruebe la restricción de clave foránea al guardar. Línea 158: try → inicio del bloque de prueba. Línea 160: context.SaveChanges(); → intenta guardar. Línea 162: catch (DbUpdateException ex) → captura la excepción. Línea 164: Console.WriteLine(\$"Error de base de datos: {ex.InnerException?.Message}"); → muestra el mensaje del error. Línea 167: public static async Task DemostrarSaveChangesAsync() → método asíncrono que demuestra SaveChangesAsync. Línea 169: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 171: var orden = new OrdenFabricacion { ... }; → crea la orden. Línea 172: context OrdenesFabricacion.Add(orden); → registra la orden. Línea 174: var filas = await context.SaveChangesAsync(); → guarda de forma asíncrona. Línea 175: Console.WriteLine(\$"Filas afectadas con SaveChangesAsync: {filas}"); → muestra el número de filas. Línea 178: public static void DemostrarUnidadDeTrabajo() → método que demuestra la unidad de trabajo. Línea 180: using var context = AceriaDbContextFactory.Create(); → unidad de trabajo. Línea 182: var orden = new OrdenFabricacion { ... }; → crea la orden. Línea 188: var plancha1 = new PlanchaAcero { ... Orden = orden }; → crea la primera plancha. Línea 189: var plancha2 = new PlanchaAcero { ... Orden = orden }; → crea la segunda plancha. Línea 191: context OrdenesFabricacion.Add(orden); → registra la orden. Línea 192: context.PlanchasAcero.AddRange(plancha1, plancha2); → registra las planchas. Línea 194: var filas = context.SaveChanges(); → guarda todo en una sola transacción. Línea 196: Console.WriteLine(\$"Filas afectadas en la unidad de trabajo: {filas}"); → muestra el número de filas. Línea 197: Console.WriteLine(\$"Orden Id: {orden.Id}, Plancha 1 Id: {plancha1.Id}, Plancha 2 Id: {plancha2.Id}"); → muestra los Ids generados.

Error común: si se llama a SaveChanges sin haber registrado ninguna entidad, el método devuelve cero y no ejecuta ninguna sentencia SQL. Si se produce un error durante el SaveChanges, la transacción se revierte y ninguna operación se guarda.

Paso 3: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen los resultados de cada método. Se observa el número de filas afectadas, el comportamiento transaccional, la propagación de claves, el manejo de errores, el uso de SaveChangesAsync y la unidad de trabajo.

| Paso 4: Observar el SQL generado con logging

Modificar el método OnConfiguring para habilitar el logging:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder
        .UseSqlServer("Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;")
        .LogTo(Console.WriteLine, Microsoft.Extensions.Logging.LogLevel.Information);
}
```

Línea 46: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → método de configuración. Línea 48: optionsBuilder → objeto de configuración. Línea 49: .UseSqlServer("Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;") → registra el proveedor de SQL Server. Línea 50: .LogTo(Console.WriteLine, Microsoft.Extensions.Logging.LogLevel.Information); → habilita el logging de EF Core en la consola.

Resultado esperado: en la consola aparecen las sentencias SQL generadas. Cuando una llamada a SaveChanges necesita varios comandos, el log permite observar la transacción automática; una llamada de una sola sentencia puede no emitir BEGIN TRANSACTION explícito.

| Paso 5: Ejecutar el proyecto con logging

```
dotnet run
```

Resultado esperado: la consola muestra las sentencias SQL y, cuando son necesarias, las transacciones automáticas. No debe esperarse un BEGIN TRANSACTION explícito para cada SaveChanges de una sola sentencia.

| Paso 6: Diagnosticar un error común

Modificar el método DemostrarValorDevuelto para llamar a SaveChanges sin haber añadido entidades:

```
public static void DemostrarValorDevuelto()
{
    using var context = AceriaDbContextFactory.Create();

    var filas = context.SaveChanges();
    Console.WriteLine($"Filas afectadas sin cambios: {filas}");
}
```

Resultado esperado: el método devuelve cero y no ejecuta ninguna sentencia SQL. No se abre transacción porque no hay cambios que guardar.

| Paso 7: Verificar los cambios en la base de datos

Abrir Visual Studio. En Ver → Explorador de objetos de SQL Server. Expandir (localdb)\MSSQLLocalDB → Bases de datos → AceriaDB → Tablas → dbo.OrdenesFabricacion. Clic derecho → Ver datos. Comprobar que aparecen las órdenes insertadas y actualizadas según las operaciones realizadas.

Resultado esperado: la tabla contiene las órdenes OF-001, OF-002, OF-004, OF-005 y OF-006. La orden OF-003 no aparece porque la llamada que intentaba guardarla junto con una plancha de clave foránea inválida se revirtió de forma atómica.

Errores comunes del ejercicio completo

Error	Causa	Solución
SaveChanges no guarda	No hay entidades en estado Added, Modified o Deleted	Verificar que las entidades estén registradas
Transacción revertida	Alguna sentencia SQL falla	Revisar el mensaje de la excepción interna
Clave foránea inválida	Se insertó una entidad dependiente antes que la principal	Usar propiedades de navegación o guardar en orden
DbUpdateException	Restricción de base de datos violada	Revisar el mensaje de la excepción interna
DbUpdateConcurrencyException	Otro usuario modificó la fila	Gestionar el conflicto con Entry
SaveChangesAsync no esperado	Falta await	Añadir await delante de la llamada

Reto resuelto: Insertar una orden con planchas usando propiedades de navegación

Reto: Crear un método que inserte una orden con dos planchas usando propiedades de navegación, sin necesidad de llamar a SaveChanges dos veces. Verificar que EF Core propaga la clave foránea automáticamente.

Solución paso a paso

Paso 1: Añadir el método InsertarOrdenConPlanchas:

```
public static void InsertarOrdenConPlanchas()
{
    using var context = AceriaDbContextFactory.Create();

    var orden = new OrdenFabricacion
    {
        NumeroOrden = "OF-007",
        Cliente = "Constructora con Planchas",
        FechaCreacion = DateTime.Now,
        Planchas = new List<PlanchaAcero>
        {
            new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000 },
            new PlanchaAcero { Espesor = 12.0, Ancho = 1200, Largo = 2500 }
        }
    };

    context OrdenesFabricacion.Add(orden);
    var filas = context.SaveChanges();

    Console.WriteLine($"Filas afectadas: {filas}");
    Console.WriteLine($"Orden Id: {orden.Id}");
    foreach (var plancha in orden.Planchas)
    {
        Console.WriteLine($"Plancha Id: {plancha.Id}, OrdenId: {plancha.OrdenId}");
    }
}
```

Línea 1: public static void InsertarOrdenConPlanchas() → declara el método. Línea 3: using var context = AceriaDbContextFactory.Create(); → crea la unidad de trabajo. Línea 5: var orden = new OrdenFabricacion → crea la orden. Línea 7: NumeroOrden = "OF-007", → asigna el número. Línea 8: Cliente = "Constructora con Planchas", → asigna el cliente. Línea 9: FechaCreacion = DateTime.Now, → asigna la fecha. Línea 10: Planchas = new List → inicializa la colección de planchas. Línea 12: new

PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000 }, → crea la primera plancha. Línea 13: new PlanchaAcero { Espesor = 12.0, Ancho = 1200, Largo = 2500 } → crea la segunda plancha. Línea 17: context.OrdenesFabricacion.Add(orden); → registra la orden y, por propagación, las planchas. Línea 18: var filas = context.SaveChanges(); → guarda todo en una sola transacción. Línea 20: Console.WriteLine(\$"Filas afectadas: {filas}"); → muestra el número de filas. Línea 21: Console.WriteLine(\$"Orden Id: {orden.Id}"); → muestra el Id de la orden. Línea 22: foreach (var plancha in orden.Planchas) → itera sobre las planchas. Línea 24: Console.WriteLine(\$"Plancha Id: {plancha.Id}, OrdenId: {plancha.OrdenId}"); → muestra el Id y la clave foránea de cada plancha.

Paso 2: Llamar al método desde Main:

```
InsertarOrdenConPlanchas();
```

Paso 3: Ejecutar dotnet run y verificar que la orden y las dos planchas se insertan en una sola transacción, con las claves propagadas correctamente.

Analogía final

SaveChanges es como el momento en que el jefe de planta de una acería firma el parte de trabajo al final del turno. Durante el turno, los operarios han anotado en el libro de producción todas las planchas que han entrado, las que han salido y las que se han modificado. Pero nada de eso se ha hecho efectivo en el almacén central hasta que el jefe firma. Al firmar, el jefe revisa el libro, ordena las tareas según las dependencias (primero las órdenes, después las planchas), abre la puerta del almacén, ejecuta todas las operaciones y cierra la puerta. Si algo falla a mitad, cierra la puerta sin haber dejado nada a medias: todo se queda como estaba. Esa firma es la unidad de trabajo: agrupa todas las operaciones del turno en una sola transacción. Si el jefe no firma, nada llega al almacén. Si firma y todo va bien, todo se guarda. Si firma y algo falla, nada se guarda. Así funciona SaveChanges: es el momento en que los cambios dejan de ser una promesa en memoria y se convierten en datos persistentes.

Resultado esperado

Al final del ejercicio, deberías haber:

Observado el valor devuelto por SaveChanges.

Comprobado el comportamiento transaccional de SaveChanges.

Verificado la propagación de claves generadas por la base de datos.

Diagnosticado errores de base de datos con DbUpdateException.

Utilizado SaveChangesAsync en un método asíncrono.

Aplicado el patrón de unidad de trabajo con varias entidades relacionadas.

Observado las transacciones en el logging de EF Core.

Insertado una orden con planchas usando propiedades de navegación.

Conclusión y enlace al siguiente punto

En este punto se ha estudiado SaveChanges en detalle: su valor devuelto, su comportamiento transaccional, la propagación de claves, el manejo de errores, la versión asíncrona y el patrón de unidad de trabajo. En el siguiente punto se estudiará la configuración inicial de EF Core: opciones, cadena de conexión y logging, con ejemplos aplicados al proyecto AceriaData.

Punto 1.10 - Introducción práctica a las migraciones: generación, aplicación y seguimiento

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Este ejercicio trabaja sobre la cadena real de migraciones que AceriaData ya arrastra de los checkpoints anteriores. El objetivo es aprender a inspeccionarla, aplicarla, revertirla y relacionarla con `Database.Migrate()`, el snapshot y la configuración del `DbContext`.

Paso 1: Abrir la solución autónoma del punto

```
cd M01/PROYECTO/1.10
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

`cd M01/PROYECTO/1.10` → entra en el checkpoint que estudia las migraciones. `dotnet restore` → restaura EF Core, SQL Server y el paquete de diseño. `dotnet build` → verifica que el modelo y las migraciones compilan juntos.

Paso 2: Verificar dotnet-ef y el paquete de diseño

```
dotnet tool install --global dotnet-ef --version 8.0.31
dotnet ef --version
```

Si la herramienta ya está instalada, se puede actualizar con `dotnet tool update --global dotnet-ef --version 8.0.31`. El proyecto referencia `Microsoft.EntityFrameworkCore.Design`, necesario para los comandos de diseño.

Paso 3: Comprender la fábrica de tiempo de diseño

El checkpoint incorpora `AceriaDesignTimeDbContextFactory.cs` para que `dotnet ef` pueda crear el contexto sin ejecutar el flujo normal de `Main`:

```
using Microsoft.EntityFrameworkCore.Design;
using Microsoft.Extensions.Configuration;

namespace AceriaData.ConsoleApp;

public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext>
{
    public AceriaDbContext CreateDbContext(string[] args)
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.json", optional: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        return new AceriaDbContext(connectionString);
    }
}
```

Línea 1: `using Microsoft.EntityFrameworkCore.Design;` → importa la interfaz de fábrica de diseño. Línea 2: `using Microsoft.Extensions.Configuration;` → permite cargar la configuración externa. Línea 6: `public sealed class AceriaDesignTimeDbContextFactory...` → declara la fábrica utilizada por las herramientas de EF Core. Línea 8: `{` → abre la clase de fábrica. Línea 9: `public AceriaDbContext CreateDbContext(string[] args)` → método que EF Core invoca en tiempo de diseño. Línea 11: `var`

`configuration = new ConfigurationBuilder()` → inicia la carga de configuración. Línea 12:
`.SetBasePath(Directory.GetCurrentDirectory())` → usa la carpeta del proyecto como base. Línea 13:
`.AddJsonFile("appsettings.json", optional: false)` → exige la configuración JSON del checkpoint.
 Línea 14: `.AddEnvironmentVariables()` → permite sobrescribir valores mediante variables de entorno.
 Línea 15: `.Build();` → construye `IConfiguration`. Línea 17: `var connectionString = configuration.GetConnectionString("AceriaDB")` → recupera la conexión usada por el curso. Línea 18: `?? throw ...` → falla de forma explícita si falta la conexión. Línea 20: `return new AceriaDbContext(connectionString);` → devuelve el contexto que usarán `migrations list`, `database update` y `migrations script`.

Paso 4: Revisar la cadena real de migraciones

```
Migrations/
├── 20260927000100_InitialCreate.cs
├── 20260927000200_AddAleacion.cs
├── 20260927000300_AddEstadoOrden.cs
└── AceriaDbContextModelSnapshot.cs
```

La cadena ya existe porque `AceriaData` ha evolucionado desde checkpoints anteriores. Aquí se aprende a administrarla sin reescribir esa historia.

Paso 5: Listar las migraciones conocidas por EF Core

```
dotnet ef migrations list --configuration Release
```

Resultado esperado: `InitialCreate`, `AddAleacion` y `AddEstadoOrden`, en ese orden.

Paso 6: Inspeccionar Up y Down

Abrir `Migrations/20260927000300_AddEstadoOrden.cs` y revisar ambas direcciones:

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.CreateTable(
        name: "EstadosOrden",
        columns: table => new
        {
            Id = table.Column<int>(type: "int", nullable: false)
                .Annotation("SqlServer:Identity", "1, 1"),
            Nombre = table.Column<string>(type: "nvarchar(max)", nullable: false),
            Descripcion = table.Column<string>(type: "nvarchar(max)", nullable: false)
        },
        constraints: table => table.PrimaryKey("PK_EstadosOrden", x => x.Id));
}

protected override void Down(MigrationBuilder migrationBuilder) =>
    migrationBuilder.DropTable(name: "EstadosOrden");
```

`Up` crea el cambio; `Down` define cómo volver al checkpoint anterior. Antes de revertir una migración real hay que considerar también la posible pérdida de datos.

Paso 7: Revisar el ModelSnapshot

`AceriaDbContextModelSnapshot.cs` representa el modelo que EF Core considera vigente después de la última migración generada. Comprobar que contiene `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden`, además de la relación entre órdenes y planchas.

Paso 8: Crear la base desde la cadena de migraciones

```
dotnet ef database update 0 --configuration Release
dotnet ef database update --configuration Release
```

El primer comando lleva una base existente al estado anterior a la primera migración. El segundo aplica de nuevo la cadena completa.

Comprobar después el historial:

```
SELECT MigrationId, ProductVersion
FROM __EFMigrationsHistory
ORDER BY MigrationId;
```

Resultado esperado: tres filas correspondientes a las tres migraciones.

Paso 9: Revertir una migración y volver a aplicarla

```
dotnet ef database update 20260927000200_AddAleacion --configuration Release
dotnet ef migrations list --configuration Release
dotnet ef database update --configuration Release
```

Tras el primer comando, `AddEstadoOrden` queda pendiente. El último comando ejecuta de nuevo su `Up` y devuelve la base al estado actual.

Paso 10: Generar SQL sin aplicarlo

```
dotnet ef migrations script --configuration Release --output migraciones.sql
```

Abrir `migraciones.sql` y localizar la creación de `__EFMigrationsHistory` y las operaciones de la cadena. Los scripts idempotentes se estudian con profundidad en 2.11.

Paso 11: Relacionar las migraciones con Database.Migrate y el logging

El código real del checkpoint conserva la configuración externa y el logging, que ahora se interpretan como infraestructura para ejecutar y observar migraciones:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}
```

```

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    private readonly string _connectionString;

    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

    public AceriaDbContext(string connectionString)
    {
        _connectionString = connectionString;
    }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            optionsBuilder
                .UseSqlServer(_connectionString, sqlOptions =>
                {
                    sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                    sqlOptions.CommandTimeout(60);
                })
                .LogTo(
                    Console.WriteLine,
                    new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableSensitiveDataLogging()
                .EnableDetailedErrors();
        }
    }
}

public static class AceriaDbContextFactory
{
    private static string? _connectionString;

    public static void Initialize(string connectionString)
    {
        _connectionString = connectionString;
    }

    public static AceriaDbContext Create()
    {
        if (_connectionString is null)
        {
            throw new InvalidOperationException("La fábrica no ha sido inicializada. Llama a Initialize primero.");
        }
        return new AceriaDbContext(_connectionString);
    }
}

public class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");
    }
}

```

```

        AceriaDbContextFactory.Initialize(connectionString);

        using (var context = AceriaDbContextFactory.Create())
        {
            context.Database.EnsureDeleted();
            context.Database.Migrate();
        }

        InsertarOrden("OF-001", "Constructora del Norte");
        InsertarOrden("OF-002", "Constructora del Sur");
        ListarOrdenes();
    }

    public static void InsertarOrden(string numero, string cliente)
    {
        using var context = AceriaDbContextFactory.Create();
        var orden = new OrdenFabricacion
        {
            NumeroOrden = numero,
            Cliente = cliente,
            FechaCreacion = DateTime.Now
        };
        context.OrdenesFabricacion.Add(orden);
        context.SaveChanges();
    }

    public static void ListarOrdenes()
    {
        using var context = AceriaDbContextFactory.Create();
        var ordenes = context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
        Console.WriteLine("--- Órdenes ---");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente:
{orden.Cliente}");
        }
    }
}

```

Línea 1: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres de EF Core. Línea 2: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres de configuración. Línea 3: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres de logging. Línea 5: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres. Línea 7: `public class OrdenFabricacion` → entidad de orden. Línea 9: `public int Id { get; set; }` → clave primaria. Línea 10: `public string NumeroOrden { get; set; } = string.Empty;` → número de orden. Línea 11: `public string Cliente { get; set; } = string.Empty;` → cliente. Línea 12: `public DateTime FechaCreacion { get; set; }` → fecha de creación. Línea 13: `public List Planchas { get; set; } = new();` → colección de planchas relacionadas. Línea 16: `public class PlanchaAcero` → entidad de plancha. Línea 18: `public int Id { get; set; }` → clave primaria. Línea 19: `public int OrdenId { get; set; }` → clave foránea. Línea 20: `public double Espesor { get; set; }` → espesor. Línea 21: `public double Ancho { get; set; }` → ancho. Línea 22: `public double Largo { get; set; }` → largo. Línea 23: `public OrdenFabricacion Orden { get; set; } = null!;` → propiedad de navegación. Línea 26: `public class Aleacion` → entidad de aleación. Línea 28: `public int Id { get; set; }` → clave primaria. Línea 29: `public string Nombre { get; set; } = string.Empty;` → nombre. Línea 30: `public double PorcentajeCarbono { get; set; }` → porcentaje de carbono. Línea 31: `public double PorcentajeManganeso { get; set; }` → porcentaje de manganeso. Línea 34: `public class EstadoOrden` → entidad de estado. Línea 36: `public int Id { get; set; }` → clave primaria. Línea 37: `public string Nombre { get; set; } = string.Empty;` → nombre del estado. Línea 38: `public string Descripcion { get; set; } = string.Empty;` → descripción. Línea 41: `public class AceriaDbContext : DbContext` → declara el DbContext. Línea 43: `private readonly string _connectionString;` → campo que almacena la cadena de conexión. Línea 45: `public DbSet OrdenesFabricacion { get; set; } = null!;` → DbSet de órdenes. Línea 46: `public DbSet PlanchasAcero { get; set; } = null!;` → DbSet de planchas. Línea 47: `public DbSet Aleaciones { get; set; } = null!;` → DbSet de aleaciones. Línea 48: `public DbSet EstadosOrden { get; set; } = null!;` → DbSet de estados. Línea 50: `public AceriaDbContext(string connectionString)` → constructor que recibe la cadena de conexión. Línea 52: `_connectionString = connectionString;` → asigna la cadena al campo.

Línea 55: `protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)` → método de configuración. Línea 57: `if (!optionsBuilder.IsConfigured)` → comprueba si las opciones ya están configuradas. Línea 59: `optionsBuilder` → objeto de configuración. Línea 60: `.UseSqlServer(_connectionString, sqlOptions =>` → registra el proveedor de SQL Server con la cadena de conexión y un delegado de configuración. Línea 62: `sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);` → habilita los reintentos automáticos en caso de error transitorio, con un máximo de cinco intentos. Línea 63: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera de comandos en sesenta segundos. Línea 65: `.LogTo()` → habilita el logging. Línea 66: `Console.WriteLine` → destino del logging: la consola. Línea 67: `new[] { "Microsoft.EntityFrameworkCore.Database.Command" }` → categorías que se registran. Línea 68: `LogLevel.Information)` → nivel mínimo de logging. Línea 69: `.EnableSensitiveDataLogging()` → muestra los valores de los parámetros en los logs. Línea 70: `.EnableDetailedErrors();` → muestra información detallada en los errores. Línea 75: `public static class AceriaDbContextFactory` → fábrica manual. Línea 77: `private static string? _connectionString;` → campo estático que almacena la cadena de conexión. Línea 79: `public static void Initialize(string connectionString)` → método que inicializa la fábrica. Línea 81: `_connectionString = connectionString;` → asigna la cadena al campo. Línea 84: `public static AceriaDbContext Create()` → método que crea el contexto. Línea 86: `if (_connectionString is null)` → comprueba si la fábrica ha sido inicializada. Línea 88: `throw new InvalidOperationException("La fábrica no ha sido inicializada. Llama a Initialize primero.");` → lanza una excepción si no se ha inicializado. Línea 90: `return new AceriaDbContext(_connectionString);` → devuelve una instancia nueva con la cadena de conexión. Línea 94: `public class Program` → clase principal. Línea 96: `public static void Main()` → punto de entrada. Línea 98: `var configuration = new ConfigurationBuilder()` → crea el constructor de configuración. Línea 99: `.SetBasePath(Directory.GetCurrentDirectory())` → establece el directorio base. Línea 100: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)` → añade el archivo JSON, obligatorio, con recarga automática. Línea 101: `.AddEnvironmentVariables()` → añade las variables de entorno. Línea 102: `.Build();` → construye la configuración. Línea 104: `var connectionString = configuration.GetConnectionString("AceriaDB")` → lee la cadena de conexión. Línea 105: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → lanza una excepción si no se encuentra. Línea 107: `AceriaDbContextFactory.Initialize(connectionString);` → inicializa la fábrica con la cadena. Línea 109: `using (var context = AceriaDbContextFactory.Create())` → unidad de trabajo para recrear la base. Línea 111: `context.Database.EnsureDeleted();` → elimina la base. Línea 112: `context.Database.Migrate();` → crea la base. Línea 115: `InsertarOrden("OF-001", "Constructora del Norte");` → inserta la primera orden. Línea 116: `InsertarOrden("OF-002", "Constructora del Sur");` → inserta la segunda orden. Línea 117: `ListarOrdenes();` → lista las órdenes. Línea 120: `public static void InsertarOrden(string numero, string cliente)` → método de inserción. Línea 122: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 123: `var orden = new OrdenFabricacion` → crea la entidad. Línea 129: `context.OrdenesFabricacion.Add(orden);` → registra la entidad. Línea 130: `context.SaveChanges();` → ejecuta el INSERT. Línea 133: `public static void ListarOrdenes()` → método de listado. Línea 135: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 136: `var ordenes = context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → consulta ordenada. Línea 137: `Console.WriteLine("--- Órdenes ---");` → separador. Línea 138: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 140: `Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");` → muestra los datos.

La llamada `context.Database.Migrate();` aplica las migraciones pendientes con el mismo modelo que usa la CLI. `LogTo` permite observar el SQL que EF Core ejecuta durante esa operación.

Paso 12: Ejecutar el checkpoint

```
dotnet run --project AceriaData.Console.csproj --configuration Release
```

Resultado esperado: el laboratorio recrea la base, aplica las migraciones y después ejecuta las operaciones acumuladas del proyecto. Las sentencias SQL aparecen en consola.

Errores comunes del ejercicio completo

Error	Causa probable	Corrección
<code>dotnet ef</code> no se reconoce	Herramienta ausente o PATH sin actualizar	Instalar/actualizar <code>dotnet-ef</code> 8.0.31
No se puede crear <code>AceriaDbContext</code> en diseño	Falta una ruta reproducible de creación	Revisar <code>AceriaDesignTimeDbContextFactory</code> y <code>appsettings.json</code>
La base tiene tablas pero el historial no coincide	Se mezcló creación directa del esquema con migraciones	En laboratorio, recrear desde la cadena; en sistemas reales, reconciliar el esquema antes de continuar
Una migración está pendiente	El archivo existe pero no está aplicada a esa base	Revisar <code>Up / Down</code> y ejecutar <code>database update</code>
El rollback elimina datos	<code>Down</code> elimina objetos que contienen información	Evaluar impacto y disponer de backup antes de revertir
El script no representa el cambio esperado	Se está usando otro contexto o un historial incorrecto	Verificar contexto, snapshot y cadena de migraciones

Reto resuelto: auditar la migración `AddAleacion` sin duplicarla

Reto: conservar el objetivo de la fuente —trabajar con una migración asociada a `Aleacion`— dentro del proyecto acumulativo. Como `AddAleacion` ya existe, no se genera una migración duplicada: se revierte temporalmente hasta `InitialCreate`, se comprueba el estado pendiente y se reaplica la cadena completa.

Paso 1: Llevar la base al estado de `InitialCreate`

```
dotnet ef database update 20260927000100_InitialCreate --configuration Release
```

Resultado esperado: `AddAleacion` y `AddEstadoOrden` quedan pendientes y la tabla `Aleaciones` deja de formar parte del esquema aplicado.

Paso 2: Verificar la historia de migraciones

```
dotnet ef migrations list --configuration Release
```

Comprobar que `20260927000100_InitialCreate` está aplicada y que `20260927000200_AddAleacion` y `20260927000300_AddEstadoOrden` quedan pendientes.

Paso 3: Reaplicar `AddAleacion` de forma explícita

```
dotnet ef database update 20260927000200_AddAleacion --configuration Release
```

Resultado esperado: vuelve a aplicarse la migración de `Aleacion`; `AddEstadoOrden` continúa pendiente.

Paso 4: Volver al estado final acumulativo

```
dotnet ef database update --configuration Release
```

Resultado esperado: las tres migraciones vuelven a estar aplicadas y el esquema queda exactamente en el estado final del punto 1.10.

| Analogía final

Las migraciones son el libro de reformas de una acería. El modelo es el plano deseado; cada migración es una reforma fechada; el snapshot es el plano consolidado tras la última reforma; y

`EFMigrationsHistory` registra qué reformas se ejecutaron realmente en una planta concreta.

| Resultado esperado

Al terminar 1.10 se sabe distinguir `EnsureCreated()` de migraciones, leer `Up` y `Down`, interpretar el snapshot y el historial, listar migraciones, aplicar y revertir checkpoints, generar SQL y entender cómo `Database.Migrate()` utiliza esa misma historia.

| Conclusión y enlace al siguiente punto

El punto 1.11 estudia el proveedor SQL Server y el SQL generado. La configuración, la cadena de conexión y el logging continúan en el proyecto porque permiten construir el contexto y observar el comportamiento del proveedor.

Punto 1.11 - Proveedores de datos: SQLite, SQL Server y PostgreSQL

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se comparan los proveedores de datos de EF Core y se configura el proyecto AceriaData para poder alternar entre SQL Server LocalDB y otros motores, manteniendo SQL Server como proveedor principal.

Corrección canónica aprobada para M01: SQL Server LocalDB es el único proveedor operativo de AceriaData en este módulo. Las referencias a SQLite/PostgreSQL de la fuente se conservan sólo como variantes pedagógicas de lectura; no se instalan, no se configuran y no se ejecutan en M01. La práctica activa continúa sobre `Microsoft.EntityFrameworkCore.SqlServer`, `(localdb)\MSSQLLocalDB` y `AceriaDB`.

Ejercicio: Analizar el papel de los proveedores de EF Core manteniendo SQL Server LocalDB como único proveedor operativo de AceriaData. Los fragmentos de SQLite y PostgreSQL se estudian como variantes pedagógicas no ejecutables.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

`cd AceriaData` → entra en la carpeta raíz del proyecto. `cd AceriaData.Console` → entra en la carpeta del proyecto de consola.

Paso 2: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Instalar el paquete del proveedor de SQLite

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

`dotnet add package` → añade una referencia a un paquete NuGet. `Microsoft.EntityFrameworkCore.Sqlite` → nombre del paquete que contiene el proveedor de SQLite.

Error común: si se olvida instalar este paquete, el método `UseSqlite` no está disponible y el código no compila.

Paso 3: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Modificar el archivo `appsettings.json`

Abrir `appsettings.json` y añadir una sección para seleccionar el proveedor:

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\
\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;MultipleActiveResultSets=true;Connect
Timeout=30;"
  },
  "Database": {
    "Provider": "SqlServer"
  }
}
```

Línea 1: `{` → inicio del objeto JSON. Línea 2: `"ConnectionStrings": {` → sección de cadenas de conexión. Línea 3: `"AceriaDB": "Server=(localdb)\`
`\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;MultipleActiveResultSets=true;Connect`

Timeout=30;" → cadena de conexión a SQL Server LocalDB. Línea 4: }, → cierre de la sección. Línea 5: "Database": { → sección de configuración de base de datos. Línea 6: "Provider": "SqlServer" → proveedor seleccionado. Los valores posibles son SqlServer y Sqlite. Línea 7: } → cierre de la sección. Línea 8: } → cierre del objeto JSON.

Error común: si la sección Database no existe o el valor de Provider no coincide con los valores esperados, el código debe usar un valor por defecto.

Paso 4: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Sustituir el contenido de Program.cs

No sustituir el Program.cs operativo del checkpoint. El bloque siguiente se conserva únicamente para estudiar cómo sería una selección multi-proveedor; el código ejecutable de M01 permanece en SQL Server. Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    private readonly string _connectionString;
    private readonly string _provider;

    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

    public AceriaDbContext(string connectionString, string provider)
    {
        _connectionString = connectionString;
        _provider = provider;
    }
}
```

```

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        switch (_provider)
        {
            case "Sqlite":
                optionsBuilder
                    .UseSqlite(_connectionString)
                    .LogTo(
                        Console.WriteLine,
                        new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                        LogLevel.Information)
                    .EnableSensitiveDataLogging()
                    .EnableDetailedErrors();
                break;

            case "SqlServer":
            default:
                optionsBuilder
                    .UseSqlServer(_connectionString, sqlOptions =>
                    {
                        sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                        sqlOptions.CommandTimeout(60);
                    })
                    .LogTo(
                        Console.WriteLine,
                        new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                        LogLevel.Information)
                    .EnableSensitiveDataLogging()
                    .EnableDetailedErrors();
                break;
        }
    }
}

public static class AceriaDbContextFactory
{
    private static string? _connectionString;
    private static string? _provider;

    public static void Initialize(string connectionString, string provider)
    {
        _connectionString = connectionString;
        _provider = provider;
    }

    public static AceriaDbContext Create()
    {
        if (_connectionString is null || _provider is null)
        {
            throw new InvalidOperationException("La fábrica no ha sido inicializada. Llama a Initialize primero.");
        }
        return new AceriaDbContext(_connectionString, _provider);
    }
}

public class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
            .AddEnvironmentVariables()
            .Build();

        var provider = configuration["Database:Provider"] ?? "SqlServer";

        var connectionString = provider == "Sqlite"

```

```

        ? "Data Source=aceria.db"
        : configuration.GetConnectionString("AceriaDB")
        ?? throw new InvalidOperationException("No se encontró la cadena de conexión
'AceriaDB'.");

    AceriaDbContextFactory.Initialize(connectionString, provider);

    Console.WriteLine($"Proveedor configurado: {provider}");
    Console.WriteLine($"Cadena de conexión: {connectionString}");

    using (var context = AceriaDbContextFactory.Create())
    {
        context.Database.EnsureDeleted();
        context.Database.EnsureCreated();
    }

    InsertarOrden("OF-001", "Constructora del Norte");
    InsertarOrden("OF-002", "Constructora del Sur");
    ListarOrdenes();

    MostrarInformacionDelProveedor();
}

public static void InsertarOrden(string numero, string cliente)
{
    using var context = AceriaDbContextFactory.Create();
    var orden = new OrdenFabricacion
    {
        NumeroOrden = numero,
        Cliente = cliente,
        FechaCreacion = DateTime.Now
    };
    context.OrdenesFabricacion.Add(orden);
    context.SaveChanges();
}

public static void ListarOrdenes()
{
    using var context = AceriaDbContextFactory.Create();
    var ordenes = context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    Console.WriteLine("--- Órdenes ---");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente:
{orden.Cliente}");
    }
}

public static void MostrarInformacionDelProveedor()
{
    using var context = AceriaDbContextFactory.Create();
    Console.WriteLine($"Proveedor activo: {context.Database.ProviderName}");
    Console.WriteLine($"Puede conectar: {context.Database.CanConnect()}");
}
}

```

Línea 1: using Microsoft.EntityFrameworkCore; → importa el espacio de nombres de EF Core. Línea 2: using Microsoft.Extensions.Configuration; → importa el espacio de nombres de configuración. Línea 3: using Microsoft.Extensions.Logging; → importa el espacio de nombres de logging. Línea 5: namespace AceriaData.ConsoleApp; → declara el espacio de nombres. Línea 7: public class OrdenFabricacion → entidad de orden. Línea 9: public int Id { get; set; } → clave primaria. Línea 10: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 11: public string Cliente { get; set; } = string.Empty; → cliente. Línea 12: public DateTime FechaCreacion { get; set; } → fecha de creación. Línea 13: public List Planchas { get; set; } = new(); → colección de planchas relacionadas. Línea 16: public class PlanchaAcero → entidad de plancha. Línea 18: public int Id { get; set; } → clave primaria. Línea 19: public int OrdenId { get; set; } → clave foránea. Línea 20: public double Espesor { get; set; } → espesor. Línea 21: public double Ancho { get; set; } → ancho. Línea 22: public double Largo { get; set; } → largo. Línea 23: public OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación. Línea 26: public class Aleacion → entidad de aleación. Línea 28: public int Id { get; set; } →

clave primaria. Línea 29: `public string Nombre { get; set; } = string.Empty;` → nombre. Línea 30: `public double PorcentajeCarbono { get; set; }` → porcentaje de carbono. Línea 31: `public double PorcentajeManganeso { get; set; }` → porcentaje de manganeso. Línea 34: `public class EstadoOrden` → entidad de estado. Línea 36: `public int Id { get; set; }` → clave primaria. Línea 37: `public string Nombre { get; set; } = string.Empty;` → nombre del estado. Línea 38: `public string Descripcion { get; set; } = string.Empty;` → descripción. Línea 41: `public class AceriaDbContext : DbContext` → declara el DbContext. Línea 43: `private readonly string _connectionString;` → campo que almacena la cadena de conexión. Línea 44: `private readonly string _provider;` → campo que almacena el nombre del proveedor. Línea 46: `public DbSet OrdenesFabricacion { get; set; } = null!;` → DbSet de órdenes. Línea 47: `public DbSet PlanchasAcero { get; set; } = null!;` → DbSet de planchas. Línea 48: `public DbSet Aleaciones { get; set; } = null!;` → DbSet de aleaciones. Línea 49: `public DbSet EstadosOrden { get; set; } = null!;` → DbSet de estados. Línea 51: `public AceriaDbContext(string connectionString, string provider)` → constructor que recibe la cadena y el proveedor. Línea 53: `_connectionString = connectionString;` → asigna la cadena al campo. Línea 54: `_provider = provider;` → asigna el proveedor al campo. Línea 57: `protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)` → método de configuración. Línea 59: `if (!optionsBuilder.IsConfigured)` → comprueba si las opciones ya están configuradas. Línea 61: `switch (_provider)` → selecciona el proveedor según el valor. Línea 63: `case "Sqlite":` → caso para SQLite. Línea 64: `optionsBuilder` → objeto de configuración. Línea 65: `.UseSqlite(_connectionString)` → registra el proveedor de SQLite. Línea 66: `.LogTo()` → habilita el logging. Línea 67: `Console.WriteLine,` → destino del logging. Línea 68: `new[] { "Microsoft.EntityFrameworkCore.Database.Command" },` → categorías. Línea 69: `LogLevel.Information)` → nivel mínimo. Línea 70: `.EnableSensitiveDataLogging()` → muestra los valores de los parámetros. Línea 71: `.EnableDetailedErrors()` → muestra información detallada en los errores. Línea 72: `break;` → fin del caso. Línea 74: `case "SqlServer":` → caso para SQL Server. Línea 75: `default:` → caso por defecto. Línea 76: `optionsBuilder` → objeto de configuración. Línea 77: `.UseSqlServer(_connectionString, sqlOptions =>` → registra el proveedor de SQL Server con delegado. Línea 79: `sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);` → habilita los reintentos automáticos. Línea 80: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera de comandos. Línea 82: `.LogTo()` → habilita el logging. Línea 83: `Console.WriteLine,` → destino del logging. Línea 84: `new[] { "Microsoft.EntityFrameworkCore.Database.Command" },` → categorías. Línea 85: `LogLevel.Information)` → nivel mínimo. Línea 86: `.EnableSensitiveDataLogging()` → muestra los valores de los parámetros. Línea 87: `.EnableDetailedErrors()` → muestra información detallada en los errores. Línea 88: `break;` → fin del caso. Línea 93: `public static class AceriaDbContextFactory` → fábrica manual. Línea 95: `private static string? _connectionString;` → campo estático para la cadena. Línea 96: `private static string? _provider;` → campo estático para el proveedor. Línea 98: `public static void Initialize(string connectionString, string provider)` → método que inicializa la fábrica. Línea 100: `_connectionString = connectionString;` → asigna la cadena al campo. Línea 101: `_provider = provider;` → asigna el proveedor al campo. Línea 104: `public static AceriaDbContext Create()` → método que crea el contexto. Línea 106: `if (_connectionString is null || _provider is null)` → comprueba si la fábrica ha sido inicializada. Línea 108: `throw new InvalidOperationException("La fábrica no ha sido inicializada. Llama a Initialize primero.");` → lanza una excepción si no se ha inicializado. Línea 110: `return new AceriaDbContext(_connectionString, _provider);` → devuelve una instancia nueva. Línea 114: `public class Program` → clase principal. Línea 116: `public static void Main()` → punto de entrada. Línea 118: `var configuration = new ConfigurationBuilder()` → crea el constructor de configuración. Línea 119: `.SetBasePath(Directory.GetCurrentDirectory())` → establece el directorio base. Línea 120: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)` → añade el archivo JSON. Línea 121: `.AddEnvironmentVariables()` → añade las variables de entorno. Línea 122: `.Build();` → construye la configuración. Línea 124: `var provider = configuration["Database:Provider"] ?? "SqlServer";` → lee el proveedor configurado, con valor por defecto SqlServer. Línea 126: `var connectionString = provider == "Sqlite"` → comprueba si el proveedor es SQLite. Línea 127: `? "Data Source=aceria.db"` → cadena de conexión para SQLite. Línea 128: `: configuration.GetConnectionString("AceriaDB")` → cadena de conexión para SQL Server. Línea 129: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → lanza una excepción si no se encuentra. Línea 131: `AceriaDbContextFactory.Initialize(connectionString, provider);` → inicializa la fábrica. Línea 133: `Console.WriteLine($"Proveedor configurado: {provider}");` → muestra el proveedor. Línea 134: `Console.WriteLine($"Cadena de conexión: {connectionString}");` → muestra la cadena. Línea 136: `using (var context = AceriaDbContextFactory.Create())` → unidad de trabajo para recrear la base. Línea 138: `context.Database.EnsureDeleted();` → elimina la base. Línea 139: `context.Database.EnsureCreated();` →

crea la base. Línea 142: `InsertarOrden("OF-001", "Constructora del Norte");` → inserta la primera orden. Línea 143: `InsertarOrden("OF-002", "Constructora del Sur");` → inserta la segunda orden. Línea 144: `ListarOrdenes();` → lista las órdenes. Línea 146: `MostrarInformacionDelProveedor();` → muestra información del proveedor. Línea 149: `public static void InsertarOrden(string numero, string cliente)` → método de inserción. Línea 151: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 152: `var orden = new OrdenFabricacion` → crea la entidad. Línea 158: `context.OrdenesFabricacion.Add(orden);` → registra la entidad. Línea 159: `context.SaveChanges();` → ejecuta el INSERT. Línea 162: `public static void ListarOrdenes()` → método de listado. Línea 164: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 165: `var ordenes = context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → consulta ordenada. Línea 166: `Console.WriteLine("--- Órdenes ---");` → separador. Línea 167: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 169: `Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");` → muestra los datos. Línea 173: `public static void MostrarInformacionDelProveedor()` → método que muestra información del proveedor. Línea 175: `using var context = AceriaDbContextFactory.Create();` → unidad de trabajo. Línea 176: `Console.WriteLine($"Proveedor activo: {context.Database.ProviderName}");` → muestra el nombre del proveedor. Línea 177: `Console.WriteLine($"Puede conectar: {context.Database.CanConnect()}");` → comprueba si puede conectar.

Error común: si se cambia el proveedor a SQLite pero no se regeneran las migraciones, las migraciones generadas para SQL Server no se pueden aplicar. Se debe eliminar la carpeta Migrations y regenerarla con el nuevo proveedor.

Paso 5: Ejecutar el checkpoint operativo con SQL Server

Conservar el `Program.cs` heredado de 1.10, sin aplicar el bloque multi-proveedor del paso anterior.

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: el programa muestra Proveedor configurado: `SqlServer`, inserta las órdenes y las lista. El logging muestra las sentencias SQL generadas para SQL Server.

Paso 6: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Cambiar a SQLite

Modificar el archivo `appsettings.json` para cambiar el proveedor:

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\\
\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;MultipleActiveResultSets=true;Connect
Timeout=30;"
  },
  "Database": {
    "Provider": "Sqlite"
  }
}
```

Línea 6: `"Provider": "Sqlite"` → cambia el proveedor a SQLite.

Paso 7: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Ejecutar el proyecto con SQLite

```
dotnet run
```

Resultado esperado: el programa muestra Proveedor configurado: `Sqlite`, inserta las órdenes y las lista. Se crea un archivo `aceria.db` en la carpeta del proyecto. El logging muestra las sentencias SQL generadas para SQLite.

Paso 8: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Comparar el SQL generado

Ejecutar el proyecto con SQL Server y con SQLite y comparar las sentencias SQL en el logging. Se observan diferencias en la sintaxis de creación de tablas, en la generación de claves y en la paginación.

Resultado esperado: las sentencias SQL son distintas según el proveedor. En SQL Server, las claves se generan con IDENTITY. En SQLite, con AUTOINCREMENT.

Paso 9: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Volver a SQL Server

Restaurar el archivo appsettings.json para usar SQL Server:

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\\
\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;MultipleActiveResultSets=true;Connect
Timeout=30;"
  },
  "Database": {
    "Provider": "SqlServer"
  }
}
```

Línea 6: "Provider": "SqlServer" → vuelve a SQL Server.

Paso 10: VARIANTE CONCEPTUAL — NO EJECUTAR EN M01 — Diagnosticar un error común

Modificar el archivo appsettings.json para usar un proveedor inexistente:

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"
  },
  "Database": {
    "Provider": "Oracle"
  }
}
```

Resultado esperado: el programa usa el caso por defecto del switch, que es SQL Server. No se lanza ninguna excepción porque el default captura el valor no reconocido. El programa funciona con SQL Server.

Solución: si se quiere que un proveedor no reconocido lance una excepción, se debe modificar el switch para que el default lance una excepción en lugar de usar SQL Server.

Paso 11: Verificar la base de datos operativa

Abrir el Explorador de objetos de SQL Server en Visual Studio y comprobar que `AceriaDB` existe en `(localdb)\\MSSQLLocalDB`. No debe crearse `aceria.db` en M01.

Resultado esperado: existe `AceriaDB` en SQL Server LocalDB y no existe ninguna base SQLite creada por esta práctica.

Errores comunes del ejercicio completo

Error Causa Solución UseSqlite no disponible Falta el paquete de SQLite Instalar Microsoft.EntityFrameworkCore.Sqlite Migraciones no válidas Se cambiaron de proveedor Regenerar las migraciones Tipos de datos incompatibles El modelo usa tipos específicos de un motor Revisar el modelo y usar tipos portables Cadena de conexión incorrecta El proveedor no coincide con la cadena Verificar la

sección Database:Provider Archivo SQLite no encontrado Ruta incorrecta Usar Data Source=aceria.db o una ruta absoluta Error de conexión SQL Server LocalDB no responde Reiniciar Visual Studio o esperar

Retos de lectura: SQLite en memoria — NO EJECUTAR EN M01

El reto original se conserva como variante pedagógica para comprender el concepto, pero no se ejecuta en M01. La ejecución con SQLite en memoria queda fuera de este módulo; el proyecto operativo debe seguir usando exclusivamente SQL Server LocalDB.

Solución paso a paso

Paso 1: Instalar el paquete de SQLite si no está instalado:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
```

Paso 2: Añadir el método ProbarConSqliteEnMemoria:

```
public static void ProbarConSqliteEnMemoria()
{
    var connection = new Microsoft.Data.Sqlite.SqliteConnection("Data Source=:memory:");
    connection.Open();

    var options = new DbContextOptionsBuilder<AceriaDbContext>()
        .UseSqlite(connection)
        .Options;

    using (var context = new AceriaDbContext("Data Source=:memory:", "Sqlite"))
    {
        context.Database.EnsureCreated();

        var orden = new OrdenFabricacion { NumeroOrden = "TEST-001", Cliente = "Cliente de Prueba",
        FechaCreacion = DateTime.Now };
        context.OrdenesFabricacion.Add(orden);
        context.SaveChanges();

        var ordenes = context.OrdenesFabricacion.ToList();
        Console.WriteLine($"Órdenes en memoria: {ordenes.Count}");
        Console.WriteLine($"Primera orden: {ordenes[0].NumeroOrden}");
    }

    connection.Close();
}
```

Línea 1: public static void ProbarConSqliteEnMemoria() → declara el método. Línea 3: var connection = new Microsoft.Data.Sqlite.SqliteConnection("Data Source=:memory:"); → crea una conexión SQLite en memoria. Línea 4: connection.Open(); → abre la conexión. Es necesario mantenerla abierta para que la base de datos en memoria exista. Línea 6: var options = new DbContextOptionsBuilder() → crea el constructor de opciones. Línea 7: .UseSqlite(connection) → registra el proveedor de SQLite con la conexión en memoria. Línea 8: .Options; → obtiene las opciones. Línea 10: using (var context = new AceriaDbContext("Data Source=:memory:", "Sqlite")) → crea el DbContext con el proveedor SQLite. Línea 12: context.Database.EnsureCreated(); → crea el esquema en la base en memoria. Línea 14: var orden = new OrdenFabricacion { ... }; → crea una orden de prueba. Línea 15: context.OrdenesFabricacion.Add(orden); → registra la orden. Línea 16: context.SaveChanges(); → inserta la orden. Línea 18: var ordenes = context.OrdenesFabricacion.ToList(); → consulta las órdenes. Línea 19: Console.WriteLine(\$"Órdenes en memoria: {ordenes.Count}"); → muestra el número de órdenes. Línea 20: Console.WriteLine(\$"Primera orden: {ordenes[0].NumeroOrden}"); → muestra el número de la primera orden. Línea 23: connection.Close(); → cierra la conexión y libera la base en memoria.

Paso 3: Llamar al método desde Main:

```
ProbarConSqliteEnMemoria();
```

Paso 4: Ejecutar dotnet run y verificar que la prueba se ejecuta sin afectar a la base de datos real.

Analogía final

Los proveedores de datos son como los distintos tipos de hornos que puede tener una acería. Un horno eléctrico, un horno de gas y un horno de inducción producen acero, pero cada uno tiene sus propias características, sus propios mandos y su propio mantenimiento. El operario que trabaja con el acero no necesita saber cómo funciona cada horno por dentro: solo necesita saber qué horno usar en cada momento y cómo configurarlo. SQL Server es como un horno industrial de alta capacidad, pensado para producción a gran escala. SQLite es como un horno de sobremesa, ligero, portátil y sin instalación, ideal para pruebas y para producción pequeña. PostgreSQL es como un horno open source que cualquiera puede instalar y modificar. EF Core permite cambiar de horno cambiando una sola línea de configuración: el resto del proceso de fabricación sigue igual. Esa es la ventaja de trabajar con proveedores: el código de la aplicación no cambia, solo cambia el motor que ejecuta las operaciones.

Resultado esperado

Al final del ejercicio, deberías haber:

Identificado el paquete del proveedor de SQLite como alternativa conceptual, sin instalarlo en el proyecto operativo.

Analizado conceptualmente cómo podría alternarse de proveedor, manteniendo AceriaData configurado sólo con SQL Server.

Ejecutado el proyecto únicamente con SQL Server LocalDB.

Comparado conceptualmente el papel de distintos proveedores sin cambiar el proveedor operativo.

Verificado `AceriaDB` en SQL Server LocalDB y la ausencia de una base SQLite creada por M01.

Diagnosticado errores comunes de configuración de proveedores.

Conservado el ejemplo de SQLite en memoria como lectura no ejecutable.

Conclusión y enlace al siguiente punto

En este punto se han estudiado los proveedores de datos de EF Core: SQL Server, SQLite y PostgreSQL. AceriaData se ha mantenido operativo exclusivamente sobre SQL Server LocalDB; SQLite y PostgreSQL se han conservado como referencias pedagógicas no ejecutables. En el siguiente punto se estudiará la integración de EF Core en aplicaciones .NET mediante inyección de dependencias y AddDbContext.

Punto 1.12 - Integración de EF Core en aplicaciones .NET: inyección de dependencias y AddDbContext

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core. Proyecto: Se refactoriza el proyecto AceriaData para usar inyección de dependencias con Microsoft.Extensions.DependencyInjection y AddDbContext, manteniendo SQL Server LocalDB como proveedor.

Ejercicio: Refactorizar el proyecto AceriaData para usar inyección de dependencias con Microsoft.Extensions.DependencyInjection. Registrar el DbContext y un repositorio, resolver los servicios desde el contenedor y ejecutar las operaciones contra SQL Server LocalDB.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Instalar los paquetes de inyección de dependencias

```
dotnet add package Microsoft.Extensions.DependencyInjection --version 8.0.1
dotnet add package Microsoft.Extensions.Hosting --version 8.0.0
```

dotnet add package Microsoft.Extensions.DependencyInjection --version 8.0.1 → añade el paquete que contiene el contenedor de servicios. dotnet add package Microsoft.Extensions.Hosting --version 8.0.0 → añade el paquete que contiene el host genérico, que simplifica la configuración del contenedor.

Error común: si se olvida instalar estos paquetes, las clases ServiceCollection, BuildServiceProvider y CreateScope no están disponibles y el código no compila.

Paso 3: Sustituir el contenido de Program.cs

Abrir Program.cs y sustituir su contenido por el siguiente código:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
```

```

}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;

    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    void Guardar();
}

public class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;

    public OrdenRepositorio(AceriaDbContext context)
    {
        _context = context;
    }

    public List<OrdenFabricacion> ObtenerTodas()
    {
        return _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    }

    public OrdenFabricacion? ObtenerPorId(int id)
    {
        return _context.OrdenesFabricacion.Find(id);
    }

    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden)
    {
        return _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    }

    public void Agregar(OrdenFabricacion orden)
    {
        _context.OrdenesFabricacion.Add(orden);
    }

    public void Eliminar(OrdenFabricacion orden)
    {
        _context.OrdenesFabricacion.Remove(orden);
    }

    public void Guardar()
    {

```

```

        _context.SaveChanges();
    }
}

public class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();

        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sqlOptions =>
                    {
                        sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                        sqlOptions.CommandTimeout(60);
                    })
                .LogTo(
                    Console.WriteLine,
                    new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableSensitiveDataLogging()
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();

        var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using (var scope = provider.CreateScope())
        {
            var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
            context.Database.EnsureDeleted();
            context.Database.Migrate();
        }

        using (var scope = provider.CreateScope())
        {
            var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

            repositorio.Agregar(new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora
del Norte", FechaCreacion = DateTime.Now });
            repositorio.Agregar(new OrdenFabricacion { NumeroOrden = "OF-002", Cliente = "Constructora
del Sur", FechaCreacion = DateTime.Now });
            repositorio.Agregar(new OrdenFabricacion { NumeroOrden = "OF-003", Cliente = "Constructora
del Este", FechaCreacion = DateTime.Now });
            repositorio.Guardar();
        }

        using (var scope = provider.CreateScope())
        {
            var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

            var ordenes = repositorio.ObtenerTodas();
            Console.WriteLine("--- Órdenes ---");
            foreach (var orden in ordenes)
            {
                Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente:
{orden.Cliente}");
            }
        }
    }
}

```

```

using (var scope = provider.CreateScope())
{
    var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

    var orden = repositorio.ObtenerPorNumero("OF-002");
    if (orden is not null)
    {
        orden.Cliente = "Constructora del Oeste";
        repositorio.Guardar();
        Console.WriteLine($"Orden {orden.NumeroOrden} actualizada a {orden.Cliente}");
    }
}

using (var scope = provider.CreateScope())
{
    var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

    var orden = repositorio.ObtenerPorNumero("OF-003");
    if (orden is not null)
    {
        repositorio.Eliminar(orden);
        repositorio.Guardar();
        Console.WriteLine($"Orden {orden.NumeroOrden} eliminada");
    }
}

using (var scope = provider.CreateScope())
{
    var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

    var ordenes = repositorio.ObtenerTodas();
    Console.WriteLine("--- Órdenes finales ---");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente:
{orden.Cliente}");
    }
}
}

```

Línea 1: using Microsoft.EntityFrameworkCore; → importa el espacio de nombres de EF Core. Línea 2: using Microsoft.Extensions.Configuration; → importa el espacio de nombres de configuración. Línea 3: using Microsoft.Extensions.DependencyInjection; → importa el espacio de nombres del contenedor de servicios. Línea 4: using Microsoft.Extensions.Logging; → importa el espacio de nombres de logging. Línea 6: namespace AceriaData.ConsoleApp; → declara el espacio de nombres. Línea 8: public class OrdenFabricacion → entidad de orden. Línea 10: public int Id { get; set; } → clave primaria. Línea 11: public string NumeroOrden { get; set; } = string.Empty; → número de orden. Línea 12: public string Cliente { get; set; } = string.Empty; → cliente. Línea 13: public DateTime FechaCreacion { get; set; } → fecha de creación. Línea 14: public List Planchas { get; set; } = new(); → colección de planchas. Línea 17: public class PlanchaAcero → entidad de plancha. Línea 19: public int Id { get; set; } → clave primaria. Línea 20: public int OrdenId { get; set; } → clave foránea. Línea 21: public double Espesor { get; set; } → espesor. Línea 22: public double Ancho { get; set; } → ancho. Línea 23: public double Largo { get; set; } → largo. Línea 24: public OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación. Línea 27: public class Aleacion → entidad de aleación. Línea 29: public int Id { get; set; } → clave primaria. Línea 30: public string Nombre { get; set; } = string.Empty; → nombre. Línea 31: public double PorcentajeCarbono { get; set; } → porcentaje de carbono. Línea 32: public double PorcentajeManganeso { get; set; } → porcentaje de manganeso. Línea 35: public class EstadoOrden → entidad de estado. Línea 37: public int Id { get; set; } → clave primaria. Línea 38: public string Nombre { get; set; } = string.Empty; → nombre del estado. Línea 39: public string Descripcion { get; set; } = string.Empty; → descripción. Línea 42: public class AceriaDbContext : DbContext → declara el DbContext. Línea 44: public DbSet OrdenesFabricacion { get; set; } = null!; → DbSet de órdenes. Línea 45: public DbSet PlanchasAcero { get; set; } = null!; → DbSet de planchas. Línea 46: public DbSet Aleaciones { get; set; } = null!; → DbSet de aleaciones. Línea 47: public DbSet EstadosOrden { get; set; } = null!; → DbSet de estados. Línea 49: public AceriaDbContext(DbContextOptions options) : base(options) { } → constructor

que recibe las opciones y las pasa a la clase base. Línea 53: `public interface IOOrdenRepositorio` → declara la interfaz del repositorio. Línea 55: `List ObtenerTodas();` → método que devuelve todas las órdenes. Línea 56: `OrdenFabricacion? ObtenerPorId(int id);` → método que devuelve una orden por Id. Línea 57: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → método que devuelve una orden por número. Línea 58: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden. Línea 59: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden. Línea 60: `void Guardar();` → método que guarda los cambios. Línea 63: `public class OrdenRepositorio : IOOrdenRepositorio` → declara la implementación del repositorio. Línea 65: `private readonly AceriaDbContext _context;` → campo que almacena el DbContext. Línea 67: `public OrdenRepositorio(AceriaDbContext context)` → constructor que recibe el DbContext. Línea 69: `_context = context;` → asigna el parámetro al campo. Línea 72: `public List ObtenerTodas()` → método que devuelve todas las órdenes. Línea 74: `return _context OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → consulta ordenada. Línea 77: `public OrdenFabricacion? ObtenerPorId(int id)` → método que devuelve una orden por Id. Línea 79: `return _context OrdenesFabricacion.Find(id);` → busca por clave primaria. Línea 82: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden)` → método que devuelve una orden por número. Línea 84: `return _context OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → busca por número. Línea 87: `public void Agregar(OrdenFabricacion orden)` → método que agrega una orden. Línea 89: `_context OrdenesFabricacion.Add(orden);` → registra la entidad. Línea 92: `public void Eliminar(OrdenFabricacion orden)` → método que elimina una orden. Línea 94: `_context OrdenesFabricacion.Remove(orden);` → marca la entidad para eliminar. Línea 97: `public void Guardar()` → método que guarda los cambios. Línea 99: `_context.SaveChanges();` → ejecuta las operaciones pendientes. Línea 103: `public class Program` → clase principal. Línea 105: `public static void Main()` → punto de entrada. Línea 107: `var configuration = new ConfigurationBuilder()` → crea el constructor de configuración. Línea 108: `.SetBasePath(Directory.GetCurrentDirectory())` → establece el directorio base. Línea 109: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)` → añade el archivo JSON. Línea 110: `.AddEnvironmentVariables()` → añade las variables de entorno. Línea 111: `.Build();` → construye la configuración. Línea 113: `var connectionString = configuration.GetConnectionString("AceriaDB")` → lee la cadena de conexión. Línea 114: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → lanza una excepción si no se encuentra. Línea 116: `var services = new ServiceCollection();` → crea la colección de servicios. Línea 118: `services.AddDbContext(options =>` → registra el DbContext. Línea 119: `options` → objeto de configuración. Línea 120: `.UseSqlServer(connectionString, sqlOptions =>` → registra el proveedor de SQL Server. Línea 121: `sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);` → habilita los reintentos automáticos. Línea 122: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera de comandos. Línea 124: `.LogTo()` → habilita el logging. Línea 125: `Console.WriteLine,` → destino del logging. Línea 126: `new[] { "Microsoft.EntityFrameworkCore.Database.Command" }` → categorías. Línea 127: `LogLevel.Information)` → nivel mínimo. Línea 128: `.EnableSensitiveDataLogging()` → muestra los valores de los parámetros. Línea 129: `.EnableDetailedErrors();` → muestra información detallada en los errores. Línea 131: `services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();` → registra el repositorio con ciclo de vida Scoped. Línea 133: `var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateScopes = true, ValidateOnBuild = true });` → construye el proveedor de servicios. Línea 135: `using (var scope = provider.CreateScope())` → crea un ámbito para recrear la base de datos. Línea 137: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el DbContext. Línea 138: `context.Database.EnsureDeleted();` → elimina la base de datos. Línea 139: `context.Database.Migrate();` → crea la base de datos. Línea 142: `using (var scope = provider.CreateScope())` → crea un ámbito para insertar órdenes. Línea 144: `var repositorio = scope.ServiceProvider.GetRequiredService();` → resuelve el repositorio. Línea 146: `repositorio.Agregar(new OrdenFabricacion { ... });` → agrega la primera orden. Línea 147: `repositorio.Agregar(new OrdenFabricacion { ... });` → agrega la segunda orden. Línea 148: `repositorio.Agregar(new OrdenFabricacion { ... });` → agrega la tercera orden. Línea 149: `repositorio.Guardar();` → guarda los cambios. Línea 152: `using (var scope = provider.CreateScope())` → crea un ámbito para listar órdenes. Línea 154: `var repositorio = scope.ServiceProvider.GetRequiredService();` → resuelve el repositorio. Línea 156: `var ordenes = repositorio.ObtenerTodas();` → consulta todas las órdenes. Línea 157: `Console.WriteLine("--- Órdenes ---");` → separador. Línea 158: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 160: `Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");` → muestra los datos. Línea 164: `using (var scope = provider.CreateScope())` → crea un ámbito para

actualizar una orden. Línea 166: `var repositorio = scope.ServiceProvider.GetRequiredService();` → resuelve el repositorio. Línea 168: `var orden = repositorio.ObtenerPorNumero("OF-002");` → busca la orden. Línea 169: `if (orden is not null)` → comprueba si existe. Línea 171: `orden.Cliente = "Constructora del Oeste";` → modifica el cliente. Línea 172: `repositorio.Guardar();` → guarda los cambios. Línea 173: `Console.WriteLine($"Orden {orden.NumeroOrden} actualizada a {orden.Cliente}");` → muestra el resultado. Línea 177: `using (var scope = provider.CreateScope())` → crea un ámbito para eliminar una orden. Línea 179: `var repositorio = scope.ServiceProvider.GetRequiredService();` → resuelve el repositorio. Línea 181: `var orden = repositorio.ObtenerPorNumero("OF-003");` → busca la orden. Línea 182: `if (orden is not null)` → comprueba si existe. Línea 184: `repositorio.Eliminar(orden);` → elimina la orden. Línea 185: `repositorio.Guardar();` → guarda los cambios. Línea 186: `Console.WriteLine($"Orden {orden.NumeroOrden} eliminada");` → muestra el resultado. Línea 190: `using (var scope = provider.CreateScope())` → crea un ámbito para listar las órdenes finales. Línea 192: `var repositorio = scope.ServiceProvider.GetRequiredService();` → resuelve el repositorio. Línea 194: `var ordenes = repositorio.ObtenerTodas();` → consulta todas las órdenes. Línea 195: `Console.WriteLine("--- Órdenes finales ---");` → separador. Línea 196: `foreach (var orden in ordenes)` → itera sobre las órdenes. Línea 198: `Console.WriteLine($"Id: {orden.Id} | Número: {orden.NumeroOrden} | Cliente: {orden.Cliente}");` → muestra los datos.

Error común: si se resuelve el `DbContext` desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException` indicando que no se puede resolver un servicio Scoped desde el proveedor raíz. Siempre se debe resolver desde un ámbito.

Soporte de migraciones en tiempo de diseño

Tras cambiar `AceriaDbContext` para recibir `DbContextOptions<AceriaDbContext>` por constructor, añadir una factoría de diseño para que `dotnet ef` pueda crear el contexto sin ejecutar la lógica de negocio de `Main`:

```
using Microsoft.EntityFrameworkCore.Design;

public sealed class AceriaDesignTimeDbContextFactory : IDesignTimeDbContextFactory<AceriaDbContext>
{
    public AceriaDbContext CreateDbContext(string[] args)
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer("Server=(localdb)\\
\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;TrustServerCertificate=True;")
            .Options;

        return new AceriaDbContext(options);
    }
}
```

Esta factoría no sustituye a la inyección de dependencias en ejecución; sólo proporciona a las herramientas de diseño una forma determinista de construir el contexto.

Paso 4: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: el programa inserta tres órdenes, las lista, actualiza la orden OF-002 y elimina la orden OF-003. Al final, se listan las órdenes restantes con los cambios aplicados.

Paso 5: Observar la separación de ámbitos

Cada bloque `using (var scope = provider.CreateScope())` crea un ámbito nuevo. Cada ámbito tiene su propia instancia del `DbContext` y del repositorio. Al salir del bloque, el ámbito se libera y con él el `DbContext`.

Modificar el método Main para imprimir el identificador de la instancia del DbContext en cada ámbito:

```
using (var scope = provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    Console.WriteLine($"DbContext en ámbito 1: {context.GetHashCode()}");
}

using (var scope = provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    Console.WriteLine($"DbContext en ámbito 2: {context.GetHashCode()}");
}
```

Línea 135: using (var scope = provider.CreateScope()) → primer ámbito. Línea 137: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext. Línea 138: Console.WriteLine(\$"DbContext en ámbito 1: {context.GetHashCode()}"); → muestra el identificador de la instancia. Línea 141: using (var scope = provider.CreateScope()) → segundo ámbito. Línea 143: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext. Línea 144: Console.WriteLine(\$"DbContext en ámbito 2: {context.GetHashCode()}"); → muestra el identificador de la instancia.

Resultado esperado: los identificadores son distintos, confirmando que cada ámbito tiene su propia instancia del DbContext.

Paso 6: Ejecutar el proyecto

```
dotnet run
```

Resultado esperado: aparecen dos identificadores distintos, confirmando la separación de ámbitos.

Paso 7: Diagnosticar un error común

Modificar el método Main para resolver el DbContext desde el proveedor raíz en lugar de desde un ámbito:

```
var context = provider.GetRequiredService<AceriaDbContext>();
```

Resultado esperado: se lanza una excepción InvalidOperationException indicando que no se puede resolver un servicio Scoped desde el proveedor raíz.

Solución: resolver siempre desde un ámbito creado con CreateScope.

Paso 8: Verificar los cambios en la base de datos

Abrir Visual Studio. En Ver → Explorador de objetos de SQL Server. Expandir (localdb)\MSSQLLocalDB → Bases de datos → AceriaDB → Tablas → dbo.OrdenesFabricacion. Clic derecho → Ver datos. Comprobar que las órdenes OF-001 y OF-002 aparecen con los clientes actualizados, y que OF-003 ha sido eliminada.

Resultado esperado: la tabla contiene OF-001 y OF-002. OF-003 no aparece.

Errores comunes del ejercicio completo

Error	Causa	Solución
Servicio Scoped desde proveedor raíz	Se resolvió desde provider en lugar de scope	Usar CreateScope y resolver desde el ámbito
DbContext no registrado	Falta AddDbContext	Añadir el registro en la colección de servicios

Error	Causa	Solución
Repositorio no registrado	Falta AddScoped	Añadir el registro con su interfaz
Ciclo de vida incorrecto	Se registró como Singleton	Usar Scoped para el DbContext y los repositorios
Cadena de conexión no encontrada	Falta appsettings.json	Verificar el archivo y el .csproj
AddDbContext no disponible	Falta el paquete	Instalar Microsoft.EntityFrameworkCore

■ Reto resuelto: Añadir un servicio de negocio que use el repositorio

Reto: Crear un servicio ServicioOrdenes que use el repositorio para calcular el número total de planchas de una orden y devolver un resumen. Registrar el servicio en el contenedor y usarlo desde el Main.

■ Solución paso a paso

■ Paso 1: Añadir la interfaz y la implementación del servicio:

```
public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;

    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null)
        {
            return $"Orden {ordenId} no encontrada";
        }

        var totalPlanchas = _context.PlanchasAcero.Count(p => p.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}
```

Línea 1: public interface IServicioOrdenes → declara la interfaz. Línea 3: string ObtenerResumen(int ordenId); → declara el método. Línea 6: public class ServicioOrdenes : IServicioOrdenes → declara la implementación. Línea 8: private readonly IOrdenRepositorio _repositorio; → campo del repositorio. Línea 9: private readonly AceriaDbContext _context; → campo del DbContext. Línea 11: public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context) → constructor que recibe las dependencias. Línea 13: _repositorio = repositorio; → asigna el repositorio. Línea 14: _context = context; → asigna el DbContext. Línea 17: public string ObtenerResumen(int ordenId) → declara el método. Línea 19: var orden = _repositorio.ObtenerPorId(ordenId); → busca la orden. Línea 20: if (orden is null) → comprueba si existe. Línea 22: return \$"Orden {ordenId} no encontrada"; → devuelve un mensaje si no existe. Línea 25: var totalPlanchas = _context.PlanchasAcero.Count(p => p.OrdenId == ordenId); → cuenta las planchas. Línea 26: return \$"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}"; → devuelve el resumen.

Paso 2: Registrar el servicio en el contenedor:

```
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();
```

Línea 1: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra el servicio con ciclo de vida Scoped.

Paso 3: Usar el servicio desde Main:

```
using (var scope = provider.CreateScope())
{
    var servicio = scope.ServiceProvider.GetRequiredService<IServicioOrdenes>();
    Console.WriteLine(servicio.ObtenerResumen(1));
    Console.WriteLine(servicio.ObtenerResumen(2));
    Console.WriteLine(servicio.ObtenerResumen(999));
}
```

Línea 1: `using (var scope = provider.CreateScope())` → crea un ámbito. Línea 3: `var servicio = scope.ServiceProvider.GetRequiredService();` → resuelve el servicio. Línea 4: `Console.WriteLine(servicio.ObtenerResumen(1));` → muestra el resumen de la orden 1. Línea 5: `Console.WriteLine(servicio.ObtenerResumen(2));` → muestra el resumen de la orden 2. Línea 6: `Console.WriteLine(servicio.ObtenerResumen(999));` → muestra el mensaje de orden no encontrada.

Paso 4: Ejecutar dotnet run y verificar que el servicio devuelve los resúmenes correctos.

Analogía final

La inyección de dependencias es como el sistema de asignación de tareas de una acería. Cada operario no va a buscar sus herramientas al almacén: el jefe de planta le entrega las herramientas que necesita al inicio del turno. El operario no sabe quién fabricó la herramienta ni cómo se almacena: solo la usa. El contenedor de servicios es el jefe de planta que conoce todas las herramientas disponibles y las asigna a cada operario según sus necesidades. Los ciclos de vida son como los turnos: un Singleton es una herramienta que se usa durante toda la vida de la planta, un Scoped es una herramienta que se asigna por turno, y un Transient es una herramienta que se usa una sola vez y se descarta. El DbContext es una herramienta de turno: se asigna al inicio, se usa durante el turno y se devuelve al final. El repositorio es el operario especializado que sabe usar esa herramienta. El servicio de negocio es el supervisor que coordina a varios operarios. Así funciona la inyección de dependencias: cada componente recibe lo que necesita, sin preocuparse por cómo se crea ni por cuánto dura.

Resultado esperado

Al final del ejercicio, deberías haber:

Instalado los paquetes de inyección de dependencias.

Refactorizado el proyecto AceriaData para usar el contenedor de servicios.

Registrado el DbContext con `AddDbContext`.

Registrado el repositorio con `AddScoped`.

Resuelto los servicios desde ámbitos con `CreateScope`.

Observado la separación de instancias entre ámbitos.

Diagnosticado errores comunes de resolución de servicios.

Añadido un servicio de negocio que usa el repositorio y el DbContext.

Conclusión y enlace al siguiente punto

En este punto se ha estudiado la integración de EF Core en aplicaciones .NET mediante inyección de dependencias. Se ha refactorizado el proyecto AceriaData para usar el contenedor de servicios, se ha registrado el DbContext con AddDbContext, se ha creado un repositorio y se ha resuelto todo desde ámbitos. Con este punto se cierra el Módulo 1, que ha cubierto los fundamentos de EF Core: qué es un ORM, la arquitectura general, los componentes principales, el DbContext, su ciclo de vida, los DbSet, las operaciones básicas, el Change Tracker, la gestión de entidades, SaveChanges, la configuración, los proveedores y la inyección de dependencias.

Resumen del estado del proyecto AceriaData al final del Módulo 1

Al final del Módulo 1, el proyecto AceriaData tiene:

Una solución con un proyecto de consola.

Las entidades OrdenFabricacion, PlanchaAcero, Aleacion y EstadoOrden.

Un DbContext configurado con SQL Server LocalDB.

Un archivo appsettings.json con la cadena de conexión.

SQL Server LocalDB como único proveedor operativo del proyecto en el Módulo 1.

Un repositorio de órdenes con operaciones CRUD básicas.

Un servicio de negocio que usa el repositorio y el DbContext.

Inyección de dependencias con el contenedor de servicios.

Configuración del logging con filtros por categoría y nivel.

Migraciones generadas y aplicadas.

En el Módulo 2 se profundizará en el modelado de datos: convenciones, relaciones, Data Annotations, Fluent API, claves, índices, restricciones, filtros globales, Soft Delete y la integración de EF Core en arquitecturas limpias y Arquitectura Hexagonal. El proyecto AceriaData evolucionará para incluir un modelo de datos completo y bien configurado, sentando las bases para los módulos posteriores de consultas, optimización y persistencia empresarial.